

open_ai.py

"""Aggregation driver / entry point for the interview modules (port of OpenAi.java).

Overview:

This file solves no problem of its own. It is the single entry point that imports every sibling interview module in this package and, from `main()`, runs each one's demo with representative inputs, printing the results. Run it directly (`python3 open_ai.py`) to exercise all modules at once; each module also has its own smaller `__main__` demo when run alone.

Modules exercised (one line each):

- `social_recommendations` - friend-of-friend follow recommendations.
- `plant_infection` - multi-source BFS ("Rotting Oranges").
- `ip_addressing` - IPv4/IPv6 addressing toolkit.
- `labeling_schedule` - human/model/task labeling schedules.
- `chat_event_counter` - sliding 15-minute chat event window.
- `gpu_credit_manager` - GPU credits with expiration.
- `persistent_kv_store` - binary-safe persistent KV store.
- `in_memory_table` - in-memory SQL-like single table.
- `credit_ledger` - GPU credit ledger with out-of-order timestamps.
- `map_serializer` - length-prefixed, binary-safe map serialization.
- `sharded_kv_store` - persistent sharded KV store with replay restore.
- `gpu_credit_ledger` - idempotent GPU credit ledger (non-negative balances).

Output is Python-native (`True/False`, quoted strings in lists, dict/bytes repr), so it differs cosmetically from the Java driver's output (`true/false`, unquoted).

```
from chat_event_counter import ChatEventCounter
from credit_ledger import CreditLedger
from gpu_credit_ledger import GpuCreditLedger
from gpu_credit_manager import GpuCreditManager
from in_memory_table import InMemoryTable
from ip_addressing import (
    parse_ipv4, adjacent_ipv4, cidr_range, ip_in_cidr, adjacent_ipv6,
)
from labeling_schedule import any_schedule, prefix_balanced_schedule
from map_serializer import solution as map_solution
from persistent_kv_store import PersistentKvStore
from plant_infection import infection_time
from sharded_kv_store import ShardedKvStore
from social_recommendations import SocialRecommendations

def main():
    # --- Module 1: Social Follow Recommendations ---
    ex1 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
        ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 3],
    ]
    print(SocialRecommendations.process_queries(ex1)) # [[4, 5, 6]]

    ex2 = [
        ["follow", 1, 1], ["follow", 1, 2], ["follow", 1, 2], ["follow", 2, 1],
        ["follow", 2, 3], ["recommend", 1, 5], ["unfollow", 1, 4],
        ["unfollow", 1, 2], ["recommend", 1, 5],
    ]
```

```

print(SocialRecommendations.process_queries(ex2)) # [[3], []]

# --- Module 2: Plant Infection Time (multi-source BFS) ---
print(infection_time([[2, 1, 1], [1, 1, 0], [0, 1, 1]])) # 4
print(infection_time([[2, 1, -1], [-1, 1, 1], [1, -1, 1]])) # -1
# -> 0 (no healthy plants)
print(infection_time([[0, 2, 0]]))

# --- Module 3: IPv4 / IPv6 addressing toolkit ---
# Part 1: validate & parse IPv4
print(parse_ipv4("192.168.0.1")) # [192, 168, 0, 1]
print(parse_ipv4("0.0.0.0")) # [0, 0, 0, 0]
# Part 2: adjacent IPv4
print(adjacent_ipv4("192.168.0.1", "up")) # 192.168.0.2
print(adjacent_ipv4("192.168.0.255", "up")) # 192.168.1.0
# Part 3: CIDR first & last
print(cidr_range("192.168.0.0/24")) # ['192.168.0.0', '192.168.0.255']
print(cidr_range("192.168.5.123/24")) # ['192.168.5.0', '192.168.5.255']
# Part 4: membership
print(ip_in_cidr("192.168.0.10", "192.168.0.0/24")) # 1
print(ip_in_cidr("192.168.0.200", "192.168.0.99/24")) # 1
# Part 5: adjacent IPv6 (period-separated)
print(adjacent_ipv6("ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffffa", "up"))
# ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffffb
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.ffff", "up"))
# 0000.0000.0000.0000.0000.0000.0001.0000

# --- Module 4: Labeling schedule generation ---
# Part 1: any valid schedule
print(any_schedule(3, 1, 2, 2))
# [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
#  ['t3', 'm1', 'h2']]
print(any_schedule(1, 1, 1, 1)) # [['t1', 'm1', 'h1']]
# Part 2: prefix-balanced schedule
print(prefix_balanced_schedule(3, 2, 4, 2))
# [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t1', 'm1', 'h3'], ['t1', 'm2', 'h4'],
#  ['t2', 'm2', 'h1'], ['t2', 'm1', 'h2'], ['t2', 'm2', 'h3'], ['t2', 'm1', 'h4']]
print(prefix_balanced_schedule(1, 1, 1, 1)) # [['t1', 'm1', 'h1']]

# --- Module 5: Chat Event Counter (15-minute sliding window) ---
chat1 = [
    ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
    ["processEvent", "u1", "c1", 1000], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat1)) # [3]
print(ChatEventCounter.simulate(["getCount", "u1", "c1"])) # [0]

# --- Module 6: GPU Credits with Expiration ---
gpu1 = [
    ["balance", 5], ["add", "a", 10, 5, 5], ["balance", 7],
    ["charge", 4, 8], ["balance", 8], ["balance", 10],
]
print(GpuCreditManager.simulate(gpu1)) # [0, 10, True, 6, 0]

gpu2 = [
    ["add", "a", 5, 10, 10], ["add", "b", 7, 5, 10], ["charge", 5, 12],
    ["balance", 12], ["balance", 17], ["charge", 6, 9], ["balance", 17],
]
print(GpuCreditManager.simulate(gpu2)) # [True, 7, 5, False, 5]

```

```

# --- Module 7: Persistent In-Memory KV Store ---
by = PersistentKvStore.by
# Part 1: serialize/deserialize a snapshot (str values)
kv1 = [
    ["put", "a", "1"], ["get", "a"], ["serialize", "snap1"],
    ["put", "a", "2"], ["deserialize", "snap1"], ["get", "a"],
]
# -> ['1', '1']
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1)))
# Part 1: binary-safe with delimiter-like bytes in key/value
kv1b = [
    ["put", by("a|b"), by("v:1")], ["serialize", "p"],
    ["delete", by("a|b")],
    ["get", by("a|b")], ["deserialize", "p"], ["get", by("a|b")],
]
# -> [None, b'v:1']
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1b)))

# Part 2: snapshot split across size-bounded segments
kv2 = [
    ["put", by("a"), by("12345")], ["put", by("b"), by("67890")],
    ["serialize", "p"], ["segment_count", "p"], ["delete", by("a")],
    ["deserialize", "p"], ["get", by("a")], ["get", by("b")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part2(15, kv2)))
# [6, b'12345', b'67890']
kv2b = [
    ["put", by("aa"), by("xx")], ["put", by("bb"), by("yy")],
    ["serialize", "snap"], ["segment_count", "snap"],
    ["reorder", "snap", [2, 0, 1]], ["delete", by("aa")],
    ["deserialize", "snap"], ["get", by("aa")], ["get", by("bb")],
]
print(PersistentKvStore.py_repr(
    PersistentKvStore.simulate_part2(20, kv2b)))
# [3, b'xx', b'yy']

# Part 3: snapshot + append-only log with replay and compaction
kv3 = [
    ["put", "a", "1"], ["put", "b", "2"], ["restart"],
    ["get", "a"], ["get", "b"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3)))
# ['1', '2', (2, 2, 0)]
kv3b = [
    ["put", "x", "1"], ["status"], ["put", "y", "2"], ["status"],
    ["restart"], ["get", "x"], ["get", "y"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(2, kv3b)))
# [(1, 1, 0), (2, 0, 1), '1', '2', (2, 0, 1)]

# --- Module 8: In-memory SQL-like table ---
sql1 = [
    "SET r1 name bob", "SET r1 age 2", "SET r2 name bob", "SET r2 age 10",
    "GET r1 name", "SELECT name bob age",
]
print(InMemoryTable.process_commands(sql1)) # ['bob', 'r2 r1']

sql2 = ["GET missing name", "SELECT status active score"]

```

open_ai.py (continued)

```
print(InMemoryTable.process_commands(sql2)) # ['NULL', '']

# --- Module 9: Credit ledger with out-of-order timestamps ---
led1 = [
    ["GRANT", "alice", 10, 5], ["CHARGE", "alice", 4, 7],
    ["GET", "alice", 7],
    ["CHARGE", "alice", 10, 6], ["GET", "alice", 7],
]
print(CreditLedger.simulate(led1)) # [6, 0]

led2 = [
    ["GRANT", "alice", 5, 10], ["GRANT", "bob", 7, 1],
    ["CHARGE", "bob", 2, 3],
    ["GET", "bob", 2], ["GET", "bob", 3], ["GET", "alice", 9],
    ["GET", "alice", 10],
]
print(CreditLedger.simulate(led2)) # [7, 5, 0, 5]

# --- Module 10: Map serialization (length-prefixed, binary-safe) ---
print(map_solution("serialize", {})) # 0#
print(map_solution("deserialize", "0#")) # {}
sample_map = {"a": "hi", "b": ""}
encoded = map_solution("serialize", sample_map)
print(encoded) # 2#1#a2#hi1#b0#
print(map_solution("deserialize", encoded)) # {'a': 'hi', 'b': ''}

# --- Module 11: Persistent sharded KV store ---
shard1 = [
    ["put", "a", "1"], ["put", "b", "22"], ["get", "a"],
    ["put", "a", "333"],
    ["get", "a"], ["restore"], ["get", "b"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, shard1)))
# ([ '1', '333', '22'], [ 'P|a|1;', 'P|b|22;', 'P|a|333;'])
shard2 = [
    ["put", "ab", "x"], ["put", "c", "yz"], ["delete", "ab"],
    ["get", "ab"],
    ["restore"], ["get", "c"], ["get", "ab"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(20, shard2)))
# ([None, 'yz', None], [ 'P|ab|x;P|c|yz;D|ab;'])

# --- Module 12: Idempotent GPU credit ledger (non-negative balances) ---
led_init1 = [
    ["e1", "tenantA", 10], ["e2", "tenantA", -3], ["e3", "tenantB", 5],
]
led_ops1 = [
    ["get", "tenantA"], ["get", "tenantB"],
    ["apply", ["e4", "tenantA", 2]],
    ["apply", ["e5", "tenantB", -6]], ["get", "tenantB"],
    ["apply", ["e6", "tenantA", -4]], ["get", "tenantB"],
]
print(GpuCreditLedger.solution(led_init1, led_ops1))
# [7, 5, True, False, 5, True, 5]
led_ops2 = [
    ["get", "ghost"], ["apply", ["x1", "ghost", -3]], ["get", "ghost"],
    ["apply", ["x1", "ghost", -3]], ["apply", ["x2", "ghost", 4]],
    ["get", "ghost"],
]
]
```

open_ai.py (continued)

```
print(GpuCreditLedger.solution([], led_ops2))
# [0, False, 0, False, True, 4]

if __name__ == "__main__":
    main()
```

cell_spreadsheet.py

```
"""A small spreadsheet whose cells are literal ints or two-cell adders.
```

Overview:

Cells are stored by name. A cell is either a literal integer or a formula that adds the values of two other cells (referenced by name). Reading a cell resolves the whole chain of references beneath it. Results are cached for speed, and editing a cell surgically invalidates only the cells whose values could have changed.

Interface:

- `Cell.literal(value)` -> `Cell`: a cell holding the integer `value`.
- `Cell.formula(child1, child2)` -> `Cell`: a cell equal to `value(child1) + value(child2)`, where `child1/child2` are cell names.
- `Spreadsheet()` -> a new, empty sheet.
- `setCell(key, cell)` -> `None`: define or redefine the cell named `key`.
- `getCellValue(key)` -> `int`: the literal value, or the sum for a formula, resolved recursively through referenced cells.

Semantics and rules:

- Evaluation is memoized: repeated reads of an unchanged cell are $O(1)$.
- `setCell` invalidates the cached value of `key` and every cell that transitively depends on it, so later reads stay correct while untouched cells are not recomputed.
- Redefining a cell rewires its dependency edges before invalidating.

Errors (both raise `EvaluationError`):

- a formula references a cell name that was never set (unknown cell);
- the references form a cycle, e.g. `A -> B -> A`.

Example:

```
setCell("A", Cell.literal(10)); setCell("B", Cell.literal(5))
setCell("C", Cell.formula("A", "B")); getCellValue("C") -> 15
setCell("A", Cell.literal(20)); getCellValue("C") -> 25
"""
```

```
class EvaluationError(Exception):
```

```
    """Raised when a cell cannot be evaluated (unknown cell or a cycle)."""
```

```
class Cell:
```

```
    """A literal integer cell or a formula adding two referenced cells."""
```

```
    def __init__(self, value, child1, child2, is_literal):
        # Time: O(1), Space: O(1).
        self.value = value
        self.child1 = child1
        self.child2 = child2
        self.is_literal = is_literal
```

```
    @staticmethod
```

```
    def literal(value):
        """A cell holding the integer value directly."""
        # Time: O(1), Space: O(1).
        return Cell(value, None, None, True)
```

```
    @staticmethod
```

```
    def formula(child1, child2):
        """A cell whose value is value(child1) + value(child2)."""
```

cell_spreadsheet.py (continued)

```
# Time: O(1), Space: O(1).
return Cell(None, child1, child2, False)

# Approach (in plain terms):
# Keep every cell in a lookup table by name. Reading a cell is a little
# depth-first walk: a literal returns its number, a formula returns the sum
# of its two children (evaluated the same way). We carry a "currently
# visiting" set down the walk; if we reach a cell already on that path the
# formulas loop, so we stop and raise. To avoid recomputing, we remember each
# answer in a cache. When a cell is changed we clear the cache for it and for
# every cell that depends on it (found via a reverse "who uses me" map), so
# the next read recomputes exactly what changed and reuses the rest.
# Data structures used:
# - dict name -> Cell - the stored cell definitions.
# - dict name -> cached value - memoized results for O(1) repeat reads.
# - dict name -> set of dependents (reverse edges) - drives invalidation
#   when a cell is updated.
# - a 'visiting' set on the DFS path - detects reference cycles.
class Spreadsheet:

    def __init__(self):
        # Time: O(1), Space: O(1) - empty maps.
        self._cells = {}
        self._cache = {}
        self._dependents = {}

    def setCell(self, key, cell):
        """Define/redefine a cell, then invalidate whatever it affects."""
        # d = cells that transitively depend on key.
        # Time: O(d). Space: O(1) beyond the edges.
        old = self._cells.get(key)
        if old is not None and not old.is_literal:
            self._detach(key, old)
        self._cells[key] = cell
        if not cell.is_literal:
            self._dependents.setdefault(cell.child1, set()).add(key)
            self._dependents.setdefault(cell.child2, set()).add(key)
        self._invalidate(key)

    def getCellValue(self, key):
        """Evaluate a cell; raise EvaluationError on unknown cell or cycle."""
        # Time: O(size of the reachable formula tree), O(1) if cached.
        # Space: O(depth) - the recursion and visiting set.
        return self._evaluate(key, set())

    def _detach(self, key, old_cell):
        """Drop key from its old children's dependent sets."""
        # Time: O(1), Space: O(1).
        for child in (old_cell.child1, old_cell.child2):
            dependents = self._dependents.get(child)
            if dependents is not None:
                dependents.discard(key)

    def _invalidate(self, key):
        """Clear the cache for key and everything depending on it."""
        # d = affected cells. Time: O(d). Space: O(d) - stack + visited set.
        visited = set()
        stack = [key]
```

```

    while stack:
        current = stack.pop()
        if current in visited:
            continue
        visited.add(current)
        self._cache.pop(current, None)
        for parent in self._dependents.get(current, ()):
            stack.append(parent)

def _evaluate(self, key, visiting):
    """Recursive value with memoization and cycle detection."""
    # Time: O(size). Space: O(depth).
    if key in self._cache:
        return self._cache[key]
    if key not in self._cells:
        raise EvaluationError(f"unknown cell: {key}")
    if key in visiting:
        raise EvaluationError(f"cycle detected at {key}")
    cell = self._cells[key]
    if cell.is_literal:
        value = cell.value
    else:
        visiting.add(key)
        left = self._evaluate(cell.child1, visiting)
        right = self._evaluate(cell.child2, visiting)
        visiting.discard(key)
        value = left + right
    self._cache[key] = value
    return value

if __name__ == "__main__":
    sheet = Spreadsheet()

    # Example 1: a literal cell.
    sheet.setCell("A", Cell.literal(10))
    print(sheet.getCellValue("A"))    # 10

    # Example 2: a simple formula C = A + B.
    sheet.setCell("B", Cell.literal(5))
    sheet.setCell("C", Cell.formula("A", "B"))
    print(sheet.getCellValue("C"))    # 15

    # Example 3: updating A re-evaluates C (cache invalidation).
    sheet.setCell("A", Cell.literal(20))
    print(sheet.getCellValue("C"))    # 25

    # A missing reference -> error.
    sheet.setCell("D", Cell.formula("A", "missing"))
    try:
        sheet.getCellValue("D")
    except EvaluationError as err:
        print("error:", err)          # error: unknown cell: missing

    # Example 4: a cycle -> error.
    cyclic = Spreadsheet()
    cyclic.setCell("A", Cell.formula("B", "B"))
    cyclic.setCell("B", Cell.formula("A", "A"))
    try:

```

cell_spreadsheet.py (continued)

```
        cyclic.getCellValue("A")
except EvaluationError as err:
    print("error:", err)          # error: cycle detected at A
```

chat_event_counter.py

"""In-memory event counter for chat activity, keyed by an exact (user_id, chat_id) pair, over a sliding 15-minute window.

Overview:

Record chat events and, on demand, count how many of a pair's events fall in the inclusive 900-second (15-minute) window that ends at that pair's most recent event. The window is measured per pair, not by a global clock.

Interface (class ChatEventCounter):

- process_event(user_id, chat_id, timestamp) -> None
Record one event for the exact (user_id, chat_id) pair.
- get_count(user_id, chat_id) -> int
Let T be the pair's latest stored timestamp. Return how many of the pair's events have a timestamp in [T - 900, T]. Return 0 if the pair has no events.
- simulate(operations) -> list[int] (staticmethod)
Replay a list of operations and return one result per getCount, in order. Each operation is a list:
["processEvent", user_id, chat_id, timestamp]
["getCount", user_id, chat_id]
Any other op kind raises ValueError.

Semantics and rules:

- Both ends of the window are inclusive: an event exactly 900 s before T is counted; one 901 s before T is not.
- Pairs are independent and matched exactly. Keys are length-prefixed so look-alike pairs such as ("a", "bc") and ("ab", "c") never collide.

Constraints/assumptions:

- For any one pair, process_event is called with non-decreasing timestamps, so each pair's list stays sorted and get_count uses a binary search (bisect_left) for the window start - no re-sorting.

Example:

```
processEvent(u1,c1,0), processEvent(u1,c1,900), getCount(u1,c1) -> 2.
```

```
from bisect import bisect_left
```

Approach (in plain terms):

```
# Keep a separate guest book (a list of timestamps) for each exact (user_id,
# chat_id) pair. Every event just stamps the current time on that pair's
# page, and because events for a pair arrive in time order the page stays
# sorted. To count the last 15 minutes (900 s): look at the newest stamp T,
# work out the cutoff T - 900, and since the page is already sorted, do a
# quick binary search to find where the cutoff lands. Everything from that
# spot to the end is inside the window, so the answer is how many stamps
# come after.
```

Data structures used:

```
# - dict mapping the (user, chat) key -> a list of timestamps kept
#   in sorted (non-decreasing) order - O(1) grouping/append.
# - binary search (bisect) over that already-sorted list - counts
#   the window in O(log m) with no re-sorting.
```

class ChatEventCounter:

```
def __init__(self):
```

```
    # Time: O(1), Space: O(1) - starts with an empty event dictionary.
```

chat_event_counter.py (continued)

```
# (user_id, chat_id) -> ascending list of event timestamps.
self._events = {}

@staticmethod
def _key(user_id, chat_id):
    # u = len(user_id), c = len(chat_id).
    # Time:  $O(u + c)$  - builds one key string from both ids.
    # Space:  $O(u + c)$  - the returned key string.
    # Length-prefix the user_id so distinct pairs can never collide
    # (e.g. ("a", "bc") vs ("ab", "c")).
    return f"{len(user_id)}:{user_id}{chat_id}"

def process_event(self, user_id, chat_id, timestamp):
    """Record one event for (user_id, chat_id) at the given timestamp."""
    # Time:  $O(1)$  amortized - one dictionary lookup plus a list append.
    # Space:  $O(1)$  - stores a single extra timestamp.
    key = self._key(user_id, chat_id)
    self._events.setdefault(key, []).append(timestamp)

def get_count(self, user_id, chat_id):
    """Events for (user_id, chat_id) within  $[T - 900, T]$ ,  $T$  = latest ts."""
    # m = number of events stored for this pair.
    # Time:  $O(\log m)$  - bisect_left binary-searches for the window start.
    # Space:  $O(1)$  - only a few index variables.
    ts = self._events.get(self._key(user_id, chat_id))
    if not ts:
        return 0
    latest = ts[-1] # list is sorted -> last is the max
    window_start = latest - 900
    # bisect_left is a stdlib binary search:  $O(\log m)$  to find the first
    # timestamp  $\geq$  window_start; everything from there to the end is inside
    # the inclusive 900 s window.
    lo = bisect_left(ts, window_start)
    return len(ts) - lo

@staticmethod
def simulate(operations):
    """Replay operations, returning the result of every getCount in
    order."""
    # p = number of operations, m = events for the busiest pair.
    # Time:  $O(p \log m)$  - each getCount does one binary search.
    # Space:  $O(p)$  - the results list plus the stored timestamps.
    counter = ChatEventCounter()
    results = []
    for op in operations:
        kind = op[0]
        if kind == "processEvent":
            counter.process_event(op[1], op[2], op[3])
        elif kind == "getCount":
            results.append(counter.get_count(op[1], op[2]))
        else:
            raise ValueError(f"Unknown op: {kind}")
    return results

if __name__ == "__main__":
    # Test 1: three events for one pair, all inside the 15-minute window.
    chat1 = [
        ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
```

chat_event_counter.py (continued)

```
    ["processEvent", "u1", "c1", 1000], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat1))                # [3]

# Test 2: asking about a pair that has no events at all -> 0.
print(ChatEventCounter.simulate(["getCount", "u1", "c1"])) # [0]

# Test 3: a single event is always inside its own window -> 1.
chat3 = [
    ["processEvent", "u1", "c1", 500], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat3))                # [1]

# Test 4: event exactly 900 s before the latest -> boundary is inclusive.
chat4 = [
    ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 900],
    ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat4))                # [2]

# Test 5: event 901 s before the latest -> just outside the window.
chat5 = [
    ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 901],
    ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat5))                # [1]

# Test 6: only the newest event stays in the window; older ones drop off.
chat6 = [
    ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 500],
    ["processEvent", "u1", "c1", 2000], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat6))                # [1]

# Test 7: two independent pairs are counted separately.
chat7 = [
    ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
    ["processEvent", "u2", "c2", 5000],
    ["getCount", "u1", "c1"], ["getCount", "u2", "c2"],
]
print(ChatEventCounter.simulate(chat7))                # [2, 1]

# Test 8: length-prefixed keys keep look-alike pairs from colliding.
chat8 = [
    ["processEvent", "a", "bc", 100], ["processEvent", "ab", "c", 200],
    ["getCount", "a", "bc"], ["getCount", "ab", "c"],
]
print(ChatEventCounter.simulate(chat8))                # [1, 1]
```

cidr_iterator.py

"""Iterator over every IPv4 address in a CIDR block, ascending order.

Overview:

Wraps a CIDR block "a.b.c.d/p" and walks its addresses one by one, from the block's network address up to its last address (inclusive). Internally an address is just a 32-bit integer and the block is the contiguous run of integers sharing the top p bits.

Interface:

- class IPv4CIDRIterator(cidr)
Build an iterator for the block described by cidr. If the given IP is not the network address, the ENCLISING block is used (the IP is masked down to its network address before iterating).
- next() -> str | None
The next address as a dotted string, or None once the block is exhausted; further calls keep returning None.
- hasNext() -> bool
True while at least one more address remains to hand out.

Semantics / rules:

- Addresses are produced in ascending order, network first.
- The range is inclusive of both the network and the last address.
- A "/32" block yields exactly one address; a "/0" block spans the whole space, 0.0.0.0 through 255.255.255.255.

Constraints / assumptions:

- cidr is a well-formed "a.b.c.d/p" with p in 0..32; inputs are not otherwise validated.

Example:

```
it = IPv4CIDRIterator("172.16.5.9/30")
it.next() -> "172.16.5.8", then "172.16.5.9", "172.16.5.10",
"172.16.5.11", then None.
"""
```

```
def _ip_to_int(ip):
    """Pack a dotted IPv4 string into a 32-bit integer."""
    # Time: O(1) - four octets. Space: O(1).
    value = 0
    for octet in ip.split("."):
        value = value * 256 + int(octet)
    return value
```

```
def _int_to_ip(value):
    """Format a 32-bit integer as a dotted IPv4 string."""
    # Time: O(1), Space: O(1).
    octet1 = (value >> 24) & 0xFF
    octet2 = (value >> 16) & 0xFF
    octet3 = (value >> 8) & 0xFF
    octet4 = value & 0xFF
    return f"{octet1}.{octet2}.{octet3}.{octet4}"
```

Approach (in plain terms):

An IPv4 address is really just a 32-bit number, and a CIDR block is a run of consecutive numbers. The "/p" says the first p bits are fixed and the rest are free, so masking the address gives the block's first number

cidr_iterator.py (continued)

```
# (network) and setting all the free bits gives its last number. Iterating
# is then simple counting: keep a cursor at the current number, hand it out
# (formatted back into dotted form) and step forward, stopping once we pass
# the last number. Calling next() past the end keeps returning None.
# Data structures used:
#   - two integers (a current cursor and the last address) - the whole
#     block is just a numeric range, so no collection is needed.
#   - integer bit masking - derives the first/last address from the prefix.
class IPv4CIDRIterator:

    def __init__(self, cidr):
        """Parse "a.b.c.d/p"; pin down the block's first and last address."""
        # Time: 0(1), Space: 0(1).
        ip_text, prefix_text = cidr.split("/")
        prefix = int(prefix_text)
        address = _ip_to_int(ip_text)
        # The 'prefix' leading bits are fixed; the rest span the block.
        if prefix == 0:
            mask = 0
        else:
            mask = (0xFFFFFFFF << (32 - prefix)) & 0xFFFFFFFF
        network = address & mask
        self._current = network                # next address to hand out
        self._last = network | (~mask & 0xFFFFFFFF)

    def hasNext(self):
        """True while another address in the block remains."""
        # Time: 0(1), Space: 0(1).
        return self._current <= self._last

    def next(self):
        """Next address as a string, or None once the block is exhausted."""
        # Time: 0(1), Space: 0(1).
        if self._current > self._last:
            return None
        address = _int_to_ip(self._current)
        self._current += 1
        return address

if __name__ == "__main__":
    it = IPv4CIDRIterator("172.16.5.9/30")
    print(it.next())      # 172.16.5.8
    print(it.next())      # 172.16.5.9
    print(it.hasNext())   # True
    print(it.next())      # 172.16.5.10
    print(it.next())      # 172.16.5.11
    print(it.next())      # None
    print(it.hasNext())   # False

    # A /32 block is a single address.
    single = IPv4CIDRIterator("10.0.0.7/32")
    print(single.next())   # 10.0.0.7
    print(single.next())   # None

    # An IP that is not the network address still uses the enclosing block.
    block = IPv4CIDRIterator("192.168.1.130/29")
    print(block.next())    # 192.168.1.128
    print(block.hasNext()) # True
```


cluster_topology.py

"""Answer structural queries on a cluster of machines shaped as a rooted tree, given by a parent array.

Overview:

The cluster is a rooted tree where `parents[i]` is the parent of node `i` and the one node whose parent is `-1` is the root. Node IDs are the dense range `0..n-1`. From this array the class can report the node count or draw the whole tree as a compact nested string.

Interface (class `ClusterTopologyQueries`):

- `ClusterTopologyQueries(parents)`
Build the tree (each node's child list) from the parent array.
 - `query(command) -> str`
 - "count": the number of nodes, as a string.
 - "topology": a nested description of the tree. A leaf is its id ("4"); a node with children is "id(c1,c2,...)" with the children listed in ASCENDING id order, e.g. "0(1(3(5),4),2)".
- Any other command raises `ValueError`.

Semantics and rules:

- Children come out in ascending id order for free: ids `0..n-1` are scanned in order and each is appended to its parent's list.
- An empty parent array has no root, so "count" is "0" and "topology" is the empty string "".

Constraints/assumptions:

- Exactly one node has parent `-1` (the root); every other parent is a valid id in `0..n-1`, describing a single connected tree.

Example:

```
parents = [-1, 0, 0, 1, 1, 3]
query("count")    -> "6"
query("topology") -> "0(1(3(5),4),2)"
"""
```

Approach (in plain terms):

- # From the parent array we build, for each node, the list of its children.
- # Scanning ids `0..n-1` in order and appending each to its parent's list means every children list is already in ascending id order for free. "count" is simply how many nodes there are. "topology" walks the tree from the root:
- # a node with no children is written as its id; otherwise it is written as "id(...)" with its rendered children joined by commas - the recursion mirrors the nested shape of the output.
- # Data structures used:
 - # - list of lists (children) - `children[i]` holds node `i`'s children in ascending id order (dense ids `0..n-1` make a list ideal).
 - # - the call stack (recursion) - renders each subtree, matching the nested "id(...)" structure.

class `ClusterTopologyQueries`:

```
def __init__(self, parents):
    """Build the tree (each node's children) from the parent array."""
    # n = number of nodes.
    # Time: O(n) - one pass to bucket children. Space: O(n) - the lists.
    self._count = len(parents)
    self._children = []
    for _ in range(self._count):
```

cluster_topology.py (continued)

```
        self._children.append([])
    self._root = -1
    for node in range(self._count):
        parent = parents[node]
        if parent == -1:
            self._root = node
        else:
            # scanning nodes in order keeps each list ascending for free
            self._children[parent].append(node)

def query(self, command):
    """Run one command ("count" or "topology") and return a string."""
    # Time: O(1) dispatch (plus the command's own cost). Space: O(1).
    if command == "count":
        return self._count_query()
    if command == "topology":
        return self._topology_query()
    raise ValueError(f"Unknown command: {command}")

def _count_query(self):
    """Total number of nodes, as a string."""
    # Time: O(1), Space: O(1).
    return str(self._count)

def _topology_query(self):
    """Nested string of the whole tree, starting at the root."""
    # n = number of nodes.
    # Time: O(n) - renders each node once. Space: O(n) - output + stack.
    if self._root == -1:
        return "" # empty tree (no root) -> nothing to render
    return self._render(self._root)

def _render(self, node):
    """Render one subtree: 'id' for a leaf, else 'id(child,child,...)'."""
    # Time: O(size of subtree). Space: O(depth) - the recursion.
    kids = self._children[node]
    if not kids:
        return str(node)
    parts = []
    for kid in kids:
        parts.append(self._render(kid))
    return f"{node}({'.'.join(parts)})"

if __name__ == "__main__":
    # ----- Example from the problem statement -----
    cluster = ClusterTopologyQueries([-1, 0, 0, 1, 1, 3])
    print(cluster.query("count")) # 6
    print(cluster.query("topology")) # 0(1(3(5),4),2)

    # ----- A single node (root only) -----
    solo = ClusterTopologyQueries([-1])
    print(solo.query("count")) # 1
    print(solo.query("topology")) # 0

    # ----- A straight chain 0 -> 1 -> 2 -> 3 -----
    chain = ClusterTopologyQueries([-1, 0, 1, 2])
    print(chain.query("topology")) # 0(1(2(3)))
```

cluster_topology.py (continued)

```
# ----- Root is not id 0; a wide root with several leaves -----  
star = ClusterTopologyQueries([2, 2, -1, 2, 2])  
print(star.query("count"))      # 5  
print(star.query("topology"))    # 2(0,1,3,4)
```

```
"""Simulate disease contagion on a grid with recovery into immunity.
```

Overview:

A 2D grid holds people in four states. Over a fixed number of rounds, infected cells spread the disease to healthy orthogonal neighbors, and cells that have been infected long enough recover and become permanently immune. Return the grid after exactly `rounds` rounds.

Interface:

```
simulate_contagion(grid, rounds, recovery_time) -> list[list[int]]
    grid is a list of equal-length int rows; rounds and recovery_time are
    non-negative ints. Returns a NEW grid (the input is never mutated). An
    empty grid ([]) is returned unchanged.
```

Cell values:

0 = empty, 1 = healthy, 2 = infected, 3 = immune.

Semantics / rules:

- Each round is decided from the board as it looked at the START of the round, then all changes are applied together, so the step is simultaneous.
- An infected cell infects every healthy (1) up/down/left/right neighbor. A cell infected during the current round does not spread until the next.
- An infected cell tracks how many rounds it has been infectious. Once it reaches `recovery\_time` it becomes immune (3), but it still spreads during that final infectious round before recovering.

Example:

```
simulate_contagion([[1, 2, 1, 1]], rounds=3, recovery_time=2)
    -> [[3, 3, 3, 2]]
(the row evolves [1,2,1,1] -> [2,2,2,1] -> [2,3,2,2] -> [3,3,3,2])
"""
```

```
# The four orthogonal moves: down, up, right, left.
```

```
_DIRECTIONS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
def _copy_grid(grid):
```

```
    """Return an independent copy of a 2D grid so the caller's grid is never
    mutated."""
```

```
    # rows = number of rows, cols = number of columns.
```

```
    # Time: 0(rows * cols) - visits every cell once.
```

```
    # Space: 0(rows * cols) - the new grid holds every cell.
```

```
    copy = []
```

```
    for row in grid:
```

```
        new_row = []
```

```
        for value in row:
```

```
            new_row.append(value)
```

```
        copy.append(new_row)
```

```
    return copy
```

```
def _make_age_grid(grid):
```

```
    """Return a 2D grid of zeros shaped like `grid`; age[r][c] counts the
    infectious rounds an infected cell has completed."""
```

```
    # rows = number of rows, cols = number of columns.
```

```
    # Time: 0(rows * cols) - one zero written per cell.
```

```
    # Space: 0(rows * cols) - the age grid holds every cell.
```

contagion.py (continued)

```
age = []
for row in grid:
    new_row = []
    for _ in range(len(row)):
        new_row.append(0)
    age.append(new_row)
return age

# Approach (in plain terms):
# Picture a map of squares: some hold a sick person, some a healthy
# person, some are empty, and some hold someone who already recovered and
# can never get sick again. Each round is like one day.
# - Every square that is sick at the start of the day coughs on its four
# up/down/left/right neighbors; any healthy neighbor is marked to fall
# sick.
# - Every sick square also grows one day older in its illness. Once it
# has been sick for `recovery_time` days it is marked to recover.
# We decide everything from the board as it looked at the START of the day,
# then apply all new infections and recoveries together at the day's end.
# So a square infected today cannot spread until tomorrow, and a square
# still coughs on the very day it recovers.
# Data structures used:
# - a copied 2D grid - the board we evolve without touching the caller's
# grid.
# - a parallel 2D age grid - counts infectious rounds each cell has
# completed so we know when it recovers.
# - a per-round set of newly infected cells - dedupes a healthy cell
# reached by two sick neighbors in the same round.
# - a tuple of direction deltas - the four orthogonal moves.
def simulate_contagion(grid, rounds, recovery_time):
    """Run the contagion model for `rounds` rounds and return the resulting
    grid (a new list of lists; the input grid is not modified)."""
    # rows, cols = grid dimensions; k = number of rounds.
    # Time: O(k * rows * cols) - each round scans every cell once.
    # Space: O(rows * cols) - the working grid plus the parallel age grid.
    if not grid:
        return grid
    work = _copy_grid(grid)
    age = _make_age_grid(grid)
    rows = len(work)
    cols = len(work[0])

    round_index = 0
    while round_index < rounds:
        newly = set()
        becoming_immune = []
        for row in range(rows):
            for col in range(cols):
                if work[row][col] != 2:
                    continue
                for d_row, d_col in _DIRECTIONS:
                    n_row = row + d_row
                    n_col = col + d_col
                    in_bounds = 0 <= n_row < rows and 0 <= n_col < cols
                    if in_bounds and work[n_row][n_col] == 1:
                        newly.add((n_row, n_col))
                age[row][col] += 1
            if age[row][col] >= recovery_time:
```

contagion.py (continued)

```
        becoming_immune.append((row, col))
    # Apply after the full scan so the round is simultaneous.
    for row, col in becoming_immune:
        work[row][col] = 3
    for n_row, n_col in newly:
        work[n_row][n_col] = 2
        age[n_row][n_col] = 0
    round_index += 1
    return work

if __name__ == "__main__":
    # Test 1: the worked example. A single row; infection spreads outward
    # while the earliest cells recover into immunity. The board evolves
    # [1,2,1,1] -> [2,2,2,1] -> [2,3,2,2] -> [3,3,3,2].
    original = [[1, 2, 1, 1]]
    print(simulate_contagion(original, rounds=3, recovery_time=2))
    # -> [[3, 3, 3, 2]]

    # Test 2: the input grid is never mutated by the simulation.
    print(original)
    # -> [[1, 2, 1, 1]]

    # Test 3: rounds == 0 returns an unchanged copy of the grid.
    print(simulate_contagion([[1, 2, 1, 1]], rounds=0, recovery_time=2))
    # -> [[1, 2, 1, 1]]

    # Test 4: an empty grid is returned as-is.
    print(simulate_contagion([], rounds=5, recovery_time=3))
    # -> []

    # Test 5: one round from a center infection spreads to the four
    # orthogonal neighbors (recovery_time high, so nobody recovers yet).
    grid_a = [
        [1, 1, 1],
        [1, 2, 1],
        [1, 1, 1],
    ]
    print(simulate_contagion(grid_a, rounds=1, recovery_time=5))
    # -> [[1, 2, 1], [2, 2, 2], [1, 2, 1]]

    # Test 6: two rounds with recovery_time == 1. The wave moves outward -
    # the first ring recovers (3) while the corners become infected (2).
    grid_b = [
        [1, 1, 1],
        [1, 2, 1],
        [1, 1, 1],
    ]
    print(simulate_contagion(grid_b, rounds=2, recovery_time=1))
    # -> [[2, 3, 2], [3, 3, 3], [2, 3, 2]]
```

credit_ledger.py

"""GPU credit ledger fed by requests whose timestamps may arrive OUT OF ORDER (a later request can carry an earlier, back-dated timestamp).

Overview:

Track per-user GPU credit as GRANT and CHARGE requests stream in. Each GET reports the balance at a query time, considering only the requests seen so far. Because every GET recomputes from scratch, a back-dated request that arrives late affects future GETs but never rewrites a GET that was already answered.

Interface (class CreditLedger):

- grant(user, amount, timestamp, arrival) -> None
Record a GRANT (arrival = its position in the input stream).
- charge(user, amount, timestamp, arrival) -> None
Record a CHARGE; whether it succeeds is decided later, at GET (replay) time, not when recorded.
- get(user, timestamp) -> int
Balance for user at the query timestamp. Returns 0 for an unknown user.
- simulate(requests) -> list[int] (staticmethod)
Process requests in arrival order and return one value per GET.
Request forms:
["GRANT", user, amount, timestamp]
["CHARGE", user, amount, timestamp]
["GET", user, timestamp]
Any other kind raises ValueError.

Semantics and rules:

- A GET at time t replays that user's events with timestamp $\leq t$, from a starting balance of 0, in (timestamp, arrival) order.
- Equal timestamps break ties by arrival order (stream position), so a CHARGE recorded before its funding GRANT at the same timestamp is evaluated first and may be skipped.
- A CHARGE applies only if the running balance is $\geq$ its amount at that point in the replay; otherwise it is ignored (never negative).

Constraints/assumptions:

- Timestamps are not assumed to be increasing. Only events with timestamp $\leq$ the query time are considered by a GET.

Example:

GRANT alice 10 @5, CHARGE alice 4 @7, GET alice @7 -> 6; then a late CHARGE alice 10 @6, GET alice @7 -> 0.

class _Event:

"""One ledger event for a user."""

```
def __init__(self, is_grant, amount, timestamp, arrival):  
    # Time: 0(1), Space: 0(1) - just stores four fields.  
    self.is_grant = is_grant  
    self.amount = amount  
    self.timestamp = timestamp  
    # index in the input stream -> stable tie-breaker  
    self.arrival = arrival
```

```
def _event_order(event):
```

credit_ledger.py (continued)

```
"""Sort key: order events by (timestamp, arrival) so equal timestamps keep
their original arrival order (a stable tie-break)."""
# Time: O(1), Space: O(1) – returns a two-field tuple.
return (event.timestamp, event.arrival)

# Approach (in plain terms):
# Imagine each user has a shoebox where we drop in slips of paper as they
# arrive: "+10 credits at time 5" or "-4 credits at time 7". Slips can be
# dropped in any order and can even be back-dated.
# When someone asks "what's the balance at time t?", we pull out only the
# slips dated at or before t, lay them out in date order (using arrival order
# to break ties), and add them up starting from zero. A "-" slip only counts
# if there is enough money at that moment; otherwise we skip it. Because we
# recompute from scratch every time, a slip that shows up late but is
# back-dated changes future answers without
# rewriting answers we already gave.
# Data structures used:
#   - dict mapping user -> list of event objects in arrival order –
#     groups events per user and keeps arrival order for stable
#     tie-breaks.
#   - a list sorted per query for just the eligible subset – avoids
#     sorting everything.
class CreditLedger:

    def __init__(self):
        # Time: O(1), Space: O(1) – starts with an empty per-user map.
        # user -> events that have arrived so far (in arrival order).
        self._ledger = {}

    def grant(self, user, amount, timestamp, arrival):
        """Record a GRANT for user."""
        # Time: O(1) amortized (dict lookup + list append),
        # Space: O(1) per call.
        self._ledger.setdefault(user, []).append(
            _Event(True, amount, timestamp, arrival))

    def charge(self, user, amount, timestamp, arrival):
        """Record a CHARGE for user (success is decided later, at replay
        time)."""
        # Time: O(1) amortized (dict lookup + list append),
        # Space: O(1) per call.
        self._ledger.setdefault(user, []).append(
            _Event(False, amount, timestamp, arrival))

    def get(self, user, timestamp):
        """Balance for user at timestamp, replaying eligible events in
        order."""
        # n = events stored for this user, k = eligible events (k <= n).
        # Time: O(n + k log k) – scan all n events, then sort the k
        #       eligible ones.
        # Space: O(k) – the list of eligible events.
        events = self._ledger.get(user)
        if events is None:
            return 0
        # Eligible events: timestamp <= query time.
        eligible = []
        for event in events:
            if event.timestamp <= timestamp:
```

credit_ledger.py (continued)

```
        eligible.append(event)
    # Apply in (timestamp, arrival) order – stable tie-break by arrival.
    eligible.sort(key=_event_order)
    balance = 0
    for event in eligible:
        if event.is_grant:
            balance += event.amount
        elif balance >= event.amount:
            balance -= event.amount # charge applied only when affordable
    return balance

@staticmethod
def simulate(requests):
    """Process requests in arrival order, returning every GET in order."""
    # m = number of requests, n = events for the busiest user.
    # Time: O(m * n log n) worst case (each GET replays and sorts events).
    # Space: O(m + n) – the results list plus the stored events.
    ledger = CreditLedger()
    results = []
    for i, req in enumerate(requests):
        kind = req[0]
        user = req[1]
        if kind == "GRANT":
            ledger.grant(user, req[2], req[3], i)
        elif kind == "CHARGE":
            ledger.charge(user, req[2], req[3], i)
        elif kind == "GET":
            results.append(ledger.get(user, req[2]))
        else:
            raise ValueError(f"Unknown request: {kind}")
    return results

if __name__ == "__main__":
    # Test 1: a charge applied, then a back-dated charge drains the balance.
    led1 = [
        ["GRANT", "alice", 10, 5], ["CHARGE", "alice", 4, 7],
        ["GET", "alice", 7],
        ["CHARGE", "alice", 10, 6], ["GET", "alice", 7],
    ]
    print(CreditLedger.simulate(led1)) # [6, 0]

    # Test 2: two users kept independent; GET honors timestamp <= query time.
    led2 = [
        ["GRANT", "alice", 5, 10], ["GRANT", "bob", 7, 1],
        ["CHARGE", "bob", 2, 3],
        ["GET", "bob", 2], ["GET", "bob", 3],
        ["GET", "alice", 9], ["GET", "alice", 10],
    ]
    print(CreditLedger.simulate(led2)) # [7, 5, 0, 5]

    # Test 3: empty input -> no GETs -> empty result.
    led3 = []
    print(CreditLedger.simulate(led3)) # []

    # Test 4: GET for a user that never appears -> balance 0 (not found).
    led4 = [
        ["GET", "nobody", 5],
    ]
```

credit_ledger.py (continued)

```
print(CreditLedger.simulate(led4)) # [0]

# Test 5: an unaffordable CHARGE is ignored, balance unchanged.
led5 = [
    ["GRANT", "u", 3, 1], ["CHARGE", "u", 5, 2],
    ["GET", "u", 2], ["GET", "u", 5],
]
print(CreditLedger.simulate(led5)) # [3, 3]

# Test 6: a late, back-dated GRANT changes future GETs but not past
# answers.
led6 = [
    ["GRANT", "u", 10, 10], ["GET", "u", 10], ["GRANT", "u", 5, 3],
    ["GET", "u", 10], ["GET", "u", 5],
]
print(CreditLedger.simulate(led6)) # [10, 15, 5]

# Test 7: equal timestamps use arrival order – CHARGE before GRANT is
# skipped.
led7 = [
    ["CHARGE", "u", 5, 5], ["GRANT", "u", 10, 5], ["GET", "u", 5],
]
print(CreditLedger.simulate(led7)) # [10]

# Test 8: same timestamps, GRANT arrives first so the CHARGE succeeds.
led8 = [
    ["GRANT", "u", 10, 5], ["CHARGE", "u", 5, 5], ["GET", "u", 5],
]
print(CreditLedger.simulate(led8)) # [5]

# Test 9: GET strictly before an event's timestamp excludes it (boundary).
led9 = [
    ["GRANT", "u", 8, 5], ["GET", "u", 4], ["GET", "u", 5],
]
print(CreditLedger.simulate(led9)) # [0, 8]
```

```
"""GPU credit calculator whose grants expire over an INCLUSIVE window.
addCredit registers a grant usable during [timestamp, timestamp +
expiration] – BOTH ends included.
```

Overview:

Credit is granted in dated chunks that each expire. Spending draws from the grants valid at that instant, taking the soonest-to-expire first so credit is used before it lapses; a balance never goes below zero.

Interface (class GPUCredit, camelCase API):

- addCredit(credit_id, amount, timestamp, expiration) → None
Register a grant of amount usable in [timestamp, timestamp + expiration] (inclusive). credit_id is only a label (grants may share one) and is not stored.
- useCredit(timestamp, amount) → None
Spend amount at timestamp from soonest-to-expire grants first. Partial spends are allowed: it takes what the usable grants hold and silently ignores any shortfall (no error, never negative). Returns nothing.
- getBalance(timestamp) → int
Total credit still usable at timestamp (0 if none).

Semantics and rules:

- Both ends of a grant's window count, so a grant added at t with expiration e is usable at t and at t + e.
- Spending order is soonest expiry first; grants that expire at the same time break ties by the order they were added.
- The credit_id does not affect the math; two grants may carry the same id.

Constraints/assumptions:

- Each call carries its own timestamp, so requests may arrive out of order without changing an already-returned result.

Example:

```
addCredit("microsoft", 10, 10, 30) → usable in [10, 40];
getBalance(10) → 10; getBalance(40) → 10; getBalance(41) → 0.
"""
```

class _Grant:

```
"""One credit grant: a remaining balance and its inclusive window."""
```

```
def __init__(self, start, end, remaining, order):
    # Time: O(1), Space: O(1) – stores four fields.
    self.start = start
    self.end = end           # inclusive: usable in [start, end]
    self.remaining = remaining
    self.order = order       # insertion index, for stable tie-breaks
```

```
def usable_at(self, t):
    # Time: O(1), Space: O(1) – two range comparisons.
    return self.start <= t <= self.end
```

def _grant_sort_key(grant):

```
"""Sort key: soonest-expiring grant first, ties by insertion order."""
# Time: O(1), Space: O(1) – returns a two-field tuple.
```

```

    return (grant.end, grant.order)

# Approach (in plain terms):
# Think of each credit grant like a prepaid gift card that only works
# during a fixed time window and still has some money left on it. We keep a
# list of every card plus a counter remembering the order they were added.
# - getBalance(t): add up the money left on every card whose window
#   covers time t (both ends count, since the window is inclusive).
# - useCredit(t, amount): look at the cards valid at time t, then spend
#   starting from the one that expires soonest so credits get used before
#   they lapse. Take as much as each card allows until the request is
#   filled; if the cards together fall short, spend what is there and let
#   the leftover request go unfilled (a balance never goes negative).
# Because every request states its own time, out-of-order requests simply
# ask which cards are valid at that instant.
# Data structures used:
# - a list of grant objects (start, end, remaining, order) - append-only;
#   each spend filters the currently-usable grants from it.
# - an integer insertion counter + a named sort key - orders spending by
#   soonest expiry, breaking ties by insertion order.
class GPUCredit:

    def __init__(self):
        # Time: O(1), Space: O(1) - empty grant list and a zero counter.
        self._grants = []
        self._counter = 0

    def addCredit(self, credit_id, amount, timestamp, expiration):
        """Register a grant of amount usable in
        [timestamp, timestamp + expiration] (inclusive)."""
        # Time: O(1) amortized (list append), Space: O(1) per call.
        # credit_id is a label only and is intentionally not stored.
        end = timestamp + expiration
        self._grants.append(_Grant(timestamp, end, amount, self._counter))
        self._counter += 1

    def useCredit(self, timestamp, amount):
        """Spend amount at timestamp, taking from soonest-expiring grants
        first; spends what it can and ignores any unaffordable remainder."""
        # n = total grants, k = grants usable at this timestamp (k <= n).
        # Time: O(n + k log k) - scan all n grants, then sort the k usable.
        # Space: O(k) - the list of usable grants.
        usable = []
        for grant in self._grants:
            if grant.usable_at(timestamp) and grant.remaining > 0:
                usable.append(grant)

        usable.sort(key=_grant_sort_key)
        need = amount
        for grant in usable:
            if need <= 0:
                break
            take = min(grant.remaining, need)
            grant.remaining -= take
            need -= take

    def getBalance(self, timestamp):
        """Total credit still usable at the given timestamp (0 if none)."""

```

gpu_credit.py (continued)

```
# Time: O(n) – scans all n grants once. Space: O(1).
total = 0
for grant in self._grants:
    if grant.usable_at(timestamp):
        total += grant.remaining
return total

if __name__ == "__main__":
    # Example 1: a single grant usable in [10, 40] inclusive.
    g = GPUCredit()
    g.addCredit("microsoft", 10, 10, 30)
    print(g.getBalance(0))      # 0
    print(g.getBalance(10))     # 10
    print(g.getBalance(40))     # 10
    print(g.getBalance(41))     # 0

    # Example 2: spend across time, then add a second grant.
    g = GPUCredit()
    g.addCredit("amazon", 40, 10, 50)
    g.useCredit(30, 30)
    print(g.getBalance(40))     # 10
    g.addCredit("google", 20, 60, 10)
    print(g.getBalance(60))     # 30
    print(g.getBalance(61))     # 20
    print(g.getBalance(70))     # 20
    print(g.getBalance(71))     # 0

    # Example 3: overspending takes all it can and ignores the excess.
    g = GPUCredit()
    g.addCredit("a", 5, 0, 100)      # usable in [0, 100]
    g.useCredit(10, 12)              # only 5 available -> drains to 0
    print(g.getBalance(10))         # 0

    # Example 4: a spend drains the soonest-expiring grant first.
    g = GPUCredit()
    g.addCredit("early", 5, 0, 10)   # usable in [0, 10]
    g.addCredit("late", 5, 0, 30)    # usable in [0, 30]
    g.useCredit(5, 5)                # both usable; drain 'early' first
    print(g.getBalance(5))           # 5
    print(g.getBalance(20))          # 5

    # Example 5: the same credit_id can label two separate grants.
    g = GPUCredit()
    g.addCredit("dup", 3, 0, 10)     # grant #0, usable [0, 10]
    g.addCredit("dup", 4, 0, 10)     # grant #1, usable [0, 10]
    print(g.getBalance(5))           # 7
    g.useCredit(5, 10)               # 7 available -> drains both
    print(g.getBalance(5))           # 0

    # Example 6: spending when nothing is usable is a harmless no-op.
    g = GPUCredit()
    g.addCredit("x", 5, 100, 10)     # usable in [100, 110]
    g.useCredit(0, 5)                 # nothing usable at t=0
    print(g.getBalance(105))         # 5
```

gpu_credit_ledger.py

"""Idempotent in-memory credit ledger for a shared GPU pool, where each event is identified by a unique event_id.

Overview:

Tenants hold a credit balance changed by signed events. A balance is never allowed to drop below zero, and replaying the same event_id is a no-op that returns its original result – safe against duplicate deliveries.

Interface (class GpuCreditLedger):

- init(events) -> None
Apply a list of initial events in order. Each event is [event_id, tenant_id, delta] and follows the same rules as apply().
- get_balance(tenant_id) -> int
Current balance for a tenant; 0 for an unknown tenant.
- apply(event_id, tenant_id, delta) -> bool
Add delta to the tenant's balance. Returns True if accepted, False if it would make the balance negative (then nothing changes). A repeated event_id returns the SAME result it gave the first time and changes no state.
- solution(init_events, operations) -> list (staticmethod)
Run init_events, then each op, collecting one result per op.
Ops:
["get", tenant_id] -> int balance
["apply", [event_id, tenant, delta]] -> bool accepted
Any other op kind raises ValueError.

Semantics and rules:

- Rejection is all-or-nothing: an event that would overdraw is dropped entirely (balance unchanged) and reports False.
- A delta of 0 is accepted and may leave the balance exactly at 0.
- Idempotency is keyed by event_id GLOBALLY, even across tenants: the first tenant to use an id wins; a later reuse just replays the stored result.

Constraints/assumptions:

- event_id values are the identity for idempotency; callers reuse an id only when they intend the same logical event.
- Average O(1) per get/apply via a balances dict and a dict of each processed event_id's first result.

Example:

```
apply("d", "t", 5) -> True (balance 5); apply("d", "t", 5) again ->
True, but the balance stays 5 (no double apply).
```

"""

Approach (in plain terms):

- # Think of one prepaid wallet per tenant. Each event is a receipt with a unique ID saying "add or subtract this much for this tenant".
- # - Unknown tenants start at 0, and a wallet is never allowed to go negative: if a charge is too big we reject it and leave the wallet untouched.
- # - To stay idempotent we keep a notebook of every receipt ID we've already handled and the yes/no answer we gave. If the same ID shows up again we simply repeat that answer and change nothing – like ignoring a duplicate payment–confirmation email.
- # Data structures used:

gpu_credit_ledger.py (continued)

```
# - two dicts: balances (tenant -> balance) and processed
# (event_id -> first result) - O(1) balance reads and O(1)
# idempotency lookups.
class GpuCreditLedger:

    def __init__(self):
        # Time: O(1), Space: O(1) - two empty dictionaries.
        self._balances = {}
        self._processed = {} # event_id -> first result

    def init(self, events):
        """Apply the initial events in order, using the same rules."""
        # n = number of initial events.
        # Time: O(n) average - one O(1)-average apply per event.
        # Space: O(n) - balances plus one remembered result per event_id.
        for event in events:
            self.apply(event[0], event[1], event[2])

    def get_balance(self, tenant_id):
        """Current balance for a tenant; 0 if unknown."""
        # Time: O(1) average - a single dict lookup. Space: O(1).
        return self._balances.get(tenant_id, 0)

    def apply(self, event_id, tenant_id, delta):
        """Apply an event idempotently. Returns True if accepted, False if
        rejected (would go negative). A duplicate event_id returns its original
        result without changing state."""
        # Time: O(1) average - a few dict lookups/inserts.
        # Space: O(1) amortized - may remember one balance and one result.
        prior = self._processed.get(event_id)
        if prior is not None:
            return prior # idempotent replay -> same result, no state change
        current = self._balances.get(tenant_id, 0)
        nxt = current + delta
        accepted = nxt >= 0
        if accepted:
            self._balances[tenant_id] = nxt
            self._processed[event_id] = accepted # remember the first-seen result
        return accepted

    @staticmethod
    def solution(init_events, operations):
        """Apply init_events first, then run each op ("get" or "apply")."""
        # n = number of init events, m = number of operations.
        # Time: O(n + m) average - each event/op is O(1) average.
        # Space: O(n + m) - stored balances and remembered event results.
        ledger = GpuCreditLedger()
        ledger.init(init_events)
        results = []
        for op in operations:
            kind = op[0]
            if kind == "get":
                results.append(ledger.get_balance(op[1]))
            elif kind == "apply":
                event = op[1]
                results.append(ledger.apply(event[0], event[1], event[2]))
            else:
                raise ValueError(f"Unknown op: {kind}")
        return results
```

```

if __name__ == "__main__":
    # Test 1: init then mixed get/apply, including a rejected overdraft.
    init1 = [
        ["e1", "tenantA", 10], ["e2", "tenantA", -3], ["e3", "tenantB", 5],
    ]
    ops1 = [
        ["get", "tenantA"], ["get", "tenantB"],
        ["apply", ["e4", "tenantA", 2]],
        ["apply", ["e5", "tenantB", -6]], ["get", "tenantB"],
        ["apply", ["e6", "tenantA", -4]], ["get", "tenantB"],
    ]
    # -> [7, 5, True, False, 5, True, 5]
    print(GpuCreditLedger.solution(init1, ops1))

    # Test 2: unknown tenant starts at 0; duplicate event_id repeats first
    # result.
    ops2 = [
        ["get", "ghost"], ["apply", ["x1", "ghost", -3]], ["get", "ghost"],
        ["apply", ["x1", "ghost", -3]], ["apply", ["x2", "ghost", 4]],
        ["get", "ghost"],
    ]
    print(GpuCreditLedger.solution([], ops2)) # [0, False, 0, False, True, 4]

    # Test 3: no init events and no ops -> empty result.
    print(GpuCreditLedger.solution([], [])) # []

    # Test 4: a duplicate accepted event_id must NOT apply the delta twice.
    init4 = [["g1", "t", 5]]
    ops4 = [
        ["apply", ["g2", "t", 3]], ["get", "t"],
        ["apply", ["g2", "t", 3]], ["get", "t"],
    ]
    print(GpuCreditLedger.solution(init4, ops4)) # [True, 8, True, 8]

    # Test 5: reject an overdraft, then a smaller charge succeeds.
    init5 = [["a", "t", 10]]
    ops5 = [
        ["apply", ["b", "t", -15]], ["get", "t"],
        ["apply", ["c", "t", -4]], ["get", "t"],
    ]
    print(GpuCreditLedger.solution(init5, ops5)) # [False, 10, True, 6]

    # Test 6: zero delta is accepted; balance may land exactly on 0.
    ops6 = [
        ["apply", ["z", "t", 0]], ["get", "t"], ["apply", ["p", "t", 5]],
        ["apply", ["m", "t", -5]], ["get", "t"],
    ]
    print(GpuCreditLedger.solution([], ops6)) # [True, 0, True, True, 0]

    # Test 7: a rejecting event inside init leaves the balance untouched.
    init7 = [["i1", "t", -5], ["i2", "t", 8]]
    print(GpuCreditLedger.solution(init7, [["get", "t"]])) # [8]

    # Test 8: idempotency is keyed by event_id globally, across tenants.
    ops8 = [
        ["apply", ["dup", "tA", 5]], ["apply", ["dup", "tB", 5]],
        ["get", "tA"], ["get", "tB"],
    ]

```

```
]
print(GpuCreditLedger.solution([], ops8)) # [True, True, 5, 0]
```

gpu_credit_manager.py

```
"""Manager for GPU credit batches that expire over a HALF-OPEN window.
A batch added at (amount, timestamp, expiration) is usable during
[timestamp, timestamp + expiration): start inclusive, end exclusive.
```

Overview:

Credits arrive in dated batches and later become unusable once their window ends. Charges spend from whatever is currently valid, using the soonest-to-expire credits first so nothing is wasted.

Interface (class GpuCreditManager):

- add(amount, timestamp, expiration) -> None
Register a credit batch active in [timestamp, timestamp + expiration).
- charge(amount, timestamp) -> bool
If the batches active at timestamp together cover amount, deduct it (soonest-to-expire first) and return True; otherwise change nothing and return False (all-or-nothing).
- balance(timestamp) -> int
Total remaining credit across batches active at timestamp.
- simulate(operations) -> list (staticmethod)
Apply operations in order, returning one result per non-add op.
Ops (id is unused metadata):
 - ["add", id, amount, timestamp, expiration]
 - ["charge", amount, timestamp] -> bool
 - ["balance", timestamp] -> intAny other op kind raises ValueError.

Semantics and rules:

- The window is half-open: a batch is active at its start time but NOT at timestamp + expiration.
- expiration == 0 gives an empty window, so the batch is never active.
- A charge is all-or-nothing: if the active total is short, no batch is touched. Otherwise it drains soonest-expiring batches first.

Constraints/assumptions:

- Operations are applied in input order. A later op may name an earlier timestamp, but that never retroactively changes a result already returned.

Example:

```
add(10, 5, 5) -> active in [5, 10); balance(7) -> 10;
charge(4, 8) -> True; balance(8) -> 6; balance(10) -> 0.
"""
```

class _Batch:

```
"""A credit batch with a mutable remaining amount and its active window."""
```

```
def __init__(self, start, end, remaining):
    # Time: 0(1), Space: 0(1) - just stores three fields.
    self.start = start
    self.end = end          # exclusive: active in [start, end)
    self.remaining = remaining

def active_at(self, t):
    # Time: 0(1), Space: 0(1) - two range comparisons.
    return self.start <= t < self.end and self.remaining > 0
```

```

def _batch_end(batch):
    """Sort key: a batch's expiration time (used to take
    soonest-expiring first)."""
    # Time: O(1), Space: O(1) - returns a single field.
    return batch.end

# Approach (in plain terms):
# Think of each credit grant like a gift card that only works during a
# certain time window and has some money left on it. We just keep a list
# of all the gift cards.
# - balance(t): add up the money left on every card that is still valid at
#   time t.
# - charge(amount, t): look at all the cards valid at time t. If together
#   they don't have enough, don't charge anything (all-or-nothing). If they
#   do, spend from the card that expires soonest first, so credits get used
#   before they expire and go to waste.
# Every request tells us its own time, so even if requests arrive out of
# order, we simply check which cards are valid at that moment.
# Data structures used:
# - a list of credit-batch objects - append-only; each charge
#   filters the currently-active batches and sorts them by soonest
#   expiry.
class GpuCreditManager:

    def __init__(self):
        # Time: O(1), Space: O(1) - starts with an empty batch list.
        self._batches = []

    def add(self, amount, timestamp, expiration):
        """Register a new credit batch active in
        [timestamp, timestamp + expiration)."""
        # Time: O(1) amortized (list append), Space: O(1) per call.
        self._batches.append(_Batch(timestamp, timestamp + expiration, amount))

    def charge(self, amount, timestamp):
        """Consume amount from batches active at timestamp, soonest-expiring
        first. Returns True and applies the deduction only if the active
        balance covers the full amount; otherwise returns False and changes
        nothing."""
        # n = total batches, k = batches active at this timestamp (k <= n).
        # Time: O(n + k log k) - scan all n batches, then sort the k
        #   active ones.
        # Space: O(k) - the list of active batches.
        # Collect the batches that are active at this timestamp.
        active = []
        total = 0
        for batch in self._batches:
            if batch.active_at(timestamp):
                active.append(batch)
                total += batch.remaining

        if total < amount:
            return False # insufficient active balance -> no change

        # Consume from the soonest-to-expire batches first.
        active.sort(key=_batch_end)
        need = amount

```

```

    for batch in active:
        if need <= 0:
            break
        take = min(batch.remaining, need)
        batch.remaining -= take
        need -= take
    return True

def balance(self, timestamp):
    """Total remaining credits usable at the given timestamp."""
    # Time: O(n) – scans all n batches once. Space: O(1).
    total = 0
    for batch in self._batches:
        if batch.active_at(timestamp):
            total += batch.remaining
    return total

@staticmethod
def simulate(operations):
    """Simulate the manager, returning the result of every non-add op."""
    # m = number of operations, n = batches accumulated so far.
    # Time: O(m * n log n) worst case (each charge sorts the active
    #       batches).
    # Space: O(m + n) – the results list plus the stored batches.
    mgr = GpuCreditManager()
    results = []
    for op in operations:
        kind = op[0]
        if kind == "add":
            # op[1] = id (unique metadata, unused by the logic)
            mgr.add(op[2], op[3], op[4])
        elif kind == "charge":
            results.append(mgr.charge(op[1], op[2]))
        elif kind == "balance":
            results.append(mgr.balance(op[1]))
        else:
            raise ValueError(f"Unknown op: {kind}")
    return results

if __name__ == "__main__":
    # Test 1: partial charge, then balances before and after expiry.
    gpu1 = [
        ["balance", 5], ["add", "a", 10, 5, 5], ["balance", 7],
        ["charge", 4, 8], ["balance", 8], ["balance", 10],
    ]
    print(GpuCreditManager.simulate(gpu1)) # [0, 10, True, 6, 0]

    # Test 2: two batches, soonest-to-expire consumed first.
    gpu2 = [
        ["add", "a", 5, 10, 10], ["add", "b", 7, 5, 10], ["charge", 5, 12],
        ["balance", 12], ["balance", 17], ["charge", 6, 9], ["balance", 17],
    ]
    print(GpuCreditManager.simulate(gpu2)) # [True, 7, 5, False, 5]

    # Test 3: no batches at all -> nothing to read or charge.
    gpu3 = [
        ["balance", 0], ["charge", 1, 0],
    ]

```

gpu_credit_manager.py (continued)

```
print(GpuCreditManager.simulate(gpu3)) # [0, False]

# Test 4: expiration == 0 -> empty interval, batch is never active.
gpu4 = [
    ["add", "z", 100, 5, 0], ["balance", 5], ["charge", 1, 5],
]
print(GpuCreditManager.simulate(gpu4)) # [0, False]

# Test 5: window boundaries – start is inclusive, end is exclusive.
gpu5 = [
    ["add", "a", 10, 5, 5], ["balance", 5], ["balance", 10],
    ["balance", 4], ["balance", 9],
]
print(GpuCreditManager.simulate(gpu5)) # [10, 0, 0, 10]

# Test 6: soonest-to-expire batch is drained before the later one.
gpu6 = [
    ["add", "early", 5, 0, 10], ["add", "late", 5, 0, 20],
    ["charge", 5, 5], ["balance", 12], ["balance", 8],
]
print(GpuCreditManager.simulate(gpu6)) # [True, 5, 5]

# Test 7: all-or-nothing – an unaffordable charge changes nothing.
gpu7 = [
    ["add", "a", 3, 0, 100], ["charge", 5, 10], ["balance", 10],
    ["charge", 3, 10], ["balance", 10],
]
print(GpuCreditManager.simulate(gpu7)) # [False, 3, True, 0]

# Test 8: several charges, including an out-of-order (earlier) timestamp.
gpu8 = [
    ["add", "a", 10, 100, 50], ["charge", 4, 120], ["charge", 3, 110],
    ["balance", 130], ["charge", 5, 130], ["balance", 200],
]
print(GpuCreditManager.simulate(gpu8)) # [True, True, 3, False, 0]
```

"""In-memory database of schema-less records with SQL-like SELECT queries.

Overview:

Records are plain dicts of field -> value (int, float, str, or bool) with no fixed schema. Each insert is filed under an auto-assigned integer id and mirrored into an equality inverted index so that "field = value" filters resolve by set lookup instead of a full scan.

Interface:

```
- class InMemoryDb
    insert(record) -> int
        Store a COPY of the record under a fresh id (ids count up
        from 0) and index each field=value pair; returns the new id.
    query(query_string) -> list[dict]
        Run a SELECT and return the matching records, each projected
        to the requested fields, in the requested order.
```

Query grammar:

```
SELECT <*[f1,f2,...> [WHERE cond AND cond ...]
    [ORDER BY field [ASC|DESC]] [LIMIT n]
```

Each cond is "field OP literal"; OP is one of =, !=, <, >, <=, >=.
A literal is a quoted string ('..' or ".."), true/false (bool), an int, a float, or otherwise a bare string.

Semantics / rules:

- A condition holds only if the record HAS the field AND the compare is true; a missing field fails the condition, and comparing incompatible types (e.g. str vs number) counts as no match.
- Equality (=) conditions are served from the index; other operators are checked by scanning the narrowed candidate ids.
- SELECT * returns a full copy; a field list keeps only the requested fields that the record actually has.
- ORDER BY sorts by the field ascending (DESC reverses); records missing that field sort LAST, and the id is the final tie-break.
- LIMIT n keeps the first n rows after ordering.
- Clause keywords are matched case-sensitively as " WHERE ", " ORDER BY ", and " LIMIT " with surrounding spaces.

Example:

```
insert {'id':1,'name':'Alice','age':30}; insert Bob age 25;
query("SELECT * WHERE age > 25 ORDER BY name ASC")
-> [{'id': 1, 'name': 'Alice', 'age': 30}]
"""
```

```
from operator import itemgetter
```

def _parse_query(text):

```
    """Split a query into (select_fields, conditions, order, limit)."""
    # Time: O(length of the query). Space: O(number of clauses).
    text = text.strip()
    limit = None
    order = None
    limit_at = text.rfind(" LIMIT ")
    if limit_at != -1:
        limit = int(text[limit_at + 7:].strip())
        text = text[:limit_at].rstrip()
    order_at = text.rfind(" ORDER BY ")
    if order_at != -1:
```

```

        order = _parse_order(text[order_at + 10:].strip())
        text = text[:order_at].rstrip()
    where_at = text.find(" WHERE ")
    if where_at != -1:
        conditions = _parse_conditions(text[where_at + 7:].strip())
        select_clause = text[:where_at].strip()
    else:
        conditions = []
        select_clause = text
    return _parse_select(select_clause), conditions, order, limit

def _parse_select(clause):
    """Parse the SELECT list: None for '*', else a list of field names."""
    # Time: O(length). Space: O(number of fields).
    body = clause
    if body.startswith("SELECT"):
        body = body[len("SELECT"):]
    body = body.strip()
    if body == "*":
        return None
    fields = []
    for part in body.split(","):
        fields.append(part.strip())
    return fields

def _parse_order(clause):
    """Parse 'field [ASC|DESC]' into (field, descending)."""
    # Time: O(length). Space: O(1).
    parts = clause.split()
    field = parts[0]
    descending = len(parts) > 1 and parts[1].upper() == "DESC"
    return (field, descending)

def _parse_conditions(clause):
    """Parse 'a > 1 AND b = 2' into a list of (field, operator, value)."""
    # c = number of conditions. Time: O(length). Space: O(c).
    conditions = []
    for piece in clause.split(" AND "):
        conditions.append(_parse_condition(piece.strip()))
    return conditions

def _parse_condition(piece):
    """Parse one 'field OP literal' condition."""
    # Time: O(length). Space: O(1).
    for operator in ("<=", ">=", "!=", "="):      # two-char ops (and '=') first
        at = piece.find(operator)
        if at != -1:
            field = piece[:at].strip()
            literal = _parse_literal(piece[at + len(operator):].strip())
            return (field, operator, literal)
    for operator in ("<", ">"):
        at = piece.find(operator)
        if at != -1:
            field = piece[:at].strip()
            literal = _parse_literal(piece[at + len(operator):].strip())

```

```
        return (field, operator, literal)
    raise ValueError(f"cannot parse condition: {piece}")

def _parse_literal(text):
    """Turn a literal into a str (quoted), bool, int, or float."""
    # Time: 0(length). Space: 0(1).
    if len(text) >= 2 and text[0] in '"' and text[-1] == text[0]:
        return text[1:-1]
    if text == "true":
        return True
    if text == "false":
        return False
    try:
        return int(text)
    except ValueError:
        pass
    try:
        return float(text)
    except ValueError:
        return text

def _compare(actual, operator, value):
    """Apply one comparison; incomparable types count as no match."""
    # Time: 0(1), Space: 0(1).
    try:
        if operator == "=":
            return actual == value
        if operator == "!=":
            return actual != value
        if operator == "<":
            return actual < value
        if operator == ">":
            return actual > value
        if operator == "<=":
            return actual <= value
        if operator == ">=":
            return actual >= value
    except TypeError:
        return False          # e.g. comparing a str with a number
    return False

def _matches_all(record, conditions):
    """True if the record meets all conditions (a missing field -> False)."""
    # c = number of conditions. Time: 0(c). Space: 0(1).
    for field, operator, value in conditions:
        if field not in record or not _compare(record[field], operator, value):
            return False
    return True

def _project(record, select_fields):
    """Full copy for SELECT *, else only the requested fields present."""
    # Time: 0(number of fields). Space: 0(same).
    if select_fields is None:
        return dict(record)
    projected = {}
```

in_memory_db.py (continued)

```
    for field in select_fields:
        if field in record:
            projected[field] = record[field]
    return projected

# Approach (in plain terms):
# Keep every record in a table keyed by an id we hand out on insert. To make
# "field = value" lookups fast we also keep an inverted index: for each field
# and value, the set of ids that have it – so an equality filter is an
# instant set lookup instead of a full scan. A query first narrows to
# candidate ids (intersecting the index sets of any equality filters, or all
# ids if there are none), then checks the remaining comparisons by scanning
# just those candidates. Finally we sort the survivors by the ORDER BY field
# (records missing that field go last, id breaks ties) and project the
# requested columns.
# Data structures used:
# - dict id -> record – the primary store.
# - nested dict field -> value -> set of ids – the equality inverted
#   index; equality filters become set lookups/intersections.
# - lists + stable sort – ORDER BY with missing-last and id tie-break.
class InMemoryDb:

    def __init__(self):
        # Time: O(1), Space: O(1) – empty store and index.
        self._records = {}      # id -> record dict
        self._index = {}        # field -> value -> set of ids
        self._next_id = 0

    def insert(self, record):
        """Store a record under a fresh id and index its equality lookups."""
        # f = fields in the record. Time: O(f). Space: O(f).
        record_id = self._next_id
        self._next_id += 1
        self._records[record_id] = dict(record)
        for field in record:
            by_value = self._index.setdefault(field, {})
            by_value.setdefault(record[field], set()).add(record_id)
        return record_id

    def query(self, query_string):
        """Run a SELECT and return the matching records (projected/ordered)."""
        # r = candidates scanned, m = matches. Time: O(r + m log m).
        # Space: O(m).
        select_fields, conditions, order, limit = _parse_query(query_string)
        matches = []
        for record_id in self._candidate_ids(conditions):
            if _matches_all(self._records[record_id], conditions):
                matches.append(record_id)
        matches = self._order_ids(matches, order)
        if limit is not None:
            matches = matches[:limit]
        results = []
        for record_id in matches:
            results.append(_project(self._records[record_id], select_fields))
        return results

    def _candidate_ids(self, conditions):
        """Ids to consider: intersect the index sets of equality conditions,
```

```

    or all ids when there is no equality condition."""
    # Time: O(size of the smallest id sets). Space: O(that).
    equality_sets = []
    for field, operator, value in conditions:
        if operator == "=":
            ids = self._index.get(field, {}).get(value)
            if ids is None:
                return [] # nothing has this field = value
            equality_sets.append(ids)
    if not equality_sets:
        return list(self._records.keys())
    equality_sets.sort(key=len) # start from the smallest set
    candidates = set(equality_sets[0])
    for ids in equality_sets[1:]:
        candidates = candidates & ids
    return candidates

def _order_ids(self, matches, order):
    """Sort matches by ORDER BY (missing field last, id as tie-break)."""
    # m = number of matches. Time: O(m log m). Space: O(m).
    if order is None:
        matches.sort() # deterministic: id ascending
        return matches
    field, descending = order
    present = []
    missing = []
    for record_id in matches:
        record = self._records[record_id]
        if field in record:
            present.append((record[field], record_id))
        else:
            missing.append(record_id)
    present.sort(key=itemgetter(1)) # id ascending (base)
    present.sort(key=itemgetter(0), reverse=descending) # value, stable
    missing.sort()
    ordered = []
    for value, record_id in present:
        ordered.append(record_id)
    for record_id in missing:
        ordered.append(record_id)
    return ordered

if __name__ == "__main__":
    db = InMemoryDb()
    db.insert({"id": 1, "name": "Alice", "age": 30})
    db.insert({"id": 2, "name": "Bob", "age": 25})

    print(db.query("SELECT * WHERE age > 25 ORDER BY name ASC"))
    # [{'id': 1, 'name': 'Alice', 'age': 30}]

    # Equality filter served by the inverted index.
    print(db.query("SELECT name WHERE name = 'Bob'"))
    # [{'name': 'Bob'}]

    # ORDER BY DESC, then a projection.
    db.insert({"id": 3, "name": "Cara", "age": 30})
    print(db.query("SELECT name, age ORDER BY age DESC LIMIT 2"))
    # [{'name': 'Alice', 'age': 30}, {'name': 'Cara', 'age': 30}]

```

```
# Missing sort field sorts last; a condition on a missing field fails.
db.insert({"id": 4, "name": "Dan"})
print(db.query("SELECT name ORDER BY age ASC"))
# [{'name': 'Bob'}, {'name': 'Alice'}, {'name': 'Cara'}, {'name': 'Dan'}]
print(db.query("SELECT name WHERE age >= 30"))
# [{'name': 'Alice'}, {'name': 'Cara'}]
```

in_memory_table.py

"""In-memory single-table store of string cells with SQL-like commands.

Overview:

A tiny database holding ONE table. Each row has a string row_key and maps column keys to string values (every cell is a string). Commands arrive as whitespace-separated token lists and are run in order.

Interface:

```
- class InMemoryTable
    set(row_key, col_key, value) -> None
        Create or overwrite table[row_key][col_key].
    get(row_key, col_key) -> str
        The stored value, or "NULL" if the row or the cell is absent.
    select(where_col, where_value, order_by_col) -> str
        Space-joined row_keys whose where_col equals where_value,
        sorted by order_by_col value then row_key; "" if none match.
    process_commands(commands) -> list[str] (static)
        Run a list of command strings; one output per GET/SELECT.
```

Commands (tokens split on single spaces):

```
- SET row_key col_key value      store/overwrite a cell (no output).
- GET row_key col_key           emit the value, or "NULL" if absent.
- SELECT where_col where_value order_by_col
    emit the matching row_keys (see SELECT semantics below).
```

Semantics / rules:

```
- GET returns "NULL" when the row is unknown OR the column is unset.
- SELECT keeps rows where row[where_col] == where_value, then sorts
  by the order_by_col value ascending, breaks ties by row_key
  ascending, and joins the row_keys with single spaces ("" when
  nothing matches).
- A row missing the order_by_col sorts as "" (before any real value).
- Ordering is lexicographic on the string value, NOT numeric, so
  "10" sorts before "2".
- process_commands emits one string per GET and SELECT (SET emits
  nothing); an unrecognized command raises ValueError.
```

Constraints / assumptions:

```
- Values are single tokens: a command is split on spaces, so keys and
  values cannot themselves contain spaces.
```

Example:

```
SET r1 age 2; SET r2 age 10; SET r1 name bob; SET r2 name bob;
SELECT name bob age -> "r2 r1" ("10" < "2" lexicographically).
"""
```

```
def _select_sort_key(order_value_and_row_key):
```

```
    """Sort key for SELECT: the (order_by value, row_key) pair, both
    ascending."""
```

```
    # Time: O(1), Space: O(1) - returns the decorated pair unchanged as
    # the key.
```

```
    return order_value_and_row_key
```

```
# Approach (in plain terms):
```

```
# Picture a spreadsheet where every row has a name (row_key) and some
# labelled boxes (columns) holding text. We store it as a dictionary of rows,
# where each row is itself a small dictionary of column -> value.
```

in_memory_table.py (continued)

```
# - SET writes into a box, creating the row if it's new or overwriting what
# was already there.
# - GET reads one box, answering "NULL" when that row or box was never
# filled.
# - SELECT means "give me every row whose column X equals Y", then line
# those rows up sorted by another column's value (ties broken by the
# row's name).
# To sort safely we first pair each matching row with the value we sort on,
# sort those pairs, then read the names back out in order.
# Data structures used:
# - dict of dicts (row_key -> {col_key: value}) - O(1) cell get/set;
# iterate the rows for SELECT then sort the matches.
class InMemoryTable:

    def __init__(self):
        """Start with an empty table."""
        # Time: O(1), Space: O(1) - creates one empty dict.
        # row_key -> (col_key -> value); dict preserves insertion order.
        self._table = {}

    def set(self, row_key, col_key, value):
        """SET: set or overwrite a cell."""
        # Time: O(1) average, Space: O(1) - one dict insert (plus a new row
        # dict the first time a row_key is seen).
        self._table.setdefault(row_key, {})[col_key] = value

    def get(self, row_key, col_key):
        """GET: the cell value, or "NULL" if the row or column is absent."""
        # Time: O(1) average, Space: O(1) - at most two dict lookups.
        row = self._table.get(row_key)
        if row is None:
            return "NULL"
        value = row.get(col_key)
        return "NULL" if value is None else value

    def select(self, where_col, where_value, order_by_col):
        """SELECT: matching row_keys sorted by order_by_col then row_key."""
        # n = number of rows in the table, k = number of matching rows
        # (k <= n).
        # Time: O(n + k log k) - scan all n rows, then sort the k matches.
        # Space: O(k) - the list of decorated matches.
        # Pair every matching row with the value we will sort it by.
        decorated = []
        for row_key, row in self._table.items():
            if row.get(where_col) == where_value:
                sort_value = self._sort_value(row_key, order_by_col)
                decorated.append((sort_value, row_key))
        # Sort by order_by_col value, ties broken by row_key (both ascending).
        decorated.sort(key=_select_sort_key)
        # Read the row_keys back out, now in sorted order.
        matches = []
        for sort_value, row_key in decorated:
            matches.append(row_key)
        return " ".join(matches)

    def _sort_value(self, row_key, order_by_col):
        """The order_by_col value for a row, or "" if the row lacks it."""
        # Time: O(1) average, Space: O(1) - one dict lookup.
        value = self._table[row_key].get(order_by_col)
```

in_memory_table.py (continued)

```
    return "" if value is None else value

    @staticmethod
    def process_commands(commands):
        """Process commands, returning one output per GET/SELECT in order."""
        # m = number of commands, n = rows in the table.
        # Time: O(m * n log n) worst case (each SELECT scans and sorts the
        #       rows).
        # Space: O(m + n) - the outputs list plus the stored rows.
        db = InMemoryTable()
        outputs = []
        for command in commands:
            tokens = command.split(" ")
            if tokens[0] == "SET":
                db.set(tokens[1], tokens[2], tokens[3])
            elif tokens[0] == "GET":
                outputs.append(db.get(tokens[1], tokens[2]))
            elif tokens[0] == "SELECT":
                outputs.append(db.select(tokens[1], tokens[2], tokens[3]))
            else:
                raise ValueError(f"Unknown command: {tokens[0]}")
        return outputs

if __name__ == "__main__":
    # Test 1: GET a cell, then SELECT ordered by a numeric-looking string
    # column.
    sql1 = [
        "SET r1 name bob", "SET r1 age 2", "SET r2 name bob", "SET r2 age 10",
        "GET r1 name", "SELECT name bob age",
    ]
    print(InMemoryTable.process_commands(sql1)) # ['bob', 'r2 r1']

    # Test 2: GET a missing row -> NULL; SELECT with no matches -> "".
    print(InMemoryTable.process_commands(
        ["GET missing name", "SELECT status active score"])) # ['NULL', '']

    # Test 3: empty program -> no outputs at all.
    print(InMemoryTable.process_commands([])) # []

    # Test 4: overwrite -> the latest SET to a cell wins.
    over = ["SET r1 v a", "SET r1 v b", "GET r1 v"]
    print(InMemoryTable.process_commands(over)) # ['b']

    # Test 5: GET a column the row never set -> NULL (row exists, cell does
    # not).
    absent = ["SET r1 name x", "GET r1 age"]
    print(InMemoryTable.process_commands(absent)) # ['NULL']

    # Test 6: SELECT whose filter matches nothing -> "".
    no_match = ["SET r1 name x", "SELECT name y age"]
    print(InMemoryTable.process_commands(no_match)) # ['']

    # Test 7: ties on the order_by value are broken by row_key ascending.
    ties = [
        "SET r3 name x", "SET r3 k 5", "SET r1 name x", "SET r1 k 5",
        "SET r2 name x", "SET r2 k 5", "SELECT name x k",
    ]
    print(InMemoryTable.process_commands(ties)) # ['r1 r2 r3']
```

```
# Test 8: a missing order_by value sorts as "" (before any real value).
missing_order = ["SET a f m", "SET b f m", "SET b s 9", "SELECT f m s"]
print(InMemoryTable.process_commands(missing_order)) # ['a b']

# Test 9: ordering is lexicographic on the string value, not numeric.
numeric = [
    "SET r1 g h", "SET r1 n 9", "SET r2 g h", "SET r2 n 10",
    "SET r3 g h", "SET r3 n 100", "SELECT g h n",
]
print(InMemoryTable.process_commands(numeric)) # ['r2 r3 r1']
```

```
"""IPv4/IPv6 toolkit built from manual parsing and bit math (no libs).
```

Overview:

Five self-contained exercises over IP addresses. IPv4 helpers treat an address as four decimal octets packed into a 32-bit integer; the IPv6 helper treats an address as eight PERIOD-separated 4-digit hex groups (compressed "::" is NOT supported). Invalid input is reported by a sentinel return, never an exception.

Interface (all module-level functions):

- `parse_ipv4(ip)` -> `list[int]`
The four octets, or `[]` if `ip` is not a valid dotted IPv4.
- `adjacent_ipv4(ip, direction)` -> `str`
The address one step "up" or "down"; "" on invalid `ip`, bad direction, or over/underflow past the 32-bit ends.
- `cidr_range(cidr)` -> `list[str]`
[first, last] (network and last address) of "A.B.C.D/P", or `[]` if the address or prefix is invalid.
- `ip_in_cidr(ip, cidr)` -> `int`
1 if `ip` lies in `cidr` (inclusive), 0 if not, -1 if either input is invalid.
- `adjacent_ipv6(ip, direction)` -> `str`
The address one step "up" or "down", normalized to eight lowercase 4-digit hex groups joined by "."; "" on invalid input, bad direction, or 128-bit over/underflow.

Semantics / rules:

- An IPv4 octet is digits only, 0..255; leading zeros are allowed and there must be exactly four octets.
- A CIDR prefix must be an integer 0..32; the range is derived by masking (first = network, last = all host bits set).
- Membership and CIDR ranges are inclusive of both endpoints.
- direction must be exactly "up" or "down"; anything else yields "".
- Each IPv6 group is 1..4 hex digits (0..0xFFFF); the output always re-pads every group to four lowercase hex digits.

Example:

```
parse_ipv4("192.168.0.1")          -> [192, 168, 0, 1]
adjacent_ipv4("192.168.0.255", "up") -> "192.168.1.0"
cidr_range("192.168.0.0/24")       -> ["192.168.0.0", "192.168.0.255"]
ip_in_cidr("192.168.0.10", "192.168.0.0/24") -> 1
"""
```

Approach (in plain terms):

- # Treat an IP address like the number on a car's odometer. Each octet (or hex group) is one "wheel", and the whole address is just one big number written in separate pieces.
- # - parse: read the dotted text and make sure every wheel holds a legal number.
- # - adjacent: roll the odometer one click up or down; when a wheel passes its max it rolls over to 0 and nudges the wheel on its left (a carry), and the mirror image happens when rolling down (a borrow).
- # - CIDR block: a "street" where every address shares the same leading bits; the prefix says how many leading bits are fixed, which pins down the first and last address on that street.
- # - membership: check whether an address falls between that street's first and last address.
- # It is all plain integer and bit-mask math – no networking libraries.

ip_addressing.py (continued)

```
# Data structures used:
#   - fixed-size lists of integer octets (4) or hex groups (8) - makes
#     per-part validation simple.
#   - a single integer used as a 32-bit mask/value - turns range and CIDR
#     checks into simple bit math.

# ----- Part 1: validate & parse IPv4 -----
def parse_ipv4(ip):
    """Parse a dotted-decimal IPv4 into 4 octets, or [] if invalid."""
    # n = length of the ip string.
    # Time: O(n) - scans each character once while validating.
    # Space: O(1) - 4 octets.
    octets = _to_octets(ip)
    if octets is None:
        return [] # invalid -> empty list
    return list(octets)

# ----- Part 2: adjacent IPv4 -----
def adjacent_ipv4(ip, direction):
    """The IPv4 one above ('up') or below ('down'); "" on
    invalid/overflow/underflow."""
    # n = length of the ip string.
    # Time: O(n) - one parse pass plus O(1) arithmetic. Space: O(1).
    octets = _to_octets(ip)
    if octets is None:
        return ""
    value = _octets_to_long(octets)
    if direction == "up":
        if value == 0xFFFFFFFF:
            return "" # would overflow above 255.255.255.255
        value += 1
    elif direction == "down":
        if value == 0:
            return "" # would underflow below 0.0.0.0
        value -= 1
    else:
        return "" # invalid direction
    return _long_to_ipv4(value)

# ----- Part 3: first & last address of a CIDR block -----
def cidr_range(cidr):
    """Inclusive [first, last] addresses of A.B.C.D/P; [] if invalid."""
    # n = length of the cidr string.
    # Time: O(n) - parse the address and prefix once, then O(1) math.
    # Space: O(1).
    first_last = _cidr_first_last(cidr)
    if first_last is None:
        return []
    return [_long_to_ipv4(first_last[0]), _long_to_ipv4(first_last[1])]

# ----- Part 4: membership in a CIDR block -----
def ip_in_cidr(ip, cidr):
    """1 if ip is inside cidr (inclusive), 0 if not, -1 if either input is
    invalid."""
    # n = combined length of the ip and cidr strings.
```

ip_addressing.py (continued)

```
# Time: O(n) – parse both inputs once, then O(1) comparisons. Space: O(1).
octets = _to_octets(ip)
first_last = _cidr_first_last(cidr)
if octets is None or first_last is None:
    return -1
value = _octets_to_long(octets)
return 1 if first_last[0] <= value <= first_last[1] else 0

# ----- Part 5: adjacent IPv6 (8 period-separated hex groups) -----
def adjacent_ipv6(ip, direction):
    """The IPv6 one above ('up') or below ('down'), normalized to eight
    lowercase 4-digit hex groups separated by '.'; "" on invalid input, invalid
    direction, or 128-bit over/underflow. Compressed '::' is NOT allowed."""
    # n = number of hex groups (always 8 when the input is valid).
    # Time: O(n) – parse the groups, propagate one carry/borrow, then
    #         format each group.
    # Space: O(n) – the list of 8 groups plus the hex pieces we join.
    groups = _to_groups(ip)
    if groups is None:
        return ""
    if direction == "up":
        up = True
    elif direction == "down":
        up = False
    else:
        return "" # invalid direction

    idx = 7
    if up:
        # propagate carry from the least-significant group
        while idx >= 0 and groups[idx] == 0xFFFF:
            groups[idx] = 0
            idx -= 1
        if idx < 0:
            return "" # overflow above ffff.ffff...ffff
        groups[idx] += 1
    else:
        # propagate borrow from the least-significant group
        while idx >= 0 and groups[idx] == 0:
            groups[idx] = 0xFFFF
            idx -= 1
        if idx < 0:
            return "" # underflow below 0000.0000...0000
        groups[idx] -= 1

    # Build the eight hex pieces one at a time, then join them with '.'.
    pieces = []
    for group in groups:
        pieces.append(_to_hex4(group))
    return ".".join(pieces)

# ----- shared helpers -----

def _to_octets(ip):
    """Validate and split a dotted-decimal IPv4 into 4 octets, or None if
    invalid."""
    # n = length of the ip string.
```

ip_addressing.py (continued)

```
# Time: 0(n) – split, then validate every character once.
# Space: 0(1) – 4 octets.
parts = _split_exact(ip, ".", 4)
if parts is None:
    return None
octets = []
for part in parts:
    value = _parse_octet(part)
    if value < 0:
        return None
    octets.append(value)
return octets

def _parse_octet(s):
    """Parse a decimal octet (digits only, leading zeros allowed, 0..255);
    -1 if invalid."""
    # n = number of characters in s (at most a handful).
    # Time: 0(n) – one pass over the digits. Space: 0(1).
    if not s:
        return -1
    value = 0
    for ch in s:
        if ch < "0" or ch > "9":
            return -1
        value = value * 10 + (ord(ch) - ord("0"))
        if value > 255:
            return -1 # also guards against overflow on long inputs
    return value

def _cidr_first_last(cidr):
    """[first, last] of a CIDR block, or None if invalid."""
    # n = length of the cidr string.
    # Time: 0(n) – parse the address and prefix once, then 0(1) bit math.
    # Space: 0(1).
    parts = _split_exact(cidr, "/", 2)
    if parts is None:
        return None
    octets = _to_octets(parts[0])
    if octets is None:
        return None
    prefix = _parse_prefix(parts[1])
    if prefix < 0:
        return None # prefix must be 0..32
    base = _octets_to_long(octets)
    mask = 0 if prefix == 0 else ((0xFFFFFFFF << (32 - prefix)) & 0xFFFFFFFF)
    first = base & mask
    last = first | (~mask & 0xFFFFFFFF)
    return [first, last]

def _parse_prefix(s):
    """Parse a CIDR prefix length (digits only); -1 if not an integer in
    [0, 32]."""
    # n = number of characters in s (at most a couple).
    # Time: 0(n) – one pass over the digits. Space: 0(1).
    if not s:
        return -1
```

ip_addressing.py (continued)

```
value = 0
for ch in s:
    if ch < "0" or ch > "9":
        return -1
    value = value * 10 + (ord(ch) - ord("0"))
    if value > 32:
        return -1
return value

def _to_groups(ip):
    """Parse the 8 period-separated hex groups (1..4 hex digits each); None
    if invalid."""
    # n = length of the ip string.
    # Time: O(n) - split, then validate every hex digit once.
    # Space: O(1) - 8 groups.
    parts = _split_exact(ip, ".", 8)
    if parts is None:
        return None
    groups = []
    for part in parts:
        value = _parse_hex_group(part)
        if value < 0:
            return None
        groups.append(value)
    return groups

def _parse_hex_group(s):
    """Parse a 1..4 digit hex group into 0..0xFFFF; -1 if invalid."""
    # n = number of characters in s (at most 4).
    # Time: O(n) - one pass over the hex digits. Space: O(1).
    if not s or len(s) > 4:
        return -1
    value = 0
    for ch in s:
        digit = _hex_val(ch)
        if digit < 0:
            return -1
        value = value * 16 + digit
    return value

def _hex_val(c):
    # Time: O(1) - a few range checks on a single character. Space: O(1).
    if "0" <= c <= "9":
        return ord(c) - ord("0")
    if "a" <= c <= "f":
        return ord(c) - ord("a") + 10
    if "A" <= c <= "F":
        return ord(c) - ord("A") + 10
    return -1

def _to_hex4(v):
    """Format a 16-bit value as exactly 4 lowercase hex digits."""
    # Time: O(1) - formats a fixed-width 4-digit string. Space: O(1).
    return f"{v:04x}"
```

```

def _split_exact(s, delim, expected):
    """Split s on delim (no merging, trailing empties kept); return the parts
    only if there are exactly 'expected' of them, else None."""
    # n = length of the string s.
    # Time: O(n) – a single split pass. Space: O(n) – the list of split parts.
    if s is None:
        return None
    parts = s.split(delim)
    if len(parts) != expected:
        return None
    return parts

def _octets_to_long(octets):
    # Time: O(1) – four shifts/ors on a fixed-size list. Space: O(1).
    return (octets[0] << 24) | (octets[1] << 16) | (octets[2] << 8) | octets[3]

def _long_to_ipv4(value):
    # Time: O(1) – four shifts/masks and one format. Space: O(1).
    octet1 = (value >> 24) & 0xFF
    octet2 = (value >> 16) & 0xFF
    octet3 = (value >> 8) & 0xFF
    octet4 = value & 0xFF
    return f"{octet1}.{octet2}.{octet3}.{octet4}"

if __name__ == "__main__":
    # ----- parse_ipv4 -----
    print(parse_ipv4("192.168.0.1"))          # [192, 168, 0, 1]
    print(parse_ipv4("0.0.0.0"))             # [0, 0, 0, 0]
    print(parse_ipv4("255.255.255.255"))     # [255, 255, 255, 255]
    # -> [10, 0, 0, 1] (leading zeros allowed)
    print(parse_ipv4("010.0.0.1"))
    print(parse_ipv4("256.1.1.1"))          # [] (octet > 255)
    print(parse_ipv4("1.2.3"))              # [] (too few octets)
    print(parse_ipv4("1.2.3.4.5"))          # [] (too many octets)
    print(parse_ipv4("1.2.3.a"))            # [] (non-digit)
    print(parse_ipv4(""))                   # [] (empty string)

    # ----- adjacent_ipv4 -----
    print(adjacent_ipv4("192.168.0.255", "up"))      # 192.168.1.0
    print(adjacent_ipv4("192.168.1.0", "down"))      # 192.168.0.255
    print(adjacent_ipv4("0.0.0.0", "up"))            # 0.0.0.1
    # -> "" (overflow -> empty line)
    print(adjacent_ipv4("255.255.255.255", "up"))
    # -> "" (underflow -> empty line)
    print(adjacent_ipv4("0.0.0.0", "down"))
    # -> "" (bad direction -> empty line)
    print(adjacent_ipv4("192.168.0.1", "sideways"))
    # -> "" (invalid ip -> empty line)
    print(adjacent_ipv4("999.1.1.1", "up"))

    # ----- cidr_range -----
    print(cidr_range("192.168.0.0/24"))          # ['192.168.0.0', '192.168.0.255']
    print(cidr_range("0.0.0.0/0"))              # ['0.0.0.0', '255.255.255.255']
    print(cidr_range("192.168.1.130/25"))       # ['192.168.1.128', '192.168.1.255']
    print(cidr_range("10.0.0.5/32"))            # ['10.0.0.5', '10.0.0.5']

```

ip_addressing.py (continued)

```
print(cidr_range("1.2.3.4/33"))      # [] (prefix > 32)
print(cidr_range("1.2.3.4"))         # [] (no prefix)

# ----- ip_in_cidr -----
print(ip_in_cidr("192.168.0.10", "192.168.0.0/24")) # 1
print(ip_in_cidr("192.168.0.0", "192.168.0.0/24"))  # 1 (first address)
print(ip_in_cidr("192.168.0.255", "192.168.0.0/24")) # 1 (last address)
print(ip_in_cidr("192.168.1.10", "192.168.0.0/24"))  # 0 (outside block)
# -> 1 (whole address space)
print(ip_in_cidr("10.0.0.0", "0.0.0.0/0"))
print(ip_in_cidr("bad", "192.168.0.0/24"))           # -1 (invalid ip)
print(ip_in_cidr("192.168.0.1", "192.168.0.0/33"))   # -1 (invalid cidr)

# ----- adjacent_ipv6 -----
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.ffff", "up"))
# 0000.0000.0000.0000.0000.0000.0001.0000
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0001.0000", "down"))
# 0000.0000.0000.0000.0000.0000.0000.ffff
# -> "" (overflow)
print(adjacent_ipv6("ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffff", "up"))
# -> "" (underflow)
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.0000", "down"))
# -> "" (bad direction)
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.0001", "left"))
# -> "" (only 7 groups)
print(adjacent_ipv6("0.0.0.0.0.0.0", "up"))
```

kv_store.py

"""In-memory string->string key-value store that (de)serializes as JSON.

Overview:

A thin wrapper around a single dict mapping string keys to string values, with helpers to snapshot the whole store to a JSON string and to reload it from one. All operations are ordinary dict work.

Interface:

```
- class KVStore
    get(key) -> str | None
        The stored value, or None when the key is absent.
    set(key, value) -> None
        Insert a new pair or overwrite an existing key.
    delete(key) -> bool
        Remove the key; True if it existed, False if it was missing.
    keys() -> list[str]
        Every key in ascending sorted order.
    serialize() -> str
        The whole store as a JSON object string, keys sorted so the
        output is stable/deterministic.
    deserialize(data) -> None
        REPLACE the store's contents with the pairs parsed from the
        JSON string data.
```

Semantics / rules:

- Keys and values are expected to be strings; get on an unknown key returns None rather than raising.
- serialize always sorts keys, so equal stores serialize identically.
- deserialize discards the current contents and adopts the parsed map wholesale.

Example:

```
set("theme", "dark"); set("region", "eu")
serialize() -> '{"region": "eu", "theme": "dark"}'
keys()      -> ["region", "theme"]
"""
```

```
import json
```

Approach (in plain terms):

```
# A key-value store is exactly what a Python dict is for, so the whole store
# is just one dict. get/set/delete are direct dict operations; delete also
# reports whether the key was actually present. Saving and loading reuse the
# json module: serialize turns the dict into a JSON string, and deserialize
# parses a JSON string back into a fresh dict, replacing what we had. keys()
# hands back the dict's keys run through sorted() so they come out in order.
# Data structures used:
#   - dict - the store itself; O(1) average get/set/delete by key.
#   - sorted list (built on demand) - keys() returns keys in order.
```

```
class KVStore:
```

```
    def __init__(self):
        """Start with an empty store."""
        # Time: O(1), Space: O(1) - one empty dict.
        self._store = {}

    def get(self, key):
        """Value for key, or None if the key is absent."""
```

kv_store.py (continued)

```
# Time: O(1) average – one dict lookup. Space: O(1).
return self._store.get(key)

def set(self, key, value):
    """Insert or overwrite the value for key."""
    # Time: O(1) average – one dict insert. Space: O(1).
    self._store[key] = value

def delete(self, key):
    """Remove key; return True if it existed, False if it was missing."""
    # Time: O(1) average. Space: O(1).
    if key in self._store:
        del self._store[key]
        return True
    return False

def keys(self):
    """All keys in ascending sorted order."""
    # n = number of keys. Time: O(n log n) – sorts the keys.
    # Space: O(n) – the returned list.
    return sorted(self._store.keys())

def serialize(self):
    """The whole store as a JSON string (keys sorted, stable form)."""
    # n = number of pairs, m = total characters.
    # Time: O(n log n + m). Space: O(m) – the JSON string.
    return json.dumps(self._store, sort_keys=True)

def deserialize(self, data):
    """Replace the store's contents with the pairs from a JSON string."""
    # m = length of data. Time: O(m) – parse the JSON. Space: O(m).
    self._store = json.loads(data)

if __name__ == "__main__":
    store = KVStore()
    store.set("theme", "dark")
    store.set("region", "eu")
    print(store.get("theme"))      # dark
    print(store.get("missing"))    # None
    print(store.keys())           # ['region', 'theme']

    snapshot = store.serialize()
    print(snapshot)               # {"region": "eu", "theme": "dark"}

    print(store.delete("theme"))   # True    (was present)
    print(store.delete("theme"))   # False   (already gone)
    print(store.keys())           # ['region']

    # Restore from the earlier snapshot -> both pairs come back.
    store.deserialize(snapshot)
    print(store.keys())           # ['region', 'theme']
    print(store.get("theme"))     # dark
```

labeling_schedule.py

```
"""Generate labeling schedules of [task_id, model_id, human_id] triples.
```

Overview:

Work items are labeled by humans using models. Tasks are named $t1..t\{total_task\}$, models $m1..m\{total_model\}$, and humans $h1..h\{total_human\}$. A schedule is a flat list of [task_id, model_id, human_id] rows. Every human must receive at least k assignments, no human labels the same task twice, and every ID must be valid.

Interface:

```
any_schedule(total_task, total_model, total_human, k) -> list[list[str]]
    Part 1: any valid schedule. Each human gets  $k$  distinct tasks; the
    reference rotates each human's starting task by the human index (human
     $i$  takes tasks  $i..i+k-1 \bmod total\_task$ ). model_id is always "m1".
prefix_balanced_schedule(total_task, total_model, total_human, k)
    -> list[list[str]]
    Part 2: additionally prefix-balanced. Emitted in  $k$  rounds; round  $r$ 
    assigns task  $t\{r+1\}$  to every human, and human  $i$  uses model
     $m\{((i + r) \% total\_model) + 1\}$ . Because every round completes before
    the next starts and models rotate, cutting the output at any prefix
    keeps model usage (and per-human work) balanced.
```

Semantics / rules:

- any_schedule output is human-major (all of human 1's rows, then human 2's, ...); prefix_balanced_schedule output is round-major (all humans in round 1, then round 2, ...).
- Both return an empty list when no valid schedule exists: $k == 0$ (nothing to schedule) or $k > total_task$ (cannot pick k distinct tasks). Zero humans likewise yields an empty list.

Constraints / assumptions:

- All counts and k are non-negative ints; total_model ≥ 1 whenever rows are produced (it is used as a rotation modulus).

Example:

```
any_schedule(3, 1, 2, 2) ->
[[ 't1', 'm1', 'h1'], [ 't2', 'm1', 'h1'], [ 't2', 'm1', 'h2'], [ 't3', 'm1', 'h2']]
,,,,,
```

Approach (in plain terms):

```
# Picture running a workshop where every helper (human) must finish  $k$ 
# different tasks, and you want to hand out the work in a tidy, repeatable
# way.
# - any_schedule: give each helper  $k$  tasks in a row, but start each helper
#   on a different task by rotating the starting point by the helper's
#   number - like dealing cards - so nobody repeats a task and the work
#   spreads out. Model is always m1 because Part 1 only cares about a valid
#   assignment.
# - prefix_balanced_schedule: instead of finishing one helper at a time, go
#   round by round. In round  $r$  every helper does the same task  $t\{r+1\}$  at
#   once, and we rotate which model each helper uses. Because a whole round
#   finishes before the next starts, if you stop reading the list at ANY
#   point everyone has done nearly the same amount - that is what
#   "prefix-balanced" means.
# No schedule is possible when  $k$  distinct tasks cannot fit ( $k > total\_task$ );
# when  $k$  is 0 there is simply nothing to schedule, so an empty list is the
# answer.
# Data structures used:
```

labeling_schedule.py (continued)

```
# - a plain list that accumulates the [task, model, human] triples,
# filled by modulo (rotation) index arithmetic - no auxiliary
# structure needed since the schedule is produced directly in order.
def any_schedule(total_task, total_model, total_human, k):
    """Part 1: any valid schedule. Each human gets k distinct tasks; the
    reference rotates each human's starting task by the human index (human i
    takes tasks (i+0)..(i+k-1) mod total_task). Model is always m1."""
    # h = number of humans, k = required assignments per human.
    # Time:  $O(h * k)$  - emits exactly k rows for each of the h humans.
    # Space:  $O(h * k)$  - the returned schedule holds one row per assignment.
    result = []
    if k == 0:
        return result # no assignments required -> empty schedule is valid
    if k > total_task:
        return result # cannot give a human k distinct tasks -> impossible
    for human_index in range(total_human): # human-major order
        for task_offset in range(k): # the k distinct tasks for this human
            task_index = (human_index + task_offset) % total_task
            result.append([f"t{task_index + 1}", "m1", f"h{human_index + 1}"])
    return result

def prefix_balanced_schedule(total_task, total_model, total_human, k):
    """Part 2: a prefix-balanced schedule. Emitted in k rounds; round r assigns
    task t{r+1} to every human, with human i using model
    m{((i + r) % total_model) + 1}. Every prefix satisfies both balance
    rules."""
    # h = number of humans, k = number of rounds (assignments per human).
    # Time:  $O(h * k)$  - emits one row per human across k rounds.
    # Space:  $O(h * k)$  - the returned schedule holds one row per assignment.
    result = []
    if k == 0:
        return result
    if k > total_task:
        return result
    for round_index in range(k): # round r -> task t{r+1}
        for human_index in range(total_human): # human index within the round
            model_index = (human_index + round_index) % total_model
            result.append(
                [f"t{round_index + 1}", f"m{model_index + 1}",
                 f"h{human_index + 1}"])
    return result

if __name__ == "__main__":
    # ----- any_schedule -----
    print(any_schedule(3, 1, 2, 2))
    # [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
    #  ['t3', 'm1', 'h2']]
    print(any_schedule(3, 1, 1, 1)) # [['t1', 'm1', 'h1']]
    print(any_schedule(2, 1, 3, 2)) # rotation wraps around the 2 tasks
    # [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
    #  ['t1', 'm1', 'h2'], ['t1', 'm1', 'h3'], ['t2', 'm1', 'h3']]
    print(any_schedule(3, 1, 2, 0)) # [] (k == 0 -> nothing to schedule)
    print(any_schedule(2, 1, 2, 3)) # [] (k > total_task -> impossible)
    print(any_schedule(5, 1, 0, 2)) # [] (no humans)

    # ----- prefix_balanced_schedule -----
    print(prefix_balanced_schedule(3, 2, 4, 2))
```

labeling_schedule.py (continued)

```
# [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t1', 'm1', 'h3'],  
#  ['t1', 'm2', 'h4'], ['t2', 'm2', 'h1'], ['t2', 'm1', 'h2'],  
#  ['t2', 'm2', 'h3'], ['t2', 'm1', 'h4']]  
print(prefix_balanced_schedule(3, 2, 2, 1))    # single round, models m1/m2  
# [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2']]  
# model index rotates each round  
print(prefix_balanced_schedule(2, 3, 2, 2))  
# [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t2', 'm2', 'h1'],  
#  ['t2', 'm3', 'h2']]  
print(prefix_balanced_schedule(3, 2, 4, 0))    # [] (k == 0)  
print(prefix_balanced_schedule(2, 2, 2, 3))    # [] (k > total_task)
```

```
"""Invertible, binary-safe length-prefixed encoding of a dict[str, str].
```

Overview:

Flattens a string->string map into one string and rebuilds it exactly, even when keys or values contain otherwise-dangerous characters such as '#', ':', '|', '=', or spaces. Every field is written as <length>#<content>, so a reader takes a length and then consumes exactly that many characters – the content is never scanned for a delimiter and may hold anything.

Interface (module-level functions):

- serialize(mapping) -> str
Encode the map. Pairs are written in lexicographically sorted key order, so the same map always yields the same string.
- deserialize(data) -> dict
Rebuild the original map from a serialized string; raises ValueError on malformed input.
- solution(operation, data) -> str | dict
Dispatch helper: "serialize" runs serialize(data), "deserialize" runs deserialize(data), anything else raises ValueError.

Format:

<pair_count>#<key_len>#<key><value_len>#<value>...
A length is decimal digits terminated by '#'; the content that follows is taken verbatim for exactly that many characters.

Semantics / rules:

- Sorting keys makes serialize deterministic: equal maps encode to byte-identical strings.
- Empty pieces are fine: the empty map encodes as "0#" and an empty key or value contributes "0#".
- deserialize raises ValueError when a length is missing, a length token holds a non-digit, or a field runs past the end of the input.

Example:

```
serialize({"a": "hi", "b": ""}) -> "2#1#a2#hi1#b0#"
deserialize("2#1#a2#hi1#b0#") -> {"a": "hi", "b": ""}
serialize({}) -> "0#"
"""
```

Approach (in plain terms):

```
# We need to flatten a dictionary into one string and later rebuild it
# exactly, even when keys or values contain tricky characters like '#', ':'
# or spaces. The trick is the one shipping labels use: write the LENGTH of
# each piece right before the piece itself. To read it back you first read
# the number, then grab exactly that many characters – so the content can be
# anything without ever confusing the reader about where a piece ends.
# - serialize: first write how many pairs there are, then for each key and
#   each value write "<length>#<text>". Keys are sorted so the same map
#   always produces the exact same string.
# - deserialize: read the pair count, then repeatedly read a length, take
#   that many characters, and rebuild each key and value.
# It is like tearing off exactly the number of squares of tape someone told
# you to take, instead of guessing where one piece stops and the next begins.
# Data structures used:
# - a dict for the map, with keys sorted for deterministic output.
# - a single-element list used as a movable read cursor while parsing -
```

map_serializer.py (continued)

```
# lets a helper advance the parse position by reference.
def serialize(mapping):
    """Serialize a map to the length-prefixed format (keys sorted
    lexicographically)."""
    # n = number of pairs, m = total characters across all keys and values.
    # Time:  $O(n \log n + m)$  – sort the n keys, then copy m characters of
    # content.
    # Space:  $O(n + m)$  – the collected pieces and the joined output string.
    parts = [f"{len(mapping)}#"]
    for key in sorted(mapping): # deterministic key order
        value = mapping[key]
        parts.append(f"{len(key)}#{key}")
        parts.append(f"{len(value)}#{value}")
    return "".join(parts)

def deserialize(data):
    """Reconstruct the original map from a serialized string."""
    # L = length of the serialized string.
    # Time:  $O(L)$  – one left-to-right scan, reading each character a constant
    # number of times. Space:  $O(L)$  – the rebuilt keys, values, and map.
    mapping = {}
    pos = [0]
    count = _read_length(data, pos)
    for pair_number in range(count):
        key = _read_field(data, pos)
        value = _read_field(data, pos)
        mapping[key] = value
    return mapping

def _read_length(s, pos):
    """Read a run of digits terminated by '#', returning the int; advance
    past '#'."""
    # d = number of digit characters in this length token.
    # Time:  $O(d)$  – walks the digits once. Space:  $O(1)$ .
    start = pos[0]
    while pos[0] < len(s) and s[pos[0]] != "#":
        ch = s[pos[0]]
        if ch < "0" or ch > "9":
            raise ValueError(f"Malformed: expected digit at index {pos[0]}")
        pos[0] += 1
    if pos[0] >= len(s) or start == pos[0]:
        raise ValueError("Malformed: missing length or '#')
    length = int(s[start:pos[0]])
    pos[0] += 1 # skip the '#'
    return length

def _read_field(s, pos):
    """Read a length-prefixed field: <len>#<exactly len chars>."""
    # f = length of this field's content (taken from its length prefix).
    # Time:  $O(f)$  – reads the prefix and slices f characters.
    # Space:  $O(f)$  – the slice.
    length = _read_length(s, pos)
    if pos[0] + length > len(s):
        raise ValueError("Malformed: field length exceeds input")
    field = s[pos[0]:pos[0] + length]
    pos[0] += length
```

```

    return field

def solution(operation, data):
    """Judge-style entry point: "serialize" a dict -> str, or "deserialize" a
    str -> dict."""
    # Time/Space: O(1) dispatch that delegates to serialize or deserialize, so
    # the cost is whichever of those runs (see their complexity notes above).
    if operation == "serialize":
        return serialize(data)
    if operation == "deserialize":
        return deserialize(data)
    raise ValueError(f"Unknown operation: {operation}")

if __name__ == "__main__":
    # ----- serialize -----
    print(solution("serialize", {})) # 0#
    encoded = solution("serialize", {"a": "hi", "b": ""})
    print(encoded) # 2#1#a2#hi1#b0#
    print(serialize({"key": "value"})) # 1#3#key5#value
    print(serialize({"": ""})) # 1#0#0#
    print(serialize({"a#b": "c#d", "x": "y"})) # 2#3#a#b3#c#d1#x1#y
    print(serialize({"n": "12#34"})) # 1#1#n5#12#34

    # ----- deserialize (round-trips) -----
    print(solution("deserialize", encoded)) # {'a': 'hi', 'b': ''}
    print(deserialize("0#")) # {}
    print(deserialize("1#0#0#")) # {'': ''}
    # -> {'a#b': 'c#d', 'x': 'y'}
    print(deserialize(serialize({"a#b": "c#d", "x": "y"})))
    print(deserialize(serialize({"n": "12#34"}))) # {'n': '12#34'}

    # ----- malformed input raises ValueError -----
    try:
        deserialize("")
    except ValueError as err:
        print("error:", err) # error: Malformed: missing length or '#'
    try:
        deserialize("xyz")
    except ValueError as err:
        print("error:", err) # error: Malformed: expected digit at index 0
    try:
        deserialize("1#5#ab")
    except ValueError as err:
        print("error:", err) # error: Malformed: field length exceeds input

    # ----- unknown operation raises ValueError -----
    try:
        solution("froblicate", {})
    except ValueError as err:
        print("error:", err) # error: Unknown operation: frobnicate

```

monster_battle.py

```
"""Simulate a battle between two ordered teams of monsters.
```

Overview:

Two teams send their monsters into the ring one champion at a time. The team passed first attacks first, then turns alternate. A monster whose life reaches 0 is defeated and replaced by the next monster in its team. The first team to run out of monsters loses.

Interface:

```
Monster(name, life_points, attack, type_=None, weakness=None)
    A combatant. `type_` is stored as attribute `type`; `type`/`weakness`
    are optional strings consumed by the damage modifiers.
Team(name, monsters)
    A named, ordered list of Monster objects.
Outcome(winning_team_name, log, is_stalemate=False)
    Result of a battle. `winning_team_name` is the winner's name, or None
    on a stalemate; `log` is a list of event strings; `is_stalemate` is
    True only for a stalemate.
battle(team1, team2) -> Outcome
    Runs the fight (team1 attacks first) and returns the Outcome. Both
    teams are cloned, so the caller's monsters are never mutated.
```

Semantics / rules:

- Damage: a hit subtracts the attacker's `attack` from the defender's life, floored at 0. It is doubled ("super effective") when the attacker's type equals the defender's weakness, and halved with round-down ("not very effective") when the defender's type equals the attacker's weakness.
- Stalemate: before each hit, if neither current monster can damage the other (both effective damages are 0), the fight cannot progress and battle returns Outcome(None, log, is_stalemate=True) instead of looping forever.

Example:

```
battle(Team("Blue", [Monster("Dog", 3, 2)]),
       Team("Red", [Monster("Cat", 3, 3)])) -> Outcome with
winning_team_name == "Red" (Dog hits for 2, Cat hits for 3 and wins).
"""
```

class Monster:

```
"""A combatant with life, attack, and an optional type/weakness."""

def __init__(self, name, life_points, attack, type_=None, weakness=None):
    # Time: O(1), Space: O(1) - stores its fields.
    self.name = name
    self.life_points = life_points
    self.attack = attack
    self.type = type_
    self.weakness = weakness
```

class Team:

```
"""A named, ordered roster of monsters."""

def __init__(self, name, monsters):
    # Time: O(1), Space: O(1) - keeps a reference to the roster.
    self.name = name
    self.monsters = monsters
```

```
class Outcome:
    """The battle result: the winning team's name (None on a stalemate), the
    event log, and a stalemate flag."""

    def __init__(self, winning_team_name, log, is_stalemate=False):
        # Time: 0(1), Space: 0(1).
        self.winning_team_name = winning_team_name
        self.log = log
        self.is_stalemate = is_stalemate

def _clone_monsters(monsters):
    """Copy the monsters so a battle never mutates the caller's teams."""
    # Time: 0(m) for m monsters. Space: 0(m) – the new list.
    clones = []
    for monster in monsters:
        clones.append(Monster(monster.name, monster.life_points,
                               monster.attack, monster.type, monster.weakness))
    return clones

def _effective_damage(attacker, defender):
    """Damage the attacker deals, plus a modifier label. Doubled when the
    attacker's type is the defender's weakness, halved when the defender's type
    is the attacker's weakness. modifier is 'super effective', 'not very
    effective', or None."""
    # Time: 0(1), Space: 0(1) – a couple of comparisons.
    base = attacker.attack
    if attacker.type is not None and attacker.type == defender.weakness:
        return base * 2, "super effective"
    if defender.type is not None and defender.type == attacker.weakness:
        return base // 2, "not very effective" # halved, rounded down
    return base, None

def _describe(monster):
    """'Name (Type)' when the monster has a type, else just 'Name'."""
    # Time: 0(1), Space: 0(1).
    if monster.type is not None:
        return f"{monster.name} ({monster.type})"
    return monster.name

def _attack_message(attacker, defender, damage, modifier):
    """One attack log line, noting the modifier when one applied."""
    # Time: 0(1), Space: 0(1) – builds one string.
    message = (f"{_describe(attacker)} attacks {_describe(defender)} "
               f"for {damage} damage")
    if modifier is not None:
        message += f" ({modifier})"
    return message

# Approach (in plain terms):
#   Picture two line-ups of monsters, one champion from each side in the ring
#   at a time. The teams take turns; on its turn the active team's champion
#   hits the enemy champion, subtracting its attack from their life (types can
```

monster_battle.py (continued)

```
# double or halve the hit). When a champion's life hits 0 it is knocked out
# and the next monster in that line-up steps in. The first team to run out of
# monsters loses. Before each hit we check the current pair: if neither can
# dent the other (both would deal 0 damage), the fight is stuck, so we stop
# and call it a stalemate instead of looping forever.
# Data structures used:
#   - two ordered lists of monsters, each with an integer index
#     pointer to the current champion - the pointer advances to
#     the next monster on defeat.
#   - a list that accumulates the event log in order.
def battle(team1, team2):
    """Simulate team1 vs team2 (team1 attacks first). Returns an Outcome with
    the winner's name and an event log, or a stalemate Outcome if the fight
    cannot progress."""
    # t = total monsters, L = total starting life across both teams.
    # Time: O(t + L) - each landed hit removes >= 1 life; matchups advance at
    # most t times. Space: O(t + L) - the working copies plus the log.
    log = []
    fighters1 = _clone_monsters(team1.monsters)
    fighters2 = _clone_monsters(team2.monsters)
    index1 = 0
    index2 = 0
    team1_attacks = True                # team1 (first argument) attacks first

    while index1 < len(fighters1) and index2 < len(fighters2):
        blue = fighters1[index1]
        red = fighters2[index2]

        # Stalemate: if neither current monster can hurt the other, stop.
        if (_effective_damage(blue, red)[0] == 0
            and _effective_damage(red, blue)[0] == 0):
            log.append("Stalemate - neither team can make progress")
            return Outcome(None, log, is_stalemate=True)

        if team1_attacks:
            attacker_team, attacker, defender = team1, blue, red
        else:
            attacker_team, attacker, defender = team2, red, blue

        log.append(f"{attacker_team.name} turn")
        damage, modifier = _effective_damage(attacker, defender)
        defender.life_points = max(0, defender.life_points - damage)
        log.append(_attack_message(attacker, defender, damage, modifier))

        if defender.life_points == 0:
            log.append(f"{defender.name} is defeated")
            if team1_attacks:
                index2 += 1                # team2's champion falls
            else:
                index1 += 1                # team1's champion falls

        team1_attacks = not team1_attacks  # turns alternate

    winner = team2 if index1 >= len(fighters1) else team1
    log.append(f"{winner.name} wins")
    return Outcome(winner.name, log)

if __name__ == "__main__":
```

monster_battle.py (continued)

```
# ----- Example: basic battle (from the problem statement) -----
blue_team = Team("Team Blue",
                 [Monster("Dog", 3, 2), Monster("Wolf", 4, 1)])
red_team = Team("Team Red",
               [Monster("Cat", 3, 3), Monster("Tiger", 4, 5)])
result = battle(blue_team, red_team)
for line in result.log:
    print(line)
print("winner:", result.winning_team_name)
# Team Blue turn / Dog attacks Cat for 2 damage / Team Red turn /
# Cat attacks Dog for 3 damage / Dog is defeated / Team Blue turn /
# Wolf attacks Cat for 1 damage / Cat is defeated / Team Red turn /
# Tiger attacks Wolf for 5 damage / Wolf is defeated / Team Red wins
# winner: Team Red

print("----")

# ----- Follow-up: type advantages (super / not very effective) -----
fire_team = Team("Fire", [Monster("Dragon", 12, 5, "Fire", "Water")])
earth_team = Team("Earth", [Monster("Troll", 20, 4, "Earth", "Fire")])
result = battle(fire_team, earth_team)
for line in result.log:
    print(line)
print("winner:", result.winning_team_name)
# Dragon (Fire) attacks Troll (Earth) for 10 damage (super effective)
# Troll (Earth) attacks Dragon (Fire) for 2 damage (not very effective)
# ... Fire wins

print("----")

# ----- Stalemate: neither monster can deal damage -----
rocks = Team("Rocks", [Monster("Pebble", 5, 0)])
stones = Team("Stones", [Monster("Boulder", 5, 0)])
result = battle(rocks, stones)
for line in result.log:
    print(line)
print("stalemate:", result.is_stalemate)
print("winner:", result.winning_team_name)
# Stalemate - neither team can make progress
# stalemate: True
# winner: None
```

path_resolution.py

```
"""Resolve a shell-style `cd` and return the destination path.
```

Overview:

Given the current working directory and a `cd` argument, compute the resulting absolute path. The problem grows in three parts, solved here as three independent functions that share the same path helpers.

Interface:

```
cd_part1(current_dir, new_dir) -> str | None
    Standard traversal. A normal name steps into a folder, '.' stays put,
    '..' goes to the parent, and a leading '/' makes new_dir absolute
    (resolved from the root). Rising above the root returns None.
cd_part2(current_dir, new_dir) -> str | None
    Part 1 plus '~', which expands to HOME (/home/user) as an absolute
    path ('~' -> HOME, '~/x' -> HOME + '/x').
cd_part3(current_dir, new_dir, soft_links=None) -> str | None
    Part 2 plus symbolic links. soft_links is a dict of path -> target.
    After normalizing '.' and '..', the path prefix is rewritten through
    soft_links using the LONGEST matching key, repeating so links that
    point to other links cascade. A key matches only on a folder boundary
    ('/a' matches '/a' and '/a/b', not '/ab'). Links that loop give None.
```

Semantics / rules:

- All three return an absolute path string, or None when the walk rises above the root (or, for part 3, when the links form a cycle).
- soft_links defaults to None; when falsy, cd_part3 behaves like Part 2.

Example:

```
cd_part1("/foo/bar", "baz") -> "/foo/bar/baz"
cd_part2("/foo/bar", "~") -> "/home/user"
cd_part3("/foo/bar", "baz", {"/foo/bar": "/abc"}) -> "/abc/baz"
"""
```

```
HOME = "/home/user"      # where a leading '~' expands to
_MAX_LINK_HOPS = 1000    # safety net against self-growing link chains

# ----- Part 1: standard traversal -----
# Approach (in plain terms):
#   Think of a path as a stack of folder names. Start from the current
#   directory, unless new_dir is absolute (begins with '/') - then start from
#   the root instead. Read new_dir piece by piece:
#   - a normal name is pushed on the stack (step into that folder),
#   - '.' means stay put,
#   - '..' pops the top folder (go up); rising above the root is illegal,
#   so we return None.
# Data structures used:
#   - a list used as a stack of path folders - push a name, pop
#   on '..' (exactly stack behavior).
def cd_part1(current_dir, new_dir):
    """Part 1: resolve '.', '..', and absolute '/'; None above root."""
    # p = total path length. Time: O(p) - one pass. Space: O(p) - the stack.
    if new_dir.startswith("/"):
        base = []                      # absolute new_dir -> start from root
    else:
        base = _resolve_stack([], _to_components(current_dir))
        if base is None:
            return None
    stack = _resolve_stack(base, _to_components(new_dir))
    if stack is None:
```

path_resolution.py (continued)

```
        return None
    return _join(stack)

def _resolve_stack(base, tokens):
    """Apply tokens to a copy of base, handling '.' and '..'. Returns the
    resulting component list, or None if '..' rises above the root."""
    # b = len(base), t = len(tokens).
    # Time: O(b + t) - copy the base, then one pass over the tokens.
    # Space: O(b + t) - the working stack.
    stack = list(base)
    for token in tokens:
        if token == "." or token == "":
            continue
        if token == "..":
            if not stack:
                return None # cannot go above the root
            stack.pop()
        else:
            stack.append(token)
    return stack

def _to_components(path):
    """Split a path on '/', dropping empty pieces (keeps '.' and '..')."""
    # Time: O(L) over the L characters of path. Space: O(L) - the parts list.
    parts = []
    for token in path.split("/"):
        if token != "":
            parts.append(token)
    return parts

def _join(components):
    """Join components into an absolute path ('/' when empty)."""
    # Time: O(total characters). Space: O(total characters) - the new string.
    if not components:
        return "/"
    return "/" + "/".join(components)

# ----- Part 2: home directory (~) -----
# Approach (in plain terms):
#   Same as Part 1, but first swap a leading '~' for the home folder. That
#   turns a '~' path into an ordinary absolute path; the rest is unchanged.
#   Data structures used:
#     - the same folder stack; a leading '~' is swapped with plain
#       string ops - reuse the stack, '~' only changes the starting
#       point.
def cd_part2(current_dir, new_dir):
    """Part 2: Part 1 plus '~' expanding to HOME (an absolute path)."""
    # p = total path length. Time: O(p). Space: O(p) - the stack.
    current = _expand_home(current_dir)
    target = _expand_home(new_dir)
    if target.startswith("/"):
        base = [] # absolute new_dir -> start from root
    else:
        base = _resolve_stack([], _to_components(current))
        if base is None:
            return None
    stack = _resolve_stack(base, _to_components(target))
    if stack is None:
        return None
```

path_resolution.py (continued)

```
    return _join(stack)

def _expand_home(path):
    """Expand a leading '~' to HOME ('~' -> HOME, '~/x' -> HOME + '/x')."""
    # Time: O(L) over the L characters of path. Space: O(L) - the new string.
    if path == "~":
        return HOME
    if path.startswith("~/"):
        return HOME + path[1:] # path[1:] keeps its leading '/'
    return path

# ----- Part 3: symbolic links -----
# Approach (in plain terms):
#   Build the plain path exactly like Part 2, then keep rewriting it: find the
#   LONGEST link key that matches the front of the path, swap that prefix for
#   the link's target, and repeat (so links pointing to other links unwind).
#   Remember every path seen; if one comes back, the links loop -> None.
#   Data structures used:
#       - a dict (soft_links) for O(1) link lookup, plus string
#       - prefix checks to pick the longest matching key.
#       - a set of already-seen paths for O(1) cycle detection.
def cd_part3(current_dir, new_dir, soft_links=None):
    """Part 3: Part 2 plus symbolic-link resolution (longest match, cascade,
    cycle detection -> None)."""
    # p = path length, n = number of links, d = link-chain depth.
    # Time: O(p + d * n * p). Space: O(p + d).
    current = _expand_home(current_dir)
    target = _expand_home(new_dir)
    if target.startswith("/"):
        base = [] # absolute new_dir -> start from root
    else:
        base = _resolve_stack([], _to_components(current))
        if base is None:
            return None
    stack = _resolve_stack(base, _to_components(target))
    if stack is None:
        return None
    path = _join(stack)
    if soft_links:
        path = _resolve_links(path, soft_links)
    return path

def _resolve_links(path, soft_links):
    """Follow soft_links until the path stops changing. Each step swaps the
    LONGEST matching key at the front of the path for its target. Returns None
    if the links form a cycle or never settle (keep growing)."""
    # n = links, p = path length, d = number of swaps before it settles.
    # Time: O(d * n * p) - each swap scans every link. Space: O(d + p).
    seen = set()
    # hops = 0
    while path not in seen:
        seen.add(path)
        # hops += 1
        # if hops > _MAX_LINK_HOPS:
        #     return None # runaway growth, e.g. '/a' -> '/a/b'
        key = _longest_matching_link(path, soft_links)
        if key is None:
            return path # nothing matches -> this is the final path
```

path_resolution.py (continued)

```
    # Swap the matched prefix (key) for its target, keep the rest.
    path = soft_links[key] + path[len(key):]
    return None    # we came back to a path we've seen -> a cycle

def _longest_matching_link(path, soft_links):
    """The soft-link key that matches the most of path's leading folders, or
    None. A key matches only on a folder boundary: '/a' matches '/a' and
    '/a/b', but not '/ab'."""
    # n = number of links, p = path length. Time: O(n * p). Space: O(1).
    best_key = None
    for key in soft_links:
        if path == key or path.startswith(key + "/"):
            if best_key is None or len(key) > len(best_key):
                best_key = key
    return best_key

if __name__ == "__main__":
    # ----- Part 1: standard traversal -----
    print(cd_part1("/foo/bar", "baz"))    # /foo/bar/baz
    print(cd_part1("/foo/./", "./baz"))    # /baz
    print(cd_part1("/", "foo/bar/./../baz"))    # /baz
    print(cd_part1("/", ".."))    # None (above root)
    print(cd_part1("/foo", "/bar/qux"))    # /bar/qux (absolute)

    # ----- Part 2: home directory (~) -----
    print(cd_part2("/foo/bar", "~"))    # /home/user
    print(cd_part2("/tmp", "~/docs/../pics"))    # /home/user/pics
    print(cd_part2("~", ".."))    # /home

    # ----- Part 3: symbolic links -----
    print(cd_part3("/foo/bar", "baz", {"/foo/bar": "/abc"}))    # /abc/baz

    cascade = {"/foo/bar": "/abc", "/abc": "/bcd", "/bcd/baz": "/xyz"}
    print(cd_part3("/foo/bar", "baz", cascade))    # /xyz

    longest = {"/foo/bar": "/abc", "/foo/bar/baz": "/xyz"}
    print(cd_part3("/foo/bar", "baz", longest))    # /xyz

    cycle = {"/x": "/y", "/y": "/x"}
    print(cd_part3("/", "x", cycle))    # None (cycle)
```

persistent_kv_store.py

"""Persistent key-value store with a binary-safe serialization format.

Overview:

Keys and values are Python str or bytes blobs. Persistence uses LENGTH-PREFIXED encoding (no delimiters), so a key or value may hold ANY byte (including '|', ':', spaces, newlines):

```
field = [type:1][len:4 big-endian][raw bytes]
```

```
map   = [count:4][field key][field value] x count
```

The type byte records whether a field was str (1) or bytes (0), so a str round-trips as str (UTF-8) and bytes round-trips as bytes.

Interface (class PersistentKvStore, all @staticmethod):

- simulate_part1(operations) -> list
Run put/delete/get/serialize/deserialize ops against one live map plus a fake disk (label -> blob). Returns the list of get results in order (a value, or None if the key is absent).
- simulate_part2(max_segment_size, operations) -> list
Adds serialize/segment_count/reorder to Part 1. serialize splits the snapshot into size-bounded segments, each stamped [total:4][index:4][chunk]; reorder physically shuffles them yet deserialize still rebuilds the map because the index header, not position, fixes order. Returns get and segment_count results.
- simulate_part3(compact_threshold, operations) -> list
Keeps a saved snapshot plus an append-only log of (put/delete) changes. 'restart' reloads the snapshot then replays the log; once the log reaches compact_threshold records it is folded into a fresh snapshot (compaction) and cleared. Returns get results and status triples (live keys, pending log records, compactions).
- by(s) -> bytes: demo helper, str -> latin-1 bytes (1 char == 1 byte).
- py_repr(results) -> str: render a results list Python-style (str -> 'x', bytes -> b'x', None -> None, tuple -> (...)).

Semantics / rules:

- get on a missing (or deleted) key yields None; delete of a missing key is a harmless no-op.
- deserialize replaces the whole live map with a decoded snapshot, so it can undo puts made after that snapshot was taken.
- An unknown operation name raises ValueError.

Constraints / assumptions:

- Part 2 assumes max_segment_size >= 9 so each chunk holds >= 1 byte after its 8-byte header.
- Operations are given as sequences whose first element names the op.

Example (Part 1):

```
ops = [['put','a','1'], ['get','a'], ['serialize','snap1'],  
       ['put','a','2'], ['deserialize','snap1'], ['get','a']]  
simulate_part1(ops) -> ['1', '1']  # the second get sees the snapshot  
"""
```

```
# ----- binary-safe codec -----
```

```
def _write_int(out, v):
```

```
    """Append a 4-byte big-endian integer (length or count) to the buffer."""
```

```
    # Time: O(1), Space: O(1) - writes a fixed 4 bytes.
```

```
    out.extend(v.to_bytes(4, "big"))
```

persistent_kv_store.py (continued)

```
def _write_field(out, field):
    """Append one length-prefixed field: [type:1][len:4][raw bytes]."""
    # n = number of bytes in the field.
    # Time: O(n) - encode (if str) and copy n bytes.
    # Space: O(n) - the encoded copy.
    is_str = isinstance(field, str)
    data = field.encode("utf-8") if is_str else field
    out.append(1 if is_str else 0)
    _write_int(out, len(data))
    out.extend(data)

def _encode_map(mapping):
    """Serialize a map to a binary-safe blob."""
    # n = number of entries, b = total bytes across all keys and values.
    # Time: O(n + b) - one field per key and value. Space: O(n + b) - the blob.
    out = bytearray()
    _write_int(out, len(mapping))
    for key, value in mapping.items():
        _write_field(out, key)
        _write_field(out, value)
    return bytes(out)

class _Reader:
    """Sequential reader over a blob produced by _encode_map."""

    def __init__(self, blob):
        # Time: O(1), Space: O(1) - stores the blob and a cursor position.
        self._b = blob
        self._i = 0

    def read_int(self):
        # Time: O(1), Space: O(1) - reads a fixed 4-byte integer.
        v = int.from_bytes(self._b[self._i:self._i + 4], "big")
        self._i += 4
        return v

    def read_field(self):
        # n = length of this field. Time: O(n) - slice n bytes (plus a UTF-8
        # decode for str). Space: O(n) - the returned bytes/str.
        tag = self._b[self._i]
        self._i += 1
        length = self.read_int()
        data = bytes(self._b[self._i:self._i + length])
        self._i += length
        return data.decode("utf-8") if tag == 1 else data

def _decode_map(blob):
    """Rebuild a map from a blob produced by _encode_map."""
    # n = number of entries, b = total key/value bytes.
    # Time: O(n + b) - read every field once.
    # Space: O(n + b) - the rebuilt map.
    reader = _Reader(blob)
    count = reader.read_int()
    mapping = {}
    for _ in range(count):
        key = reader.read_field()
```

persistent_kv_store.py (continued)

```
        value = reader.read_field()
        mapping[key] = value
    return mapping

def _read_int_at(b, off):
    """Read a 4-byte big-endian integer at a fixed offset in a blob."""
    # Time: O(1), Space: O(1) – reads 4 bytes.
    return int.from_bytes(b[off:off + 4], "big")

# ----- Part 2 segment helpers -----
_SEG_HEADER = 8 # [total:4][index:4]

def _segment_index(segment):
    """Sort key: a segment's stored index header (bytes 4..8)."""
    # Time: O(1), Space: O(1) – reads one 4-byte integer header.
    return _read_int_at(segment, 4)

def _split_into_segments(snap, max_segment_size):
    """Split a snapshot blob into size-bounded, index-stamped segments."""
    # s = len(snap), t = number of segments produced.
    # Time: O(s) – copy the snapshot into chunks. Space: O(s) – the segments
    # together hold the whole snapshot (plus t small headers).
    chunk = max_segment_size - _SEG_HEADER # >= 1 since max_segment_size >= 9
    total = max(1, (len(snap) + chunk - 1) // chunk)
    segs = []
    for idx in range(total):
        off = idx * chunk
        length = min(chunk, len(snap) - off)
        seg = bytearray()
        _write_int(seg, total)
        _write_int(seg, idx)
        if length > 0:
            seg.extend(snap[off:off + length])
        segs.append(bytes(seg))
    return segs

def _join_segments(segments):
    """Reassemble the snapshot from segments given in any order."""
    # t = number of segments, s = total payload bytes.
    # Time: O(t log t + s) – sort by index header, then concatenate.
    # Space: O(s).
    # Segments may be in any order; sort by their stored index header.
    ordered = sorted(segments, key=_segment_index)
    out = bytearray()
    for seg in ordered:
        out.extend(seg[_SEG_HEADER:])
    return bytes(out)

# Approach (in plain terms):
# We need to save a dictionary to "disk" and read it back, even when
# keys or values contain tricky bytes (spaces, '|', ':', newlines). The
# trick: never rely on separators. Instead, right before every chunk we
# write its exact length – like labelling a parcel "5 bytes follow". The
```

persistent_kv_store.py (continued)

```
# reader then grabs exactly that many bytes, so nothing inside can ever be
# mistaken for a separator (binary-safe).
# - Part 1: write the whole map as one length-prefixed blob and
#   read it back.
# - Part 2: chop that blob into fixed-size parcels, each stamped
#   with its index, so they can be stored or shuffled and later
#   sorted back into the right order.
# - Part 3: keep a saved snapshot plus a running list (the log) of
#   recent changes. On restart, load the snapshot then re-apply the
#   log. When the log grows past a threshold, fold everything into a
#   fresh snapshot (compaction) and clear the log.
# Data structures used:
# - dict - the in-memory key/value store.
# - bytearray - length-prefixed, binary-safe encoding.
# - lists - the on-disk segments and the pending append-only
#   log for replay.
class PersistentKvStore:

    @staticmethod
    def by(s):
        """Demo helper: Python bytes from a str (ISO-8859-1, 1 char == 1
        byte)."""
        # n = len(s). Time: O(n) - encode n characters.
        # Space: O(n) - the bytes.
        return s.encode("latin-1")

    @staticmethod
    def py_repr(results):
        """Render a results list Python-style (str -> 'x', bytes -> b'x',
        None -> None, tuple -> (...))."""
        # n = total size of results.
        # Time: O(n), Space: O(n) - builds a repr string.
        return repr(results)

# ----- Part 1: serialize / deserialize a single snapshot -----
@staticmethod
def simulate_part1(operations):
    """Part 1: serialize/deserialize a whole snapshot to and from a
    label."""
    # m = number of ops, s = current store's serialized size,
    # r = get-results.
    # Time: O(m * s) worst case (serialize/deserialize copy the whole
    # store).
    # Space: O(s + r) - a snapshot blob plus the results list.
    store = {}
    disk = {} # fake disk: label -> blob
    results = []
    for op in operations:
        kind = op[0]
        if kind == "put":
            store[op[1]] = op[2]
        elif kind == "delete":
            store.pop(op[1], None)
        elif kind == "get":
            results.append(store.get(op[1])) # value or None
        elif kind == "serialize":
            disk[op[1]] = _encode_map(store)
        elif kind == "deserialize":
            store = _decode_map(disk[op[1]])
```

```

        else:
            raise ValueError(f"Unknown op: {kind}")
        return results

# ----- Part 2: snapshot split across size-bounded segments -----
@staticmethod
def simulate_part2(max_segment_size, operations):
    """Part 2: serialize into size-bounded segments that survive
    reordering."""
    # m = number of ops, s = serialized store size,
    # t = segments per snapshot.
    # Time: O(m * (s + t log t)) worst case.
    # Space: O(s + r) - segments plus the r results.
    store = {}
    disk = {} # prefix -> list of segment blobs
    results = []
    for op in operations:
        kind = op[0]
        if kind == "put":
            store[op[1]] = op[2]
        elif kind == "delete":
            store.pop(op[1], None)
        elif kind == "get":
            results.append(store.get(op[1]))
        elif kind == "serialize":
            disk[op[1]] = _split_into_segments(
                _encode_map(store), max_segment_size
            )
        elif kind == "segment_count":
            results.append(len(disk[op[1]]))
        elif kind == "reorder":
            cur = disk[op[1]]
            order = op[2]
            # Physical shuffle; the index header preserves the true order.
            shuffled = []
            for idx in order:
                shuffled.append(cur[idx])
            disk[op[1]] = shuffled
        elif kind == "deserialize":
            store = _decode_map(_join_segments(disk[op[1]]))
        else:
            raise ValueError(f"Unknown op: {kind}")
    return results

# ----- Part 3: snapshot + append-only log with replay & compaction -----
@staticmethod
def simulate_part3(compact_threshold, operations):
    """Part 3: snapshot + append-only log, replayed on restart and
    folded in on compaction."""
    # m = number of ops, s = store size, L = pending log length,
    # r = results.
    # Time: O(m * s) worst case (compaction/restart re-encode or replay).
    # Space: O(s + L + r) - snapshot, log, and results.
    store = {}
    # Persisted artifacts (survive a restart): a snapshot blob + a
    # pending log.
    snapshot = [_encode_map(store)]
    log = [] # list of (is_put, key, val); val is None for delete
    compactions = [0]

```

persistent_kv_store.py (continued)

```
results = []

def maybe_compact():
    # Time: O(s) when it compacts (re-encode the whole store),
    # else O(1).
    # Space: O(s) for the fresh snapshot when compaction happens.
    if len(log) >= compact_threshold:
        snapshot[0] = _encode_map(store) # fresh full snapshot
        log.clear()                      # clear the pending log
        compactions[0] += 1

for op in operations:
    kind = op[0]
    if kind == "put":
        k, v = op[1], op[2]
        store[k] = v
        log.append((True, k, v))
        maybe_compact()
    elif kind == "delete":
        k = op[1]
        store.pop(k, None)
        log.append((False, k, None))
        maybe_compact()
    elif kind == "get":
        results.append(store.get(op[1]))
    elif kind == "restart":
        # Throw away memory; reload the snapshot, then replay the log.
        store = _decode_map(snapshot[0])
        for is_put, k, v in log:
            if is_put:
                store[k] = v
            else:
                store.pop(k, None)
    elif kind == "status":
        # (live keys, pending log records, snapshots written)
        results.append((len(store), len(log), compactions[0]))
    else:
        raise ValueError(f"Unknown op: {kind}")
return results

if __name__ == "__main__":
    by = PersistentKvStore.by

    # ----- Part 1: single-snapshot serialize / deserialize -----
    # Test 1: deserialize restores an older snapshot, undoing a later put.
    kv1 = [
        ["put", "a", "1"], ["get", "a"], ["serialize", "snap1"],
        ["put", "a", "2"], ["deserialize", "snap1"], ["get", "a"],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1)))
    # ['1', '1']

    # Test 2: get a missing key -> None; delete then get -> None.
    kv1b = [["get", "nope"], ["put", "a", "1"], ["delete", "a"], ["get", "a"]]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1b)))
    # [None, None]

    # Test 3: binary-safe - keys/values with '|', ':' and a newline round-trip.
```

persistent_kv_store.py (continued)

```
kv1c = [
    ["put", by("a|b"), by("v:1\n")], ["serialize", "s"],
    ["put", by("a|b"), by("zzz")], ["deserialize", "s"],
    ["get", by("a|b")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1c)))
# [b'v:1\n']

# Test 4: a str value stays str and a bytes value stays bytes.
kv1d = [
    ["put", "k", "x"], ["put", by("k2"), by("x")],
    ["get", "k"], ["get", by("k2")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1d)))
# ['x', b'x']

# ----- Part 2: snapshot split into reorderable segments -----
# Test 5: count segments, then read values back after a shuffle-proof join.
kv2 = [
    ["put", by("a"), by("12345")], ["put", by("b"), by("67890")],
    ["serialize", "p"], ["segment_count", "p"], ["delete", by("a")],
    ["deserialize", "p"], ["get", by("a")], ["get", by("b")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part2(15, kv2)))
# [6, b'12345', b'67890']

# Test 6: reversing the segment order still reassembles the snapshot
# correctly.
kv2b = [
    ["put", by("k1"), by("AAAA")], ["put", by("k2"), by("BBBB")],
    ["serialize", "p"], ["segment_count", "p"],
    ["reorder", "p", [2, 1, 0]],
    ["deserialize", "p"], ["get", by("k1")], ["get", by("k2")],
]
print(PersistentKvStore.py_repr(
    PersistentKvStore.simulate_part2(20, kv2b)))
# [3, b'AAAA', b'BBBB']

# Test 7: an empty store still serializes to exactly one segment.
kv2c = [
    ["serialize", "e"], ["segment_count", "e"],
    ["deserialize", "e"], ["get", by("x")],
]
print(PersistentKvStore.py_repr(
    PersistentKvStore.simulate_part2(15, kv2c)))
# [1, None]

# ----- Part 3: snapshot + append-only log with replay & compaction -----
# Test 8: restart with no compaction yet -> empty snapshot + replayed log.
kv3 = [
    ["put", "a", "1"], ["put", "b", "2"], ["restart"],
    ["get", "a"], ["get", "b"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3)))
# ['1', '2', (2, 2, 0)]

# Test 9: hitting the threshold compacts, clearing the log and
# bumping the count.
kv3b = [
```

persistent_kv_store.py (continued)

```
        ["put", "a", "1"], ["put", "b", "2"],
        ["put", "c", "3"], ["status"],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3b)))
    # [(3, 0, 1)]

    # Test 10: restart after a compaction reloads the snapshot then
    # replays the log.
    kv3c = [
        # compaction here
        ["put", "a", "1"], ["put", "b", "2"], ["put", "c", "3"],
        ["put", "d", "4"], ["delete", "a"], ["restart"],
        ["get", "a"], ["get", "d"], ["status"],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3c)))
    # [None, '4', (3, 2, 1)]

    # Test 11: binary-safe snapshot survives compaction + restart
    # (mixed str/bytes).
    kv3d = [
        ["put", by("k|1"), by("v:1")], ["put", "x", "y"], ["put", "p", "q"],
        ["restart"], ["get", by("k|1")],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3d)))
    # [b'v:1']
```

plant_infection.py

```
"""Compute Plant Infection Time (multi-source BFS, "Rotting Oranges").
```

Overview:

A 2D grid holds plants and obstacles. Every infected plant spreads to its orthogonally adjacent (up/down/left/right) healthy plants once per minute, with all infected plants spreading simultaneously. Find how long until the infection can no longer progress.

Interface:

```
infection_time(grid) -> int
    grid is a list of equal-length rows of ints. Returns the number of
    minutes elapsed. The input grid is mutated in place (infected cells
    double as the visited marker).
```

Cell values:

0 = empty ground, 1 = healthy plant, 2 = infected plant, -1 = wall.

Semantics / rules:

- Returns the minutes until no healthy plant can be infected.
- Returns -1 if at least one healthy plant is unreachable (walled off, or with no path from any infected plant).
- Returns 0 when there are no healthy plants to begin with, including an empty grid ([] or [[]]).

Example:

```
infection_time([[2, 1, 1], [1, 1, 0], [0, 1, 1]]) -> 4
infection_time([[2, 1, -1], [-1, 1, 1], [1, -1, 1]]) -> -1
"""
from collections import deque
```

Approach (in plain terms):

```
# Picture every infected plant as a small fire that all start spreading at
# the same moment. Each minute, every fire jumps to the healthy plants
# directly next to it (up, down, left, right). We flood outward one ring at a
# time and count the minutes as we go, stopping once no healthy plant can
# catch fire. If every healthy plant was reached, the answer is the minutes
# it took; if a plant is walled off and can never be reached, we return -1;
# and if there were no healthy plants to begin with, it takes 0 minutes.
```

Data structures used:

```
# - deque - the BFS frontier; O(1) pop/left spreads outward
# level-by-level (minute-by-minute).
# - grid (2D list) - reused as the visited marker; overwriting
# infected cells avoids a separate visited set.
```

def infection_time(grid):

```
    """Minutes until every healthy plant is infected via multi-source BFS,
    -1 if some healthy plant is unreachable, or 0 if none need infecting."""
```

```
    # m = number of rows, n = number of columns.
```

```
    # Time: O(m * n) - each cell is enqueued and visited at most once.
```

```
    # Space: O(m * n) - the BFS queue can hold up to every cell.
```

```
    if not grid or not grid[0]:
```

```
        return 0 # no plants at all -> nothing to infect
```

```
    m, n = len(grid), len(grid[0])
```

```
    queue = deque()
```

```
    healthy = 0
```

```
    # Seed the BFS with every initially-infected plant; count healthy ones.
```

```
    for r in range(m):
```

```
        for c in range(n):
```

plant_infection.py (continued)

```
        if grid[r][c] == 2:
            queue.append((r, c))
        elif grid[r][c] == 1:
            healthy += 1
    if healthy == 0:
        return 0 # nothing healthy -> 0 minutes

    dirs = ((1, 0), (-1, 0), (0, 1), (0, -1))
    minutes = 0

    # Expand the infection one layer (one minute) at a time.
    while queue and healthy > 0:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:
                    grid[nr][nc] = 2 # infect (also marks visited)
                    healthy -= 1
                    queue.append((nr, nc))
        minutes += 1

    # Any healthy plant left was unreachable.
    return minutes if healthy == 0 else -1

if __name__ == "__main__":
    # Test 1: classic spread from one corner reaches every plant in 4 minutes.
    print(infection_time([[2, 1, 1], [1, 1, 0], [0, 1, 1]])) # 4

    # Test 2: walls trap some healthy plants -> unreachable, so -1.
    print(infection_time([[2, 1, -1], [-1, 1, 1], [1, -1, 1]])) # -1

    # Test 3: no healthy plants (only infected/empty) -> 0 minutes.
    print(infection_time([[0, 2, 0]])) # 0

    # Test 4: completely empty grid -> nothing to infect -> 0.
    print(infection_time([])) # 0

    # Test 5: grid with only an empty row -> still nothing to infect -> 0.
    print(infection_time([[ ]])) # 0

    # Test 6: healthy plants but no infection source -> all unreachable -> -1.
    print(infection_time([[1, 1], [1, 1]])) # -1

    # Test 7: every plant already infected -> 0 minutes needed.
    print(infection_time([[2, 2], [2, 2]])) # 0

    # Test 8: a wall seals off the lone healthy plant -> -1.
    print(infection_time([[2, -1, 1]])) # -1

    # Test 9: single infected cell, nothing healthy -> 0.
    print(infection_time([[2]])) # 0

    # Test 10: single healthy cell with no source -> unreachable -> -1.
    print(infection_time([[1]])) # -1

    # Test 11: single empty cell -> 0.
    print(infection_time([[0]])) # 0
```


resumable_iterator.py

```
"""Iterate a list of integer chunks value-by-value, pausable/resumable.
```

Overview:

The input is a list of chunks (each chunk a list of ints), some of which may be empty. The iterator walks it as one flat stream of values, skipping empty chunks entirely. Its distinguishing feature is that progress can be frozen into a small string token and later handed to a factory to rebuild an iterator that resumes at exactly the same spot.

Interface:

- ResumableIterator(data) -> a new iterator at the first value.
- hasNext() -> bool: True while at least one value remains.
- next() -> int: return the next value and advance; raises StopIteration (message "no more elements") once the chunks are exhausted.
- getState() -> str: an opaque "chunk:item" token naming the position of the NEXT value to be returned.
- fromState(data, state) -> ResumableIterator: static factory that rebuilds an iterator over data parked at a previously saved token.

Semantics and rules:

- Empty chunks contribute nothing and are transparently skipped, both at construction and after each next().
- The saved token always points at the next real value (empty and end-of-stream positions are normalized away), so a token captured after the last value simply resumes an exhausted iterator.
- fromState assumes the same data list the token was captured against; it does not validate the token beyond parsing "chunk:item".

Constraints and assumptions:

- Values are ints; the data list is read in place and never mutated.

Example:

```
ResumableIterator([[7, 8, 9], [10], [], [11, 12]]) yields
7, 8, 9, 10, 11, 12.
"""
```

```
def _drain_all(iterator):
```

```
    """Collect every remaining value from an iterator into a list."""
    # Time: O(m) for m remaining values. Space: O(m) for the result.
    values = []
    while iterator.hasNext():
        values.append(iterator.next())
    return values
```

```
# Approach (in plain terms):
```

```
# Picture a stack of numbered pages where each page holds a handful of
# numbers, and some pages are blank. We keep two counters: which page we
# are on and which number on that page comes next. After every read - and
# whenever we resume - we "normalize": if the current spot is past the end
# of its page (or the page is blank), we flip forward to the top of the
# next page, repeating until we land on a real number or run out of pages.
# That way the two counters always point at the very next number to hand
# out, so blank pages simply get skipped. Saving progress is just writing
# those two counters down as a small "page:number" note; resuming reads the
# note back and normalizes once so it lands on the right number.
# Data structures used:
#     - the input data (a list of int chunks) - read in place, never copied.
```

resumable_iterator.py (continued)

```
# - two integer indices (chunk, item) - the next value's position,
#   normalized to a real value or one-past-the-end.
# - a compact "chunk:item" string - the opaque, resumable state token.
class ResumableIterator:

    def __init__(self, data):
        """Start at the first value of data (a list of int chunks)."""
        # Time: O(c) worst case to skip leading empty chunks (c = chunks).
        # Space: O(1) - only two index counters are stored.
        self._data = data
        self._chunk = 0      # index of the chunk holding the next value
        self._item = 0       # index within that chunk of the next value
        self._normalize()

    def _normalize(self):
        """Advance the (chunk, item) cursor past any exhausted or empty
        chunks so it points at a real next value or one-past-the-end."""
        # Time: O(c) worst case across empty chunks (c = chunks).
        # Space: O(1) - adjusts the two counters in place.
        while self._chunk < len(self._data):
            chunk = self._data[self._chunk]
            if self._item < len(chunk):
                break
            self._chunk += 1
            self._item = 0

    def hasNext(self):
        """True while another value remains to be handed out."""
        # Time: O(1), Space: O(1).
        return self._chunk < len(self._data)

    def next(self):
        """Return the next int, advancing the cursor; raise StopIteration
        once the chunks are exhausted."""
        # Time: O(c) worst case for the trailing normalize (c = chunks).
        # Space: O(1).
        if not self.hasNext():
            raise StopIteration("no more elements")
        value = self._data[self._chunk][self._item]
        self._item += 1
        self._normalize()
        return value

    def getState(self):
        """Opaque token 'chunk:item' marking the NEXT value's position."""
        # Time: O(1), Space: O(1) - formats two ints into a short string.
        return f"{self._chunk}:{self._item}"

    @staticmethod
    def fromState(data, state):
        """Rebuild an iterator over data parked at a saved 'chunk:item'
        token, normalized to the next real value."""
        # Time: O(c) worst case for the normalize (c = chunks).
        # Space: O(1) beyond the new iterator object.
        iterator = ResumableIterator(data)
        parts = state.split(":")
        iterator._chunk = int(parts[0])
        iterator._item = int(parts[1])
        iterator._normalize()
```

```
    return iterator

if __name__ == "__main__":
    # Example 1: empty chunks contribute nothing to the sequence.
    data1 = [[7, 8, 9], [10], [], [11, 12]]
    it = ResumableIterator(data1)
    print(_drain_all(it))          # [7, 8, 9, 10, 11, 12]

    # Example 2: save state after one value, then resume from it.
    data2 = [[1, 2], [3, 4]]
    it = ResumableIterator(data2)
    print(it.next())              # 1
    state = it.getState()
    it2 = ResumableIterator.fromState(data2, state)
    print(_drain_all(it2))        # [2, 3, 4]
    print(it2.hasNext())          # False

    # Example 3: next() past the end raises StopIteration.
    it3 = ResumableIterator([[5]])
    print(it3.next())             # 5
    try:
        it3.next()
    except StopIteration as exc:
        print(str(exc))           # no more elements

    # Example 4: a sequence of only empty chunks yields nothing.
    it4 = ResumableIterator([], [], [])
    print(it4.hasNext())          # False
    print(_drain_all(it4))        # []

    # Example 5: state can be captured and restored mid-chunk.
    it5 = ResumableIterator([[1], [2, 3]])
    print(it5.next())             # 1
    print(it5.next())             # 2
    saved = it5.getState()
    revived = ResumableIterator.fromState([[1], [2, 3]], saved)
    print(_drain_all(revived))    # [3]
```

"""

Simulate a persistent, log-structured sharded key-value store. Instead of real files, each shard "file" is represented by a Python string, and the full set of shards is a list of such strings.

You are given a `shard_size` and a list of operations. Process the operations in order, maintaining both the persisted shard strings and a live in-memory key->value map, then return what the get operations saw together with the final shard contents.

What to implement

```
def solution(shard_size, operations):
    ...

    shard_size - the maximum allowed length of any single shard string.
    operations - a list of operation tuples (described below).
```

Operations

Each operation is a tuple whose first element names the operation:

- ('put', key, value) – write a put record for key, then set the current value of key to value (overwriting any existing value).
- ('get', key) – read the current value of key. Append the result to the list of get-results: the stored value if the key exists, otherwise None.
- ('delete', key) – if key currently exists, write a delete record for it and remove it from the live map. If key does not exist, do nothing (no record is written).
- ('shutdown',) – a no-op, included only for API completeness. Every write is already persisted to the shard strings, so there is nothing to flush.
- ('restore',) – discard the entire in-memory map and rebuild it by replaying the shard contents from the first shard to the last (see Restore below).

Record encoding

Each put and delete is encoded as a single semicolon-terminated record:

```
Put record: P|key|value;
Delete record: D|key;
```

The size of a record is its string length. Keys and values are ASCII (letters and digits only), so a character equals one byte, and keys/values never contain | or ;.

Sharding rule (where each new record is written)

Maintain shards as an ordered list, appended to in creation order. When a new record must be written:

- If appending it to the last shard would keep that shard's length at most `shard_size`, append it there (the last shard's new length must be $\leq$ `shard_size`).
- Otherwise (appending would make the last shard's length exceed `shard_size`), create a new shard and write the record into it.

If there are no shards yet, the first record creates the first shard. Because every individual record's encoded length is at most `shard_size`,

sharded_kv_store.py (continued)

a record always fits in a fresh shard. If no records are ever written, the shard list is empty ([]).

Restore rule (replaying shards)

On ('restore',), throw away the current in-memory map and rebuild it from scratch by scanning the shards in creation order (first to last):

- Split each shard on the ; terminator and replay each complete record in order.

- A P|key|value; record sets key to value.

- A D|key; record removes key.

- If a shard ends with an incomplete fragment that is not terminated by ;, ignore that trailing fragment.

Because records are replayed in order, the last write to a key wins, so the rebuilt map matches the live state that produced the shards.

Return value

Return a tuple (get_results, shards) where:

- get_results is the list of results from every get operation, in the order they occurred (each entry is a value string or None).

- shards is the final list of shard strings.

Constraints

- 0 <= len(operations) <= 20000

- 1 <= shard_size <= 10000

- Keys and values are non-empty ASCII strings of letters and digits only, so they never contain | or ;.

- The encoded length of every single put or delete record is at most shard_size.

Examples

Example 1

Input

```
(10, [('put', 'a', '1'), ('put', 'b', '22'), ('get', 'a'),
      ('put', 'a', '333'), ('get', 'a'), ('restore',),
      ('get', 'b')])
```

Output

```
(['1', '333', '22'], ['P|a|1;', 'P|b|22;', 'P|a|333;'])
```

Notes

Each write would overflow the current shard, so three shards are created. After restore, the latest value of 'a' is '333' and 'b' is still '22'.

Example 2

Input

```
(20, [('put', 'ab', 'x'), ('put', 'c', 'yz'), ('delete', 'ab'),
      ('get', 'ab'), ('restore',), ('get', 'c'), ('get', 'ab')])
```

Output

```
([None, 'yz', None], ['P|ab|x;P|c|yz;D|ab;'])
```

Notes

All records fit in one shard. The delete operation writes a tombstone for 'ab', so it is missing both before and after restore.

Constraints

```
0 <= len(operations) <= 20000
1 <= shard_size <= 10000
Keys and values are non-empty ASCII strings containing only letters
and digits, so they never contain '|' or ';'
The encoded length of every single put or delete record is at most
shard_size
****

# Approach (in plain terms):
#   Imagine a notebook where you only ever ADD lines to the bottom, never
#   erasing. To remember key -> value we append "P|key|value;", and to
#   forget a key we append "D|key;" (a tombstone). When the current page
#   (shard) would overflow, we start a new page. For fast reads we also
#   keep the latest value of every key in a plain map. "Restore" is like
#   losing your memory and rebuilding it: re-read every page from the first
#   to the last and apply each line in order, so the last write wins and
#   deletes take effect. A half-written line at the very end (missing its
#   ';') is skipped.
#   Data structures used:
#       - dict - the current values; answers get in O(1).
#       - list of append-only string shards - the persisted,
#         replayable write log.
class ShardedKvStore:

    def __init__(self, shard_size):
        """Start empty: remember the shard size limit, the map, and the
        shards."""
        # Time: O(1), Space: O(1) - stores the limit and two empty containers.
        self._shard_size = shard_size
        self._store = {}          # dict preserves insertion order
        self._shards = []         # each shard is a string of appended records

    def put(self, key, value):
        """put: persist a put record and set the current value."""
        # r = length of the new record, s = length of the last shard.
        # Time: O(r + s) - append to the last shard (string concat
        # rebuilds it).
        # Space: O(r + s) - the rebuilt shard string.
        self._append_record(f"P|{key}|{value};")
        self._store[key] = value

    def get(self, key):
        """get: the current value, or None if absent."""
        # Time: O(1) average, Space: O(1) - one dict lookup.
        return self._store.get(key)

    def delete(self, key):
        """delete: if the key exists, persist a tombstone and remove it."""
        # r = length of the tombstone, s = length of the last shard.
        # Time: O(r + s) when the key exists (append), else O(1).
        # Space: O(r + s).
        if key in self._store:
            self._append_record(f"D|{key};")
            del self._store[key]

    def shutdown(self):
        """shutdown: no-op in this simulation (every write is already
```

```

        persisted)."""
        # Time: O(1), Space: O(1) – does nothing; writes are already on "disk".

def restore(self):
    """restore: discard the in-memory map and rebuild it by replaying
    shards."""
    # S = total bytes across all shards,
    # k = number of live keys afterwards.
    # Time: O(S) – replay every record once. Space: O(k) – the rebuilt map.
    self._store.clear()
    for shard in self._shards:
        self._replay_shard(shard)

def _append_record(self, record):
    """Append to the last shard, or start a new shard if it would
    overflow."""
    # r = length of the record, s = length of the last shard.
    # Time: O(r + s) to concat onto a shard, or O(r) to start a fresh one.
    # Space: O(r + s) – the (possibly) rebuilt shard string.
    if (not self._shards
        or len(self._shards[-1]) + len(record) > self._shard_size):
        self._shards.append(record) # record always fits a fresh shard
    else:
        self._shards[-1] += record

def _replay_shard(self, shard):
    """Replay every complete ';'–terminated record; ignore a trailing
    fragment."""
    # L = length of the shard string.
    # Time: O(L) – walk the shard record by record. Space: O(1) extra.
    i = 0
    while i < len(shard):
        end = shard.find(";", i)
        if end < 0:
            break # incomplete trailing fragment -> ignore
        self._apply_record(shard[i:end]) # record body without the ';'
        i = end + 1

def _apply_record(self, rec):
    """Apply one decoded record body ("P|key|value" or "D|key") to the
    map."""
    # r = length of the record body.
    # Time: O(r) – slice out the key and value. Space: O(r) – the extracted
    # key/value strings.
    if rec.startswith("P|"):
        p1 = rec.index("|") # after 'P'
        p2 = rec.index("|", p1 + 1) # between key and value
        key = rec[p1 + 1:p2]
        value = rec[p2 + 1:]
        self._store[key] = value
    elif rec.startswith("D|"):
        self._store.pop(rec[2:], None)
    # any other (malformed) fragment is ignored

def _shard_strings(self):
    """Final shard contents as plain strings."""
    # t = number of shards.
    # Time: O(t), Space: O(t) – a shallow copy of the shard list.
    return list(self._shards)

```

```

class Result:
    """Result of a run: the get outputs (in order) and the final
    shard list."""

    def __init__(self, get_results, shards):
        # Time: O(1), Space: O(1) – stores two references.
        self.get_results = get_results # entries may be None
        self.shards = shards

    @staticmethod
    def simulate(shard_size, operations):
        """Run a sequence of operations and return a Result(get_results,
        shards)."""
        # m = number of ops, S = total persisted bytes, g = number of gets.
        # Time: O(m + S) worst case (a restore replays everything).
        # Space: O(S + g) – the shards plus the collected get-results.
        db = ShardedKvStore(shard_size)
        get_results = []
        for op in operations:
            kind = op[0]
            if kind == "put":
                db.put(op[1], op[2])
            elif kind == "get":
                get_results.append(db.get(op[1]))
            elif kind == "delete":
                db.delete(op[1])
            elif kind == "shutdown":
                db.shutdown()
            elif kind == "restore":
                db.restore()
            else:
                raise ValueError(f"Unknown op: {kind}")
        return ShardedKvStore.Result(get_results, db._shard_strings())

    @staticmethod
    def py_repr(result):
        """Render a Result Python-tuple style: (['1', '333'], ['P|a|1;'])."""
        # n = total size of the result.
        # Time: O(n), Space: O(n) – a repr string.
        return repr((result.get_results, result.shards))

if __name__ == "__main__":
    # Test 1: overwrite persists a new record; restore rebuilds the
    # latest values.
    shard1 = [
        ["put", "a", "1"], ["put", "b", "22"], ["get", "a"],
        ["put", "a", "333"],
        ["get", "a"], ["restore"], ["get", "b"],
    ]
    print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, shard1)))
    # (['1', '333', '22'], ['P|a|1;', 'P|b|22;', 'P|a|333;'])

    # Test 2: delete writes a tombstone; the key stays gone after restore.
    shard2 = [
        ["put", "ab", "x"], ["put", "c", "yz"], ["delete", "ab"],
        ["get", "ab"],
        ["restore"], ["get", "c"], ["get", "ab"],
    ]

```

```
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(20, shard2)))
# ([None, 'yz', None], ['P|ab|x;P|c|yz;D|ab;'])

# Test 3: empty program -> no gets and no shards.
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, [])))
# ([], [])

# Test 4: get a never-seen key -> None; deleting a missing key
# writes nothing.
shard4 = [
    ["put", "a", "1"], ["delete", "zzz"], ["get", "a"], ["get", "zzz"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard4)))
# (['1', None], ['P|a|1;'])

# Test 5: overwrite then restore keeps only the last value (both
# records kept).
shard5 = [
    ["put", "k", "v1"], ["put", "k", "v2"], ["restore"], ["get", "k"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard5)))
# (['v2'], ['P|k|v1;P|k|v2;'])

# Test 6: delete then restore -> replaying put then tombstone yields None.
shard6 = [
    ["put", "a", "1"], ["delete", "a"], ["restore"], ["get", "a"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard6)))
# ([None], ['P|a|1;D|a;'])

# Test 7: a record exactly filling a shard forces the next one onto
# a new shard.
shard7 = [
    ["put", "a", "1"], ["put", "b", "2"], ["get", "a"], ["get", "b"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(6, shard7)))
# (['1', '2'], ['P|a|1;', 'P|b|2;'])

# Test 8: many small shards; restore honors creation order (last
# write wins).
shard8 = [
    ["put", "x", "1"], ["put", "y", "2"], ["put", "x", "3"],
    ["delete", "y"],
    ["restore"], ["get", "x"], ["get", "y"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(8, shard8)))
# (['3', None], ['P|x|1;', 'P|y|2;', 'P|x|3;', 'D|y;'])
```

social_network.py

```
"""A social network with users, directed follows, and snapshots.
```

Overview:

Users are connected by directed follow edges (a follows b). At any moment createSnapshot() freezes the whole follow graph; the frozen copy never changes even as the live graph is edited afterwards.

Interface (class SocialNetwork):

- addUser(username) -> None
Register a user; a no-op if already present.
- follow(user_a, user_b) -> None
user_a starts following user_b. Auto-registers unknown users and is idempotent, so a duplicate follow is one relationship.
- unfollow(user_a, user_b) -> None
user_a stops following user_b; a no-op if not following.
- getFollowers(user) -> list
Users currently following user, sorted (empty if unknown).
- getFollowees(user) -> list
Users user currently follows, sorted (empty if unknown).
- createSnapshot() -> int
Freeze the current graph and return its id (0, 1, 2, ... in creation order).
- isFollowInSnapshot(snapshot_id, user_a, user_b) -> bool
Whether user_a followed user_b when that snapshot was taken. An out-of-range snapshot_id returns False.

Example:

```
n = SocialNetwork(); n.follow('Nina','Omar'); n.follow('Pia','Omar')
s = n.createSnapshot(); n.unfollow('Nina','Omar')
n.getFollowers('Omar') -> ['Pia']    # live graph
n.isFollowInSnapshot(s,'Nina','Omar') -> True    # frozen snapshot
"""
```

Approach (in plain terms):

- # Keep two address books: for each user, who they follow and who follows them. Using sets makes adding the same follow twice harmless and makes unfollow easy, and the reverse book lets us list a user's followers instantly. A snapshot is just a frozen photocopy of the "who follows whom" book at that instant – we copy each user's follow set into a frozenset, so later edits to the live books can never touch a stored photo. Looking up a past follow is then a quick membership check inside that photo.
- # Data structures used:
 - # - dict user -> set of followees – directed edges; the set gives O(1) follow/unfollow and automatic de-duplication.
 - # - dict user -> set of followers – reverse edges, so followers are listed without scanning everyone.
 - # - list of snapshot dicts (frozensets) – immutable copies of the graph.

class SocialNetwork:

```
def __init__(self):
    # Time: O(1), Space: O(1) – empty maps and no snapshots yet.
    self._followees = {}    # user -> set of users they follow
    self._followers = {}    # user -> set of users following them
    self._snapshots = []    # list of frozen follow graphs

def addUser(self, username):
    """Register a user (a no-op if already present)."""
    # Time: O(1) average. Space: O(1).
```

social_network.py (continued)

```
    if username not in self._followees:
        self._followees[username] = set()
        self._followers[username] = set()

def follow(self, user_a, user_b):
    """user_a starts following user_b (idempotent)."""
    # Time: O(1) average. Space: O(1).
    self.addUser(user_a)
    self.addUser(user_b)
    self._followees[user_a].add(user_b)
    self._followers[user_b].add(user_a)

def unfollow(self, user_a, user_b):
    """user_a stops following user_b (a no-op if not following)."""
    # Time: O(1) average. Space: O(1).
    if user_a in self._followees:
        self._followees[user_a].discard(user_b)
    if user_b in self._followers:
        self._followers[user_b].discard(user_a)

def getFollowers(self, user):
    """Users currently following user, sorted."""
    # f = number of followers. Time: O(f log f). Space: O(f).
    return sorted(self._followers.get(user, set()))

def getFollowees(self, user):
    """Users currently followed by user, sorted."""
    # f = number of followees. Time: O(f log f). Space: O(f).
    return sorted(self._followees.get(user, set()))

def createSnapshot(self):
    """Freeze the current follow graph; return its snapshot id."""
    # u = users, e = edges. Time: O(u + e) – copy every edge once.
    # Space: O(u + e) – the frozen copy.
    snapshot = {}
    for user in self._followees:
        snapshot[user] = frozenset(self._followees[user])
    self._snapshots.append(snapshot)
    return len(self._snapshots) - 1

def isFollowInSnapshot(self, snapshot_id, user_a, user_b):
    """Was user_a following user_b when this snapshot was taken? False if
    the snapshot id is unknown."""
    # Time: O(1) average. Space: O(1).
    if snapshot_id < 0 or snapshot_id >= len(self._snapshots):
        return False
    followees = self._snapshots[snapshot_id].get(user_a)
    return followees is not None and user_b in followees

if __name__ == "__main__":
    network = SocialNetwork()
    network.addUser("Nina")
    network.addUser("Omar")
    network.addUser("Pia")

    network.follow("Nina", "Omar")
    network.follow("Pia", "Omar")
```

social_network.py (continued)

```
snap0 = network.createSnapshot()
network.unfollow("Nina", "Omar")

print(network.getFollowers("Omar"))           # ['Pia']
print(network.isFollowInSnapshot(snap0, "Nina", "Omar")) # True
print(network.isFollowInSnapshot(snap0, "Pia", "Omar"))  # True

# The live graph reflects the unfollow; the snapshot does not.
print(network.getFollowees("Nina"))           # []
# Duplicate follow is a single relationship.
network.follow("Pia", "Omar")
print(network.getFollowers("Omar"))           # ['Pia']
# Unknown snapshot id -> False.
print(network.isFollowInSnapshot(99, "Pia", "Omar"))  # False
```

"""Friend-of-friend follow recommendations over a directed graph.

Overview:

A directed edge A → B means user A follows user B. The graph is kept as hash-based adjacency sets, so follow and unfollow are O(1) average and duplicate or self edges are ignored.

Interface (class SocialRecommendations):

- follow(follower_id, followee_id) → None
Add the edge follower → followee. A self-follow (equal ids) or a repeated follow is ignored.
- unfollow(follower_id, followee_id) → None
Remove the edge if present; otherwise a no-op.
- recommend(user_id, k) → list
Up to k suggested user ids. A candidate is anyone reachable in exactly two hops (followed by someone user_id follows) who is neither user_id nor already followed by user_id. Candidates are ranked by the number of DISTINCT intermediate users that connect user_id to them (more first); ties break by smaller id. Returns [] when k ≤ 0 or user_id follows nobody.
- process_queries(queries) @staticmethod → list
Replay ['follow'|'unfollow'|'recommend', a, b] tuples against a fresh graph and collect one entry per 'recommend' (its list). An unknown op name raises ValueError.

Example:

```
queries = [['follow',1,2], ['follow',1,3], ['follow',2,4],
           ['follow',3,4], ['recommend',1,3]]
process_queries(queries) → [[4]] # 4 is reached via both 2 and 3
"""
```

def _candidate_rank(item):

```
"""Sort key for a (candidate, intermediate_count) pair: more shared
intermediates first (higher count), then smaller candidate id first."""
# Time: O(1), Space: O(1) – unpacks a pair into a two-field tuple.
candidate, count = item
return (-count, candidate)
```

Approach (in plain terms):

```
# Picture everyone's "following" list as a page in an address book naming who
# they follow. To suggest new people for someone, we use "friends of
# friends": look at each person they already follow, then look at who THOSE
# people follow. Every time one of your follows points at the same stranger,
# that stranger earns a point – more shared connections means a better
# suggestion. We skip yourself and anyone you already follow. Finally we hand
# back the top k strangers, most points first, breaking ties by the smaller
# id so the answer is always predictable.
# Data structures used:
#   - a dict mapping user → set of followees (adjacency list) –
#     set gives O(1) follow/unfollow and automatic de-duplication.
#   - a dict counting candidate → number of mutual intermediaries
#   - ranks friend-of-friend suggestions.
```

class SocialRecommendations:

def __init__(self):

```
# Time: O(1), Space: O(1) – starts with an empty adjacency map.
# adjacency set: user → set of users they follow.
```

```

self._following = {}

def follow(self, follower_id, followee_id):
    """Add a directed edge follower -> followee. Self-follows and
    duplicates are ignored."""
    # Time: O(1) average - a dict lookup plus a set add.
    # Space: O(1) amortized - may store one new followee.
    if follower_id == followee_id:
        return # self-follow is ignored
    # setdefault + set.add naturally dedupes duplicate follows.
    self._following.setdefault(follower_id, set()).add(followee_id)

def unfollow(self, follower_id, followee_id):
    """Remove the edge follower -> followee if present; otherwise a
    no-op."""
    # Time: O(1) average - a dict lookup plus a set discard. Space: O(1).
    followees = self._following.get(follower_id)
    if followees is not None:
        followees.discard(followee_id)
    if not followees:
        self._following.pop(follower_id, None) # keep the map tidy

def recommend(self, user_id, k):
    """Up to k friend-of-friend recommendations for user_id.

    A candidate is anyone followed by someone user_id already follows,
    excluding user_id and anyone user_id already follows. Candidates are
    ranked by the count of distinct intermediates (desc), ties broken by
    smaller user_id first."""
    # d = users user_id follows, E = total second-hop edges scanned,
    # c = number of distinct candidates found (c <= E).
    # Time: O(E + c log c) - scan every second-hop edge, then sort
    #       candidates.
    # Space: O(c) - the score map and the ranked list of candidates.
    result = []
    if k <= 0:
        return result
    direct = self._following.get(user_id)
    if not direct:
        return result # follows nobody -> no friend-of-friend candidates

    # Count how many distinct intermediates reach each candidate.
    score = {}
    for intermediate in direct:
        second_hop = self._following.get(intermediate)
        if second_hop is None:
            continue
        for candidate in second_hop:
            if candidate == user_id or candidate in direct:
                continue # exclude self and already-followed users
            score[candidate] = score.get(candidate, 0) + 1

    # Collect (candidate, count) pairs so a plain named key can rank them.
    ranked = []
    for candidate in score:
        ranked.append((candidate, score[candidate]))

    # Rank: higher score first, then smaller id first.
    ranked.sort(key=_candidate_rank)

```

```

    # Take the top k candidate ids, dropping the counts.
    limit = min(k, len(ranked))
    for index in range(limit):
        result.append(ranked[index][0])
    return result

@staticmethod
def process_queries(queries):
    """Process queries in order, returning the result of every
    recommend."""
    # q = number of queries; each recommend costs up to  $O(E + c \log c)$ 
    # (see recommend), while follow/unfollow are  $O(1)$  average.
    # Time:  $O(q * (E + c \log c))$  worst case.
    # Space:  $O(V + \text{total\_edges} + q)$  - the stored graph plus the answers
    # list.
    network = SocialRecommendations()
    answers = []
    for query in queries:
        op, a, b = query[0], query[1], query[2]
        if op == "follow":
            network.follow(a, b)
        elif op == "unfollow":
            network.unfollow(a, b)
        elif op == "recommend":
            answers.append(network.recommend(a, b))
        else:
            raise ValueError(f"Unknown op: {op}")
    return answers

if __name__ == "__main__":
    # Test 1: user 1 -> {2,3}; 4 is reached by both 2 and 3 (score 2) so ranks
    # first.
    ex1 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
        ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 3],
    ]
    print(SocialRecommendations.process_queries(ex1)) # [[4, 5, 6]]

    # Test 2: self-follows/duplicates ignored; unfollow empties 1's set -> no
    # recs.
    ex2 = [
        ["follow", 1, 1], ["follow", 1, 2], ["follow", 1, 2], ["follow", 2, 1],
        ["follow", 2, 3], ["recommend", 1, 5], ["unfollow", 1, 4],
        ["unfollow", 1, 2], ["recommend", 1, 5],
    ]
    print(SocialRecommendations.process_queries(ex2)) # [[3], []]

    # Test 3: recommend for an unknown user who follows nobody -> empty list.
    ex3 = [
        ["recommend", 99, 3],
    ]
    print(SocialRecommendations.process_queries(ex3)) # [[]]

    # Test 4: k = 0 returns an empty list even when candidates exist.
    ex4 = [
        ["follow", 1, 2], ["follow", 2, 3], ["recommend", 1, 0],
    ]

```

social_recommendations.py (continued)

```
print(SocialRecommendations.process_queries(ex4)) # [[]]

# Test 5: equal scores are tie-broken by the smaller candidate id.
ex5 = [
    ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 9], ["follow", 3, 8],
    ["recommend", 1, 5],
]
print(SocialRecommendations.process_queries(ex5)) # [[8, 9]]

# Test 6: a candidate you already follow is excluded from recommendations.
ex6 = [
    ["follow", 1, 2], ["follow", 1, 3],
    ["follow", 2, 3], ["recommend", 1, 5],
]
print(SocialRecommendations.process_queries(ex6)) # [[]]

# Test 7: k smaller than the candidate count returns only the top k.
ex7 = [
    ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
    ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 2],
]
print(SocialRecommendations.process_queries(ex7)) # [[4, 5]]

# Test 8: duplicate follows don't inflate a candidate's score.
ex8 = [
    ["follow", 1, 2], ["follow", 1, 2],
    ["follow", 2, 5], ["recommend", 1, 5],
]
print(SocialRecommendations.process_queries(ex8)) # [[5]]

# Test 9: unfollowing an intermediate removes the paths it provided.
ex9 = [
    ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
    ["unfollow", 1, 3], ["recommend", 1, 5],
]
print(SocialRecommendations.process_queries(ex9)) # [[4]]
```

```
"""A spreadsheet whose cells hold integers or '+' addition formulas.
```

Overview:

A cell value is either an integer literal or a formula: a string starting with '=' that adds cell references and/or integer literals, e.g. '=A1+B1' or '=A1+5'. A reference to an unset cell counts as 0. Circular references are not an error; a cell on a cycle values as None.

Interface (two classes, both with the same camelCase API):

- SpreadsheetDFS (Part 1, lazy):
 - setCell(cell_id, value) -> None stores the parsed cell.
 - getCell(cell_id) -> int | None evaluates on demand via a depth-first walk of the formula tree; nothing is cached, so each read costs O(size of the dependency subtree). A cell reached again on the current path (a cycle) yields None.
- Spreadsheet (Part 2, eager, O(1) reads):
 - setCell(cell_id, value) -> None stores the cell, then recomputes it and its downstream dependents in topological order and caches every result.
 - getCell(cell_id) -> int | None returns the cached value in O(1) (0 if the cell was never set, None if it is cyclic).

Semantics / rules:

- A plain int or a numeric string like '5' is a constant (no refs).
- '+' is the only operator; whitespace around terms is ignored.
- Setting a cell can flip dependents into or out of a cycle; Part 2 keeps every cached value consistent after each setCell.

Example (Part 2):

```
s = Spreadsheet()
s.setCell('A1', 6); s.setCell('B1', 2)
s.setCell('A5', '=A1+B1'); s.getCell('A5')    -> 8
s.setCell('A1', 10);      s.getCell('A5')    -> 12
"""
```

```
from collections import deque
```

```
def _is_int(text):
    """True if text parses as an integer literal."""
    # Time: O(len(text)). Space: O(1).
    try:
        int(text)
        return True
    except ValueError:
        return False

def _parse_formula(formula):
    """Parse '=A1+B1+5' into (referenced cells, sum of integer literals)."""
    # f = length of the formula string.
    # Time: O(f) - one split and scan. Space: O(r) - the r references.
    refs = set()
    const = 0
    for token in formula[1:].split("+"): # formula[1:] drops the '='
        piece = token.strip()
        if piece == "":
            continue
```

spreadsheet.py (continued)

```
    if _is_int(piece):
        const += int(piece)
    else:
        refs.add(piece)
    return refs, const

def _parse_cell_value(value):
    """Normalize a raw cell value into (referenced cells, constant). A plain
    number has no references; a formula (leading '=') is parsed."""
    # Time: O(len(value)). Space: O(r) references for a formula, else O(1).
    if isinstance(value, int):
        return set(), value
    text = value.strip()
    if text.startswith("="):
        return _parse_formula(text)
    return set(), int(text) # a plain numeric string like "5"

# Approach (in plain terms):
#   Store each cell as either a number or a parsed formula (the cells it
#   references plus a constant). getCell walks the formula like a tree: to
#   value a cell, value each cell it points at (recursively) and add them up;
#   a plain number is the base case. Nothing is cached, so every read is fresh
#   but costs O(size of the dependency subtree).
#   A 'visiting' set tracks the cells on the current path; if we reach one
#   that is already on the path, the formulas loop, so we return None.
#   Data structures used:
#       - dict - maps each cell to a number or parsed formula.
#       - set - the cells on the current DFS path, for cycle
#         detection.
class SpreadsheetDFS:
    """Part 1: lazy evaluation. getCell resolves dependencies on demand with
    DFS and detects circular references (returns None for a cyclic cell)."""

    def __init__(self):
        # Time: O(1), Space: O(1) - one map of cell -> number or (refs, const).
        self._cells = {}

    def setCell(self, cell_id, value):
        """Store a number or a formula for a cell (parsed once, up front)."""
        # Time: O(f) to parse a length-f formula. Space: O(r) references.
        refs, const = _parse_cell_value(value)
        if refs:
            self._cells[cell_id] = (refs, const)
        else:
            self._cells[cell_id] = const

    def getCell(self, cell_id):
        """Evaluate a cell now via DFS; None if it lies on a cycle."""
        # V, E = cells and edges reachable from cell_id.
        # Time: O(V + E) - visits each reachable cell/edge once.
        # Space: O(V) - the recursion stack and the 'visiting' set.
        return self._eval(cell_id, set())

    def _eval(self, cell_id, visiting):
        """Recursively value a cell; None if a cycle is found on this path."""
        # Time: O(V + E) across the traversal. Space: O(V) call depth.
        entry = self._cells.get(cell_id, 0)
```

spreadsheet.py (continued)

```
    if isinstance(entry, int):
        return entry          # a plain number (or an unset cell -> 0)
    if cell_id in visiting:
        return None           # cycle: cell_id is already on the path
    refs, const = entry
    visiting.add(cell_id)
    total = const
    cyclic = False
    for ref in refs:
        value = self._eval(ref, visiting)
        if value is None:
            cyclic = True
            break
        total += value
    visiting.discard(cell_id)
    if cyclic:
        return None
    return total

# Approach (in plain terms):
# To make getCell instant, we do the work when a cell is SET, not when it is
# read. We keep two wiring diagrams: for each cell, which cells it reads (its
# formula's references) and which cells read IT (its dependents). getCell
# just returns a cached number.
# When a cell changes, only that cell and everything downstream of it can
# change, so we recompute exactly that set - in dependency order (a cell
# after all the cells it reads) using a topological sort. That order means a
# shared upstream cell is recomputed once, before any cell that needs it.
# If some cells in that set can never be ordered, they form a loop, so we
# mark them None (a circular-reference indicator).
# Data structures used:
# - dicts - forward references, constants, and reverse
#   dependents (adjacency lists).
# - dict - a value cache that makes getCell O(1).
# - deque - Kahn's topological sort queue that orders the
#   recompute.
class Spreadsheet:
    """Part 2: O(1) getCell. All work happens in setCell, which recomputes the
    changed cell and its downstream dependents in topological order and caches
    the results. Circular references are cached as None."""

    def __init__(self):
        # Time: O(1), Space: O(1) - empty maps to start.
        self._refs = {}        # cell -> set of cells it references
        self._const = {}       # cell -> constant part of its value
        self._dependents = {}   # cell -> set of cells that reference it
        self._value = {}       # cell -> cached value (None if cyclic)

    def setCell(self, cell_id, value):
        """Store a value/formula and refresh affected cached values."""
        # a = affected cells (this cell + its transitive dependents),
        # e = dependency edges among them.
        # Time: O(a + e) - rewire O(refs) plus a topological recompute.
        # Space: O(a) - the affected set and the queue.
        refs, const = _parse_cell_value(value)
        old_refs = self._refs.get(cell_id, set())
        self._rewire(cell_id, old_refs, refs)
        self._refs[cell_id] = refs
```

spreadsheet.py (continued)

```
self._const[cell_id] = const
self._recompute(cell_id)

def getCell(self, cell_id):
    """Return the cached value (0 if unset, None if part of a cycle)."""
    # Time: O(1) – a single dictionary lookup. Space: O(1).
    return self._value.get(cell_id, 0)

def _rewire(self, cell_id, old_refs, new_refs):
    """Update reverse edges: drop cell_id from old references' dependent
    sets and add it to the new ones."""
    # Time: O(|old_refs| + |new_refs|). Space: O(1) beyond the sets.
    for ref in old_refs:
        dependents = self._dependents.get(ref)
        if dependents is not None:
            dependents.discard(cell_id)
    for ref in new_refs:
        self._dependents.setdefault(ref, set()).add(cell_id)

def _collect_affected(self, start):
    """All cells whose value may change: start plus everything downstream
    (its transitive dependents)."""
    # Time: O(a + e) over a affected cells and e edges. Space: O(a).
    affected = set()
    stack = [start]
    while stack:
        cell = stack.pop()
        if cell in affected:
            continue
        affected.add(cell)
        for dependent in self._dependents.get(cell, ()):
            if dependent not in affected:
                stack.append(dependent)
    return affected

def _recompute(self, changed):
    """Recompute affected cells in dependency order; any that cannot be
    ordered form a cycle and are cached as None."""
    # a = affected cells, e = edges among them.
    # Time: O(a + e) – Kahn's topological sort. Space: O(a).
    affected = self._collect_affected(changed)
    indegree = {}
    for cell in affected:
        indegree[cell] = 0
    for cell in affected:
        for ref in self._refs.get(cell, ()):
            if ref in affected:
                indegree[ref] += 1
    ready = deque()
    for cell in affected:
        if indegree[cell] == 0:
            ready.append(cell)
    processed = 0
    while ready:
        cell = ready.popleft()
        processed += 1
        self._value[cell] = self._compute(cell)
        for dependent in self._dependents.get(cell, ()):
            if dependent in affected:
```

```

        indegree[dependent] -= 1
        if indegree[dependent] == 0:
            ready.append(dependent)
    if processed < len(affected):
        for cell in affected:
            # leftover cells form a cycle
            if indegree[cell] > 0:
                self._value[cell] = None

def _compute(self, cell):
    """Value from cached references; None if any reference is cyclic."""
    # Time: O(r) over the cell's r references. Space: O(1).
    total = self._const.get(cell, 0)
    for ref in self._refs.get(cell, ()):
        value = self._value.get(ref, 0)
        if value is None:
            return None
        total += value
    return total

if __name__ == "__main__":
    # ----- Part 2: O(1) getCell (the class the example uses) -----
    sheet = Spreadsheet()
    sheet.setCell("A1", 1)
    sheet.setCell("A2", 2)
    sheet.setCell("A3", "=A1+A2")
    sheet.setCell("A4", "=A3+A2")
    sheet.setCell("A5", "=A3+A4")
    print(sheet.getCell("A5"))    # 8    (A3=3, A4=5, A5=8)

    sheet.setCell("A1", 10)
    print(sheet.getCell("A5"))    # 26   (A3=12, A4=14, A5=26)

    print(sheet.getCell("Z9"))    # 0    (unset cell)

    sheet.setCell("B1", "=B2+1")
    sheet.setCell("B2", "=B1+1")
    print(sheet.getCell("B1"))    # None (cycle)

    sheet.setCell("B1", 5)
    print(sheet.getCell("B2"))    # 6

    # ----- Part 1: lazy DFS getCell -----
    dfs = SpreadsheetDFS()
    dfs.setCell("A1", 1)
    dfs.setCell("A2", 2)
    dfs.setCell("A3", "=A1+A2")
    dfs.setCell("A4", "=A3+A2")
    dfs.setCell("A5", "=A3+A4")
    print(dfs.getCell("A5"))      # 8
    dfs.setCell("A1", 10)
    print(dfs.getCell("A5"))      # 26

    dfs.setCell("C1", "=C2+1")
    dfs.setCell("C2", "=C1+1")
    print(dfs.getCell("C1"))      # None (cycle)

```

"""A time-based key-value store: point-in-time history lookups.

Overview:

Each key keeps its own history of values, every one stamped with a timestamp. A read asks "what value did this key hold at time T?" and returns whatever was most recently written at or before T. This is the classic "snapshot" map used for versioned config, rate quotes, or feature flags where past states must stay queryable.

Interface:

- TimeMap() -> a new, empty store.
- set(key, value, timestamp) -> None: append `value` for `key`, recorded as effective from `timestamp` onward.
- get(key, timestamp) -> value or None: the value written at the largest timestamp that is <= the query, or None if the key was never set or its earliest timestamp is still later than the query.

Semantics and rules:

- Per key, set() is called with strictly increasing timestamps, so each key's timestamp list is already sorted and supports binary search.
- get() is inclusive of an exact match: a query equal to a stored timestamp returns that entry's value.
- A query earlier than the first stored timestamp returns None; an unknown key also returns None (the two cases are indistinguishable).
- Keys are independent: each has its own separate, ordered history.

Constraints and assumptions:

- Timestamps are comparable (typically integers); values may be anything.
- Callers must honor the strictly-increasing-per-key contract; the store does not re-sort or reject out-of-order writes.

Example:

```
set("rate", "1.10", 2); set("rate", "1.12", 5)
get("rate", 1) -> None; get("rate", 4) -> "1.10"
get("rate", 9) -> "1.12"
"""
```

```
from bisect import bisect_right
```

Approach (in plain terms):

```
# Think of each key as having its own dated logbook. Every time you set a
# value you add one dated entry to that key's logbook, and because the
# dates only ever move forward the logbook is always in date order.
# To answer "what was the value at time T?" we don't read the whole
# logbook: we jump straight to the last entry whose date is on or before T
# using binary search. If the very first entry is already later than T,
# there is no answer yet and we report nothing.
```

Data structures used:

```
# - a dict from key to a sorted list of timestamps - the ordered dates
#   in each key's logbook.
# - a parallel dict from key to a list of values - the value written at
#   each of those timestamps (same position means the same entry).
# - binary search (bisect_right) - finds the newest timestamp <= the
#   query in logarithmic time.
```

```
class TimeMap:
```

```
    def __init__(self):
        # Time: O(1), Space: O(1) - starts with two empty dicts.
```

time_map.py (continued)

```
self._times = {}
self._values = {}

def set(self, key, value, timestamp):
    """Record that `key` holds `value` starting at `timestamp`."""
    # Time: O(1) amortized (two list appends), Space: O(1) per call.
    self._times.setdefault(key, []).append(timestamp)
    self._values.setdefault(key, []).append(value)

def get(self, key, timestamp):
    """Return the value stored for `key` at the largest timestamp that is
    <= `timestamp`, or None if the key has no such entry."""
    # n = number of entries stored under this key.
    # Time: O(log n) - binary search over the key's timestamps.
    # Space: O(1) - only index bookkeeping.
    times = self._times.get(key)
    if not times:
        return None
    idx = bisect_right(times, timestamp) - 1
    if idx < 0:
        return None
    return self._values[key][idx]

if __name__ == "__main__":
    tm = TimeMap()
    tm.set("exchangeRate", "1.10", 2)
    tm.set("exchangeRate", "1.12", 5)

    print(tm.get("exchangeRate", 1))    # None (before the first timestamp)
    print(tm.get("exchangeRate", 2))    # 1.10 (exact first timestamp)
    print(tm.get("exchangeRate", 4))    # 1.10 (newest ts <= 4)
    print(tm.get("exchangeRate", 5))    # 1.12 (exact second timestamp)
    print(tm.get("exchangeRate", 9))    # 1.12 (newest ts <= 9)
    print(tm.get("unknownKey", 3))      # None (key never set)

    # A second, independent key keeps its own sorted history.
    tm.set("featureFlag", "off", 10)
    tm.set("featureFlag", "on", 20)
    print(tm.get("featureFlag", 5))      # None (before first timestamp)
    print(tm.get("featureFlag", 15))     # off (newest ts <= 15)
    print(tm.get("featureFlag", 25))     # on (newest ts <= 25)
```

type_resolver.py

```
"""A tiny generic type system: bind type variables from a call's arguments.
```

Overview:

Types are modeled as small tree-shaped objects. Given a function's declared parameter types (which may contain generic placeholders) and the concrete types of an actual call, we "unify" them position by position to learn what each generic stands for, then substitute those learnings into the declared return type to get the concrete result type.

Interface:

- Primitive(name) -> a concrete named type (int, str, bool, ...).
- Generic(name) -> a type variable; by convention its name starts with an uppercase letter (T, K, V, X).
- TupleType(elements) -> an ordered list of element types.
- FunctionType(params, output) -> a signature: a list of parameter types plus one output type.
- Every type has render() -> str: a Primitive/Generic renders to its name, a TupleType to "[a, b]", a FunctionType to "[p1, p2] -> out".
- parse(spec) -> a type: a list becomes a TupleType (recursively); a string whose first character is uppercase becomes a Generic, otherwise a Primitive.
- resolve_return(function, args) -> the substituted return type, a type object whose render() gives the concrete result (e.g. "[bool, str]").

Semantics and rules:

- Matching (unification) is structural: a Primitive must meet the same Primitive; a TupleType must meet a TupleType of equal length, matched element by element.
- A Generic binds to whatever concrete type sits opposite it the first time it is seen; meeting a different type later is a conflict.
- Substitution copies the return type and swaps each bound generic for its learned type; unbound generics are left as-is.

Errors (all raise ResolutionError):

- the argument count does not match the parameter count;
- a primitive or tuple shape does not line up;
- one generic is forced to two different concrete types.

Example:

```
sig = [[K, int], V, K] -> [V, K]; call with [[str, int], bool, str]
resolve_return(sig, args).render() -> "[bool, str]"
"""
```

```
class ResolutionError(Exception):
```

```
    """Raised when a generic function call cannot be resolved."""
```

```
class Primitive:
```

```
    """A primitive type such as int or str."""
```

```
    def __init__(self, name):
```

```
        # Time: O(1), Space: O(1).
        self.name = name
```

```
    def render(self):
```

```
        # Time: O(1), Space: O(1).
        return self.name
```

```
class Generic:
    """A generic type variable such as T or K."""

    def __init__(self, name):
        # Time: O(1), Space: O(1).
        self.name = name

    def render(self):
        # Time: O(1), Space: O(1).
        return self.name

class TupleType:
    """A tuple type: an ordered list of element types."""

    def __init__(self, elements):
        # Time: O(1), Space: O(1) – keeps a reference to the element list.
        self.elements = elements

    def render(self):
        # Time: O(size of the type tree). Space: O(same) – the string.
        parts = []
        for element in self.elements:
            parts.append(element.render())
        return "[" + ", ".join(parts) + "]"

class FunctionType:
    """A function signature: parameter types plus one output type."""

    def __init__(self, params, output):
        # Time: O(1), Space: O(1).
        self.params = params
        self.output = output

    def render(self):
        # Time: O(size of the type tree). Space: O(same) – the string.
        parts = []
        for param in self.params:
            parts.append(param.render())
        return "[" + ", ".join(parts) + "] -> " + self.output.render()

def parse(spec):
    """Build a type from a nested spec: a list -> TupleType, an uppercase-first
    string -> Generic, any other string -> Primitive."""
    # Time: O(size of spec). Space: O(size of spec) – the built type.
    if isinstance(spec, list):
        elements = []
        for item in spec:
            elements.append(parse(item))
        return TupleType(elements)
    if spec[:1].isupper():
        return Generic(spec)
    return Primitive(spec)

def _same_type(left, right):
```

type_resolver.py (continued)

```
"""True if two types have identical structure."""
# Time: O(size). Space: O(depth) – the recursion.
if isinstance(left, Primitive) and isinstance(right, Primitive):
    return left.name == right.name
if isinstance(left, Generic) and isinstance(right, Generic):
    return left.name == right.name
if isinstance(left, TupleType) and isinstance(right, TupleType):
    if len(left.elements) != len(right.elements):
        return False
    for i in range(len(left.elements)):
        if not _same_type(left.elements[i], right.elements[i]):
            return False
    return True
if isinstance(left, FunctionType) and isinstance(right, FunctionType):
    if len(left.params) != len(right.params):
        return False
    for i in range(len(left.params)):
        if not _same_type(left.params[i], right.params[i]):
            return False
    return _same_type(left.output, right.output)
return False

def _unify(declared, concrete, mapping):
    """Match a declared type against a concrete one, recording generic →
    concrete bindings in mapping. Raises ResolutionError on any mismatch."""
    # Time: O(size). Space: O(depth) – the recursion.
    if isinstance(declared, Generic):
        bound = mapping.get(declared.name)
        if bound is None:
            mapping[declared.name] = concrete
        elif not _same_type(bound, concrete):
            raise ResolutionError(f"conflicting types for {declared.name}")
        return
    if isinstance(declared, Primitive):
        if (not isinstance(concrete, Primitive)
            or concrete.name != declared.name):
            raise ResolutionError("primitive/type-shape mismatch")
        return
    if isinstance(declared, TupleType):
        if (not isinstance(concrete, TupleType)
            or len(concrete.elements) != len(declared.elements)):
            raise ResolutionError("tuple structure mismatch")
        for i in range(len(declared.elements)):
            _unify(declared.elements[i], concrete.elements[i], mapping)
        return
    raise ResolutionError("cannot match this type form")

def _substitute(declared, mapping):
    """Replace every bound generic in declared with its concrete type."""
    # Time: O(size). Space: O(depth) – the recursion.
    if isinstance(declared, Generic):
        bound = mapping.get(declared.name)
        return bound if bound is not None else declared
    if isinstance(declared, Primitive):
        return declared
    if isinstance(declared, TupleType):
        new_elements = []
```

type_resolver.py (continued)

```
    for element in declared.elements:
        new_elements.append(_substitute(element, mapping))
    return TupleType(new_elements)
if isinstance(declared, FunctionType):
    new_params = []
    for param in declared.params:
        new_params.append(_substitute(param, mapping))
    return FunctionType(new_params, _substitute(declared.output, mapping))
return declared

# Approach (in plain terms):
# Think of each type as a little tree: a leaf is a primitive (int) or a
# generic placeholder (T), and a branch is a tuple/function holding more
# types. To resolve a call we walk the declared parameter tree and the
# concrete argument tree together, position by position. A primitive must
# match the same primitive; a tuple must match a tuple of the same shape; a
# generic "learns" whatever concrete type sits opposite it – and if it later
# meets a different type, that is a conflict and we reject the call. Once
# every generic knows its type, we copy the declared return tree and swap
# each generic for what it learned.
# Data structures used:
# - small classes (Primitive/Generic/TupleType/FunctionType) – a tagged
#   tree representing each type form.
# - dict generic name -> concrete type – the bindings learned while
#   matching; a repeat with a different type means a conflict.
# - recursion – matches and substitutes the nested type trees.
def resolve_return(function, args):
    """Concrete return type of function when called with args. Binds each
    generic by matching declared params to concrete args, then fills those
    bindings into the return type. Raises ResolutionError on invalid calls."""
    # s = total size of the types involved.
    # Time: O(s) – one recursive match, then one recursive substitution.
    # Space: O(number of generics + recursion depth).
    if len(args) != len(function.params):
        raise ResolutionError("wrong number of arguments")
    mapping = {}
    for i in range(len(function.params)):
        _unify(function.params[i], args[i], mapping)
    return _substitute(function.output, mapping)

if __name__ == "__main__":
    # Example 1: [[K, int], V, K] -> [V, K] with args [[str, int], bool, str]
    signature = FunctionType(
        [parse(["K", "int"]), parse("V"), parse("K")],
        parse(["V", "K"]),
    )
    print(signature.render())      # [[K, int], V, K] -> [V, K]
    call_args = [parse(["str", "int"]), parse("bool"), parse("str")]
    print(resolve_return(signature, call_args).render())  # [bool, str]

    # Example 2: [X, [X, bool]] -> X with [int, [str, bool]] -> conflict
    conflicting = FunctionType(
        [parse("X"), parse(["X", "bool"])], parse("X"))
    try:
        resolve_return(conflicting, [parse("int"), parse(["str", "bool"])])
    except ResolutionError as err:
        print("error:", err)      # error: conflicting types for X
```

```
# Wrong number of arguments -> rejected
try:
    resolve_return(conflicting, [parse("int")])
except ResolutionError as err:
    print("error:", err)      # error: wrong number of arguments

# Primitive mismatch -> rejected
identity = FunctionType([parse("int")], parse("int"))
try:
    resolve_return(identity, [parse("str")])
except ResolutionError as err:
    print("error:", err)      # error: primitive/type-shape mismatch
```

workflow_stages.py

```
"""Group a workflow's jobs into execution stages by dependency level.
```

Overview:

A workflow is a set of named jobs, some of which must wait for others to finish. Partition the jobs into ordered stages such that every job's prerequisites all live in earlier stages; jobs sharing a stage have no dependency between them and can run in parallel. This is a level-by-level topological sort (Kahn's algorithm).

Interface:

```
workflow_stages(jobs) -> list[list[str]]
    jobs is a dict mapping job name -> config dict. A config may hold a
    "needs" key whose value is a list of job names that must finish first
    (a missing or empty "needs" means no prerequisites). Returns a list of
    stages, each a list of job names sorted alphabetically for
    deterministic output.
```

Semantics / rules:

- Stage 0 holds every job with no prerequisites; each later stage holds the jobs whose prerequisites all completed in earlier stages.
- An empty workflow returns an empty list.
- The input is assumed to be a valid DAG (every name in a "needs" list is a real job and there are no cycles); cycles are not detected.

Example:

```
workflow_stages({
    "compile": {}, "lint": {},
    "test": {"needs": ["compile", "lint"]},
    "package": {"needs": ["test"]}, "publish": {"needs": ["test"]},
}) -> [['compile', 'lint'], ['test'], ['package', 'publish']]
"""
```

Approach (in plain terms):

```
# This is a "who can go next" scheduling problem. For each job we count how
# many prerequisites it still waits on (its in-degree) and remember which
# jobs are waiting on it (its dependents). Any job waiting on nobody can run
# now - those form the first stage. We then finish that whole stage at once:
# for every job we just ran, tick down its dependents' counters; any that
# reach zero become the next stage. Repeat until no jobs remain. Each stage
# is sorted so the output is deterministic. (This is a level-by-level
# topological sort, i.e. Kahn's algorithm.)
# Data structures used:
# - dict in-degree (job -> unmet prerequisite count) - decremented as
#   earlier stages finish; hitting 0 means a job is ready.
# - dict of lists dependents (job -> jobs that need it) - a reverse
#   adjacency list used to release the next stage.
# - list of stages (each a sorted list) - the result, one per level.
```

```
def workflow_stages(jobs):
    """Return jobs grouped into parallel-executable stages (sorted per stage).
    Jobs in later stages depend on jobs from earlier ones."""
    # j = number of jobs, e = number of dependency edges.
    # Time: O(j log j + e) - each stage sorts its jobs; each edge relaxed once.
    # Space: O(j + e) - the in-degree map and dependents lists.
    indegree = {}
    dependents = {}
    for job in jobs:
        indegree[job] = 0
        dependents[job] = []
```

workflow_stages.py (continued)

```
for job in jobs:
    for need in jobs[job].get("needs", []):
        indegree[job] += 1          # one more prerequisite for job
        dependents[need].append(job) # 'need' must finish before 'job'

# First stage: every job with no unmet prerequisites.
current = []
for job in jobs:
    if indegree[job] == 0:
        current.append(job)
current.sort()

stages = []
while current:
    stages.append(current)
    next_stage = []
    for job in current:
        for dependent in dependents[job]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                next_stage.append(dependent)
    next_stage.sort()
    current = next_stage
return stages

if __name__ == "__main__":
    # ----- Example from the problem statement -----
    jobs = {
        "compile": {},
        "lint": {},
        "test": {"needs": ["compile", "lint"]},
        "package": {"needs": ["test"]},
        "publish": {"needs": ["test"]},
    }
    print(workflow_stages(jobs))
    # [['compile', 'lint'], ['test'], ['package', 'publish']]

    # ----- No dependencies -> one stage with everything (sorted) -----
    print(workflow_stages({"a": {}, "c": {}, "b": {}}))
    # [['a', 'b', 'c']]

    # ----- A straight chain a -> b -> c -> d -----
    chain = {
        "a": {},
        "b": {"needs": ["a"]},
        "c": {"needs": ["b"]},
        "d": {"needs": ["c"]},
    }
    print(workflow_stages(chain))
    # [['a'], ['b'], ['c'], ['d']]

    # ----- Empty workflow -> no stages -----
    print(workflow_stages({}))
    # []
```